Etymology + Data Science

A Computational Survey of Word Origins

Troy Altus

2026-05-01

Table of contents

Chapter 1

What Is Etymology? A Data Perspective

Learning Objectives

By the end of this chapter, you will be able to:

- Define etymology and explain what distinguishes it from ordinary vocabulary study
- Describe the comparative method in plain English
- Identify cognates in a wordlist using pattern recognition
- Frame the five core problems of computational etymology as data science tasks
- Write basic Python code to explore a multilingual wordlist

1.1 The Detective Problem

In 1786, a British judge named William Jones stood up at the Asiatic Society in Calcutta and delivered what may be the most consequential sentence ever spoken at an academic dinner. Having spent years studying Sanskrit—the ancient liturgical language of India—he announced that Sanskrit bore "a stronger affinity" to Greek and Latin "than could possibly have been produced by accident." So strong was this affinity, he suggested, that all three languages must have descended from a common source.

The sentence launched a century of scholarship. Within fifty years, scholars across Europe had worked out the main skeleton of the Indo-European language family: Sanskrit, Greek, Latin, Gothic, Old Persian, Lithuanian, Old Slavic—all cousins, all descended from a proto-language no one had ever written down, spoken by people who left no other records. The method they used was the comparative method, and it remains, in its essentials, the foundation of etymology today.

What Jones had noticed—what the Sanskrit word *pitar*, the Greek *pat r*, and the Latin *pater* share—is something more than a family resemblance. The sounds correspond *systematically*. The *p* at the start, the vowel, the *t* in the middle, the *r* at the end: these are not random coincidences. They are the fingerprints of a common ancestor.

That is what etymology studies. Not just where words come from, but *why they look the way*

they do, and how you can infer the answer from patterns that survive across languages.

1.2 The Comparative Method in Plain English

The comparative method can be summarized in three steps.

Step 1: Gather cognates. A *cognate* is a pair of words in different languages that descend from the same ancestral word. English *night* and German *Nacht* are cognates. English *night* and French *nuit* are also cognates—both descend from the Latin *nox*, which descends further from the Proto-Indo-European root **nók ts*. Cognates are not the same as loanwords: English *beef* comes from French *bœuf*, but that is borrowing, not shared descent. The distinction matters.

Step 2: Identify sound correspondences. When you line up cognates across multiple languages, patterns emerge. Latin *p* systematically corresponds to English *f* in the same word positions. Latin *pater*, English *father*; Latin *piscis*, English *fish*; Latin *pes/ped-*, English *foot*. This is Grimm's Law—a systematic shift of stops to fricatives that swept through the Germanic languages roughly two thousand years ago, like a wave passing through water.

Step 3: Reconstruct the proto-form. Given enough cognates and enough correspondences, you can infer what the ancestral form must have looked like. The asterisk (*) marks reconstructed forms—words no one ever recorded, deduced entirely from their surviving descendants. The fact that such deductions can be verified against actual discoveries (when ancient texts surface) is what makes the comparative method science rather than educated guessing.

1.3 Etymology as Data

Here is where things get interesting.

Notice what the comparative method is actually doing. It is taking a set of strings—words, transcribed as sequences of sounds—and detecting patterns of similarity and difference across them. It is grouping those strings by hypothesized ancestry. It is inferring hidden variables (the proto-forms) from observable ones (the modern words). It is, in short, doing data science on a structured dataset.

The dataset in question is a *wordlist*: a table where rows are concepts (FATHER, WATER, NIGHT, FIRE) and columns are languages, with cells containing the word for that concept in that language. This is the fundamental data structure of historical linguistics, and it is, conveniently, exactly the kind of tabular data that Python's pandas library was designed to handle.

```
data = {
    "Concept":  ["father",  "water",  "night",  "fire",   "star"],
    "Latin":    ["pater",   "aqua",   "nox",    "ignis",  "stella"],
    "Greek":    ["pat r",   "hydōr",  "nýks",   "pŷr",    "ast r"],
    "Sanskrit": ["pitar",   "udaka",  "nakta",  "agni",   "tara"],
    "English":  ["father",  "water",  "night",  "fire",   "star"],
    "PIE*":     ["*ph t r", "*wed-",  "*nók ts","*h ng ni","*h st r"],
}
```

```
df = pd.DataFrame(data)
df
```

Table 1.1: A minimal wordlist showing Indo-European cognates. The asterisk marks a reconstructed proto-form.

	Concept	Latin	Greek	Sanskrit	English	PIE*
0	father	pater	pat r	pitar	father	*ph t r
1	water	aqua	hydōr	udaka	water	*wed-
2	night	nox	nýks	nakta	night	*nók ts
3	fire	ignis	pŷr	agni	fire	*h ng ni
4	star	stella	ast r	tara	star	*h st r

Look at the column for FATHER. Every entry except FIRE shares a recognizable similarity—the *p/f* at the start, the vowel in the middle, the *t/th* sound, the final *r*. FIRE is the odd one out: the Latin, Greek, and Sanskrit forms look nothing like each other, which suggests different roots, not a shared ancestor for this particular word.

This is already a data analysis task: spotting which cells cluster with which other cells, and which are outliers. A trained linguist can do it by eye. A computer can do it at scale, across a hundred languages and ten thousand concepts, in seconds.

1.4 The Five Core Problems

Computational etymology is organized around five problems, each of which maps cleanly onto a class of data science methods:

Problem 1: Cognate detection. Given a pair of words from different languages, are they related by descent? This is a binary classification problem. Input: two word strings (or sequences of phonemes). Output: cognate or not. We will spend Chapter 5 building a classifier for this.

Problem 2: Sequence alignment. When two cognates are identified, how do we align their sounds to identify which phoneme corresponds to which? English *night* [na t] and German *Nacht* [naxt]—the *n* matches *n*, the vowel has drifted, the final *t* matches *t*. But how do you determine that alignment algorithmically, especially when one word has more sounds than the other? This is a sequence alignment problem, borrowed directly from bioinformatics. Chapter 4.

Problem 3: Phylogenetic inference. Given a set of languages and their cognate relationships, what is the most likely family tree? This is a tree inference problem. The same algorithms that evolutionary biologists use to reconstruct the evolutionary tree of species apply, with modifications, to languages. Chapter 6.

Problem 4: Proto-form reconstruction. Given aligned cognates across multiple modern languages, can we predict the ancestral proto-form? This is a sequence-to-sequence prediction problem—the kind that neural networks handle well. Chapter 7.

Problem 5: Semantic drift. Words change meaning over time. “Awful” once meant awe-inspiring; now it means terrible. “Nice” once meant foolish; now it means pleasant. Can we

detect and quantify this drift using word embeddings trained on historical text corpora? Chapter 8.

1.5 Why Machine Learning Changes Everything

The traditional comparative method works beautifully—for languages with large amounts of recorded history, spoken by cultures that kept written records, studied by teams of dedicated scholars over a century or more. But the world has approximately seven thousand living languages. Only a few hundred have been studied in any depth. Thousands more are vanishing without documentation.

Machine learning does not replace the comparative method. It scales it. A cognate detector trained on well-studied language families can be applied to underdocumented ones. A neural proto-reconstructor trained on Romance language data generalizes to other families with appropriate retraining. The algorithms are not magic—they carry assumptions baked in from their training data—but they extend the reach of historical linguistics into territory that hand methods alone could never cover.

The reverse transfer also runs. Etymology turns out to be an exceptionally good training ground for linguistic data science. The problems are well-defined, the ground truth is knowable, and the data has a satisfying structure. Concepts learned here—sequence alignment, phylogenetic inference, distributional semantics—transfer directly to speech recognition, machine translation, and clinical NLP.

1.6 Your First Wordlist

Let us build the first practical object of this book: a multilingual wordlist in Python.

```
from itertools import combinations

def simple_edit_distance(s1, s2):
    """Levenshtein distance between two strings."""
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

wordlist = {
    "father":  {"Latin": "pater",  "Greek": "pater",  "Sanskrit": "pitar",  "German": "vater"}
```

```
    "water":   {"Latin": "aqua",   "Greek": "hydor",  "Sanskrit": "udaka",  "German": "wass
    "night":   {"Latin": "noctem", "Greek": "nykta",  "Sanskrit": "nakta",  "German": "nach
    "new":     {"Latin": "novum",  "Greek": "neon",   "Sanskrit": "navam",  "German": "neu"
    "star":    {"Latin": "stella", "Greek": "aster",  "Sanskrit": "tara",   "German": "ster
}

langs = ["Latin", "Greek", "Sanskrit", "German"]
concepts = list(wordlist.keys())
english = {"father": "father", "water": "water", "night": "night",
           "new": "new", "star": "star"}

dist_matrix = np.zeros((len(concepts), len(langs)))
for i, concept in enumerate(concepts):
    eng = english[concept]
    for j, lang in enumerate(langs):
        foreign = wordlist[concept][lang]
        dist_matrix[i, j] = simple_edit_distance(eng.lower(), foreign.lower())

fig, ax = plt.subplots(figsize=(8, 4))
im = ax.imshow(dist_matrix, cmap="YlOrRd_r", aspect="auto")
ax.set_xticks(range(len(langs)))
ax.set_yticks(range(len(concepts)))
ax.set_xticklabels(langs)
ax.set_yticklabels(concepts)
for i in range(len(concepts)):
    for j in range(len(langs)):
        ax.text(j, i, f"{dist_matrix[i,j]:.0f}", ha="center", va="center",
                color="black", fontsize=10)
ax.set_title("Edit Distance from English (lower = more similar)")
plt.colorbar(im, ax=ax, label="Edit distance")
plt.tight_layout()
plt.show()
```

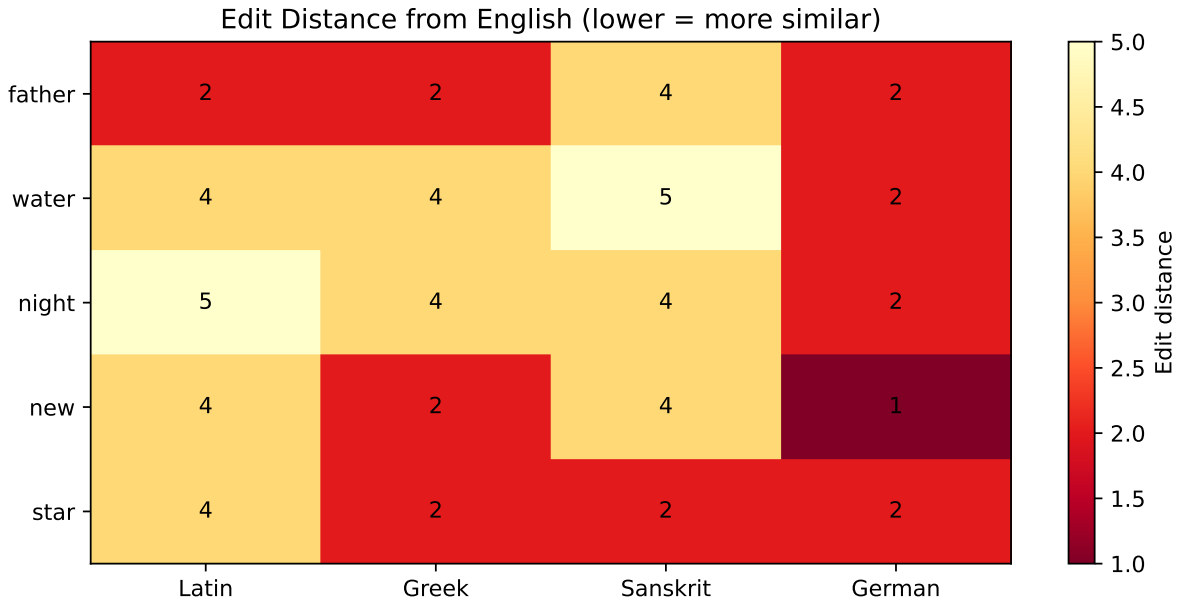

Figure 1.1: Edit distance between English words and their cognates across five languages. Darker = more similar (smaller distance). Note that WATER shows less similarity across languages because different roots were selected.

The heatmap already tells a story. FATHER and NEW show consistently low edit distances—the words are recognizably similar across all four comparison languages. NIGHT is a medium case. WATER is the outlier: *aqua* and *hydor* look nothing like the English word because the Germanic languages preserved one Indo-European root (**wed-*) while Latin and Greek preserve different ones.

This is, in miniature, exactly the pattern-detection that the comparative method does at scale. And this is, in miniature, the kind of exploratory analysis you will do throughout this book—start with the data, visualize it, let the pattern lead you to the hypothesis.

1.7 Summary

Etymology studies the origin and historical development of words. The comparative method—identifying cognates, mapping sound correspondences, and reconstructing proto-forms—has been the core tool of historical linguistics for two centuries. Computational approaches do not replace this method; they automate and scale it. The five core problems (cognate detection, sequence alignment, phylogenetic inference, proto-form reconstruction, and semantic drift tracking) each map onto established data science techniques. The data structure underlying all of this is the multilingual wordlist: a table of concepts by languages, with each cell containing a phonetically transcribed word.

The remaining chapters build these tools, one layer at a time.

1.8 Further Reading

- Jones, W. (1786). *The third anniversary discourse to the Asiatic Society.* The speech that started it all.
- Campbell, L. (2013). *Historical Linguistics: An Introduction* (3rd ed.). MIT Press. The standard textbook reference for the comparative method.
- List, J.-M., Greenhill, S., & Gray, R. (2017). The potential of automatic word comparison for historical linguistics. *PLOS ONE*. A good overview of the computational turn in the field.
- Etymonline.com — for exploratory etymological browsing, nothing beats it.

Chapter 2

Preface

Preface

Why This Book Exists

There is a game that every student of Latin is taught to play, usually around the third week of class, right after the paradigm tables start to blur. The game is called "spot the cousin." You take the Latin word *pater* and the English word *father*, and you notice that the *p* has become an *f* and the *t* has become a *th*. Then you do it again: Latin *piscis*, English *fish*. Latin *ped-*, English *foot*. Suddenly, without knowing quite when it happened, you are no longer studying vocabulary. You are watching a language drift across a thousand years like a continent, grinding against its neighbors, shedding consonants like scale.

Most people who discover this game are content to play it by hand, one word at a time. This book is for the people who look at that game and ask: could we automate it?

The answer, it turns out, is substantially yes—and the story of how computational linguists figured that out in the last twenty years is one of the more quietly remarkable episodes in the history of data science. It involves borrowed algorithms from molecular biology, machine learning models that beat expert linguists at their own task, and neural networks that have reconstructed words spoken by people who died before writing was invented. It is the kind of story that sounds like science fiction until you actually run the code.

What You Will Find Here

This book is organized as twelve chapters that move from concept to technique to application. The first three chapters build up linguistic intuition: what etymology actually studies, how traditional linguists work, and how to translate language into a form a computer can use. Chapters four through six introduce the core algorithms—sequence alignment borrowed from bioinformatics, machine learning for detecting word kinship, and phylogenetic inference for building family trees of languages. Chapters seven through eleven apply these tools to harder problems: reconstructing dead proto-languages with neural networks, tracking how word meanings drift over centuries, detecting when languages have borrowed from each other, and building queryable knowledge graphs of etymological relationships. Chapter twelve is a capstone in which you design and execute your own research question.

Each chapter begins with a concrete example—a word, a puzzle, a historical accident—before generalizing to the method. Code appears throughout, written to be run rather than merely read. Every chapter closes with exercises and a further reading list for the curious.

What You Will Need

Linguistically: nothing. We build the relevant concepts from scratch.

Computationally: basic Python—enough to know what a loop is and how to index a list. Everything else is introduced when it is needed.

Mathematically: high school algebra. The one section that involves linear algebra (Chapter 7, on neural networks) explains what it needs as it goes.

What you do need is a tolerance for the slightly unsettling feeling that arrives when you realize that a computer can do in three seconds something that took a nineteenth-century philologist three years. That feeling is not a sign that something has gone wrong. It is a sign that you are paying attention.

A Note on Style

The linguist J.E. Gordon once opened a chapter on the mechanics of elasticity with a discussion of worms. He was not being whimsical; he was being pedagogically shrewd. Abstract principles stick when they are first encountered in the flesh. This book tries to do the same: every major concept is introduced through a specific word, a specific language pair, a specific historical moment. The abstraction comes after.

Wherever equations appear, they appear because they compress something important, not because they signal that the material is serious. The material is serious with or without them.

Troy Altus May 2026

Part I

Part I: Foundations

Chapter 3

The Comparative Method

Learning Objectives

By the end of this chapter, you will be able to:

- Apply the comparative method step-by-step to a set of cognates
- Identify sound correspondences from parallel data
- Explain what makes a sound change "regular" and why regularity matters
- Reconstruct a proto-phoneme from its modern reflexes
- Recognize the limitations and failure modes of the traditional approach
- Visualize sound change correspondences as a mapping table

3.1 The Crime Scene

Imagine arriving at an archaeological dig where the only surviving artifacts are the words. No pottery, no bones, no inscriptions—just the vocabulary of a dozen languages that happened to be related, preserved in the mouths of their modern speakers. Your job is to determine: what language did they all descend from? What did that language sound like? When did the various descendants begin to diverge?

This is not a metaphor. It is precisely the problem that faced the German philologist Jakob Grimm in the early nineteenth century, and the solution he devised—which now bears his name as Grimm's Law—was the moment historical linguistics became a science.

Grimm noticed something that had been sitting in plain sight for anyone who cared to look. The Germanic languages—Gothic, Old English, Old High German, Old Norse—had systematically different consonants from Latin, Greek, and Sanskrit in corresponding words. Not randomly different. *Systematically* different, with a consistent mapping:

- Latin $p \rightarrow$ Germanic f (Latin *pater*, Gothic *fadar*)
- Latin $t \rightarrow$ Germanic *th* (Latin *tres*, English *three*)
- Latin $k \rightarrow$ Germanic h (Latin *cornu*, English *horn*)

This was not accident. This was sound change: a systematic shift in how an entire population pronounced a class of consonants, spreading through a speech community over generations like a slow-moving wave.

The comparative method is the technique for reading these waves backward—tracing from the modern shoreline back to the stone that fell in the water.

3.2 Step-by-Step: The Four Operations

The comparative method proceeds in four operations, applied iteratively.

3.2.1 1. Collect a Wordlist of Basic Vocabulary

The first step is deliberate conservatism. Basic vocabulary—words for body parts, numerals, kinship terms, common natural phenomena—changes more slowly than specialized vocabulary. You do not start with words for "television" or "algorithm." You start with words for MOTHER, HAND, THREE, WATER, FIRE.

The classic standard is the Swadesh list: 100 or 207 concepts selected by Morris Swadesh in the 1950s for their cross-linguistic stability. We will use a reduced version for illustration.

```
swadesh_romance = {
    "Concept":   ["one", "two", "water", "fire", "hand", "eye", "name", "mouth", "night", "new"
    "Latin":     ["unus", "duo", "aqua", "ignis", "manus", "oculus", "nomen", "os/oris", "nox"
    "Spanish":   ["uno", "dos", "agua", "fuego", "mano", "ojo", "nombre", "boca", "noche", "nu
    "French":    ["un", "deux", "eau", "feu", "main", "oeil", "nom", "bouche", "nuit", "nouveau
    "Italian":   ["uno", "due", "acqua", "fuoco", "mano", "occhio", "nome", "bocca", "notte",
    "Portuguese":["um", "dois", "água", "fogo", "mão", "olho", "nome", "boca", "noite", "novo"]
}

df = pd.DataFrame(swadesh_romance)
df
```

Table 3.1: A mini-Swadesh list: ten basic concepts across five Romance languages plus their Latin ancestor.

	Concept	Latin	Spanish	French	Italian	Portuguese
0	one	unus	uno	un	uno	um
1	two	duo	dos	deux	due	dois
2	water	aqua	agua	eau	acqua	água
3	fire	ignis	fuego	feu	fuoco	fogo
4	hand	manus	mano	main	mano	mão
5	eye	oculus	ojo	oeil	occhio	olho
6	name	nomen	nombre	nom	nome	nome
7	mouth	os/oris	boca	bouche	bocca	boca
8	night	nox	noche	nuit	notte	noite
9	new	novus	nuevo	nouveau	nuovo	novo

3.2.2 2. Identify Cognate Sets

A *cognate set* is a group of words—one per language—that all descend from the same proto-word. In the table above, the words for WATER are not all cognates: Spanish *agua* and Italian *acqua*

trace to Latin *aqua*, but French *eau* traces to the same Latin word via a drastically eroded path (Latin *aqua* → Old French *ewe* → French *eau*). They are still cognates—just with more weathering on one of them.

The test for cognacy is not similarity. It is *systematic* similarity, explained by known or discoverable sound changes. English *night* and Greek *nýks* look very different, but they are cognates; English *day* and Latin *dies* look superficially similar, but the etymology is more complicated. Surface similarity is a clue, not a verdict.

3.2.3 3. Establish Sound Correspondences

This is the analytical core of the method. Take all your cognate pairs and line up which sound in language A corresponds to which sound in language B.

```
correspondences = {
    "Latin": ["p", "t", "k", "f", "n", "m", "s", "l", "r", "v"],
    "Spanish": ["p", "t", "k/g", "h/f", "n", "m", "s", "l", "r", "b"],
    "Example": [
        "pater→padre", "terra→tierra", "caput→cabo",
        "filiu→hijo", "nomen→nombre", "manus→mano",
        "sole→sol", "luna→luna", "rivus→río", "vitam→vida"
    ]
}

fig, ax = plt.subplots(figsize=(9, 5))
ax.axis("off")
table = ax.table(
    cellText=list(zip(correspondences["Latin"], correspondences["Spanish"], correspondences
    colLabels=["Latin consonant", "Spanish reflex", "Example"],
    cellLoc="center",
    loc="center",
)
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.2, 1.6)
for (row, col), cell in table.get_celld().items():
    if row == 0:
        cell.set_facecolor("#dde")
ax.set_title("Latin → Spanish Consonant Correspondences (selected)", pad=20)
plt.tight_layout()
plt.show()
```

Latin → Spanish Consonant Correspondences (selected)

Latin consonant	Spanish reflex	Example
p	p	pater→padre
t	t	terra→tierra
k	k/g	caput→cabo
f	h/f	filiu→hijo
n	n	nomen→nombre
m	m	manus→mano
s	s	sole→sol
l	l	luna→luna
r	r	rivus→río
v	b	vitam→vida

Figure 3.1: Sound correspondence table for Latin vs. Spanish in word-initial position. Arrows indicate regular correspondences.

The key insight is *regularity*. If Latin *f* becomes Spanish *h* in one word (Latin *filiu* → Spanish *hijo*, "son"), it must become *h* in every word where the same conditions hold—otherwise the change is not a law, it is an accident. Jakob Grimm's insight was that the Germanic consonant shifts were regular in exactly this sense: every instance of Latin *p* in the right position became Germanic *f*, without exception.

When you find exceptions, you do not dismiss them. They are data. Either they reflect borrowing from another language (which skips the sound change), or they reflect a conditioned environment where the change behaved differently, or they reveal an error in your cognate identification. Exceptions are gifts.

3.2.4 4. Reconstruct the Proto-Form

Given the correspondences, you can work backward to what the ancestral sound must have been. If Spanish has *p*, French has *p*, Italian has *p*, and Latin has *p*, the proto-Romance form almost certainly had *p*. If one language has *h* where the others have *f*, the *h* is the locally innovated form; *f* is older.

The reconstruction is not a guess. It is the most parsimonious hypothesis that explains all the observed reflexes. Linguists write reconstructed forms with an asterisk: **pater* for the Proto-Indo-European form of FATHER.

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.set_xlim(0, 10)
ax.set_ylim(0, 8)
ax.axis("off")
```

```
# Proto-form at top
ax.text(5, 7, "*NOCTEM (Latin)", ha="center", va="center",
        fontsize=13, fontweight="bold",
        bbox=dict(boxstyle="round,pad=0.4", facecolor="#cde", edgecolor="#888"))

# Modern descendants
descendants = [
    (1.5, 2, "Spanish\nnoche"),
    (3.5, 2, "French\nnuit"),
    (5.5, 2, "Italian\nnotte"),
    (7.5, 2, "Portuguese\nnoite"),
    (9.5, 2, "Romanian\nnoapte"),
]

for x, y, label in descendants:
    ax.text(x, y, label, ha="center", va="center", fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3", facecolor="#eef", edgecolor="#888"))
    ax.annotate("", xy=(x, y + 0.5), xytext=(5, 6.5),
                arrowprops=dict(arrowstyle="->", color="#666", lw=1.5))

ax.set_title("Descent of NIGHT from Latin to Romance languages", fontsize=12, pad=10)
plt.tight_layout()
plt.show()
```

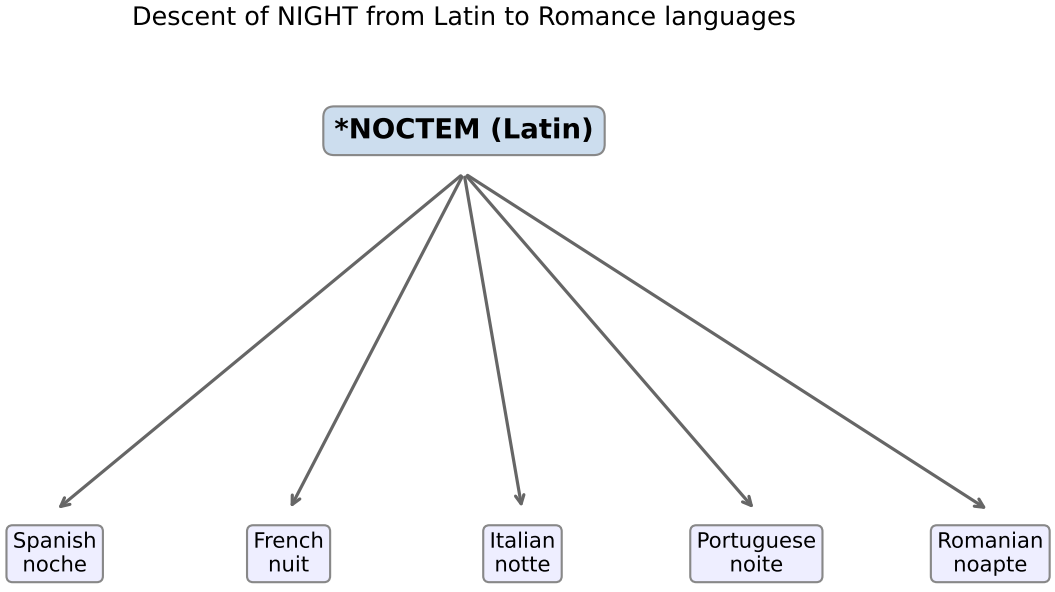

Figure 3.2: Reconstructing the Proto-Romance word for NIGHT from its modern reflexes. The Latin *noctem* (accusative of *nox*) is the direct ancestor.

3.3 Why Regularity Is Not Obvious

It is worth pausing on the claim of regularity, because it is not something you would expect if you had not seen it demonstrated.

Languages are spoken by millions of people across a geographic area and a span of time. Sound changes spread like any other social behavior—not simultaneously, not uniformly, with geographic and social variation. Why would a change be *regular*, affecting every instance of a phoneme in the same environment?

The answer, proposed by the Neogrammarians in the 1870s and still essentially correct, is that sound change operates on the physical mechanism of speech production, not on the meaning of words. A shift from *p* to *f* is a change in how the mouth moves—from a full stop to a fricative—not a lexical decision. The mouth changes its habit. Every word that used that motion changes with it.

This is why exceptions tend to cluster: words borrowed from another language arrive after the change, so they do not carry its effects. Words used in very formal or religious contexts may be pronounced conservatively. These patterns are not noise; they are diagnostic.

3.4 The Limits of the Method

The comparative method is powerful, but it has a horizon.

Time depth. Sound changes accumulate. After roughly 8,000–10,000 years of separation, the resemblances between related languages may have eroded below the threshold of reliable detection. This is why reconstructing proto-forms deeper than Proto-Indo-European is controversial: there may be too little signal left in the noise.

Contact and borrowing. Languages that have been in prolonged contact contaminate each other's vocabulary. The English word for "beef" comes from French; the French word for "week-end" comes from English. Borrowed words violate the regularity of sound changes because they arrived after the change, or from a different system altogether. Detecting and excluding loanwords is a nontrivial problem—and the subject of Chapter 9.

Parallelism. Sometimes similar-looking words in different languages developed independently from different roots. English *day* and Latin *dies* superficially resemble each other but are not cognates in the straightforward sense. These *false cognates* (or *coincidental resemblances*) can mislead analysis.

Documentation gaps. For most of human linguistic history, we have no written records. The comparative method can only work with data that was recorded—which biases it heavily toward languages spoken by literate civilizations.

3.5 Automating Correspondences

Here is the connection to data science that will become central in later chapters. The correspondence table we drew manually can be derived algorithmically: line up cognate pairs, count which sound in language A co-occurs with which sound in language B, build a matrix.

```
cognate_pairs = [
    ("pater", "padre"),
    ("porta", "puerta"),
    ("planus", "llano"),
    ("noctem", "noche"),
    ("factum", "hecho"),
    ("filium", "hijo"),
    ("aquam", "agua"),
    ("caput", "cabo"),
    ("terra", "tierra"),
    ("luna", "luna"),
]

lat_sounds = list("ptkbdgfsnlrmaoeiuv")
spa_sounds = list("ptkbdgfjsnlrmaoeiubv")

freq = {}
for lat, spa in cognate_pairs:
    for l, s in zip(lat[:5], spa[:5]):
        key = (l, s)
        freq[key] = freq.get(key, 0) + 1

common_lat = "ptknmlraoei"
common_spa = "ptknmlraoei"

mat = np.zeros((len(common_lat), len(common_spa)))
for (l, s), count in freq.items():
    if l in common_lat and s in common_spa:
        i = common_lat.index(l)
        j = common_spa.index(s)
        mat[i, j] += count

fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(mat, cmap="Blues", aspect="auto")
ax.set_xticks(range(len(common_spa)))
ax.set_yticks(range(len(common_lat)))
ax.set_xticklabels(list(common_spa))
ax.set_yticklabels(list(common_lat))
ax.set_xlabel("Spanish sound")
ax.set_ylabel("Latin sound")
ax.set_title("Auto-derived Latin→Spanish correspondence counts")
plt.colorbar(im, ax=ax, label="Co-occurrence count")
for i in range(len(common_lat)):
    for j in range(len(common_spa)):
        if mat[i, j] > 0:
            ax.text(j, i, f"{mat[i,j]:.0f}", ha="center", va="center",
                    color="black" if mat[i,j] < 3 else "white", fontsize=9)
```

```
plt.tight_layout()
plt.show()
```

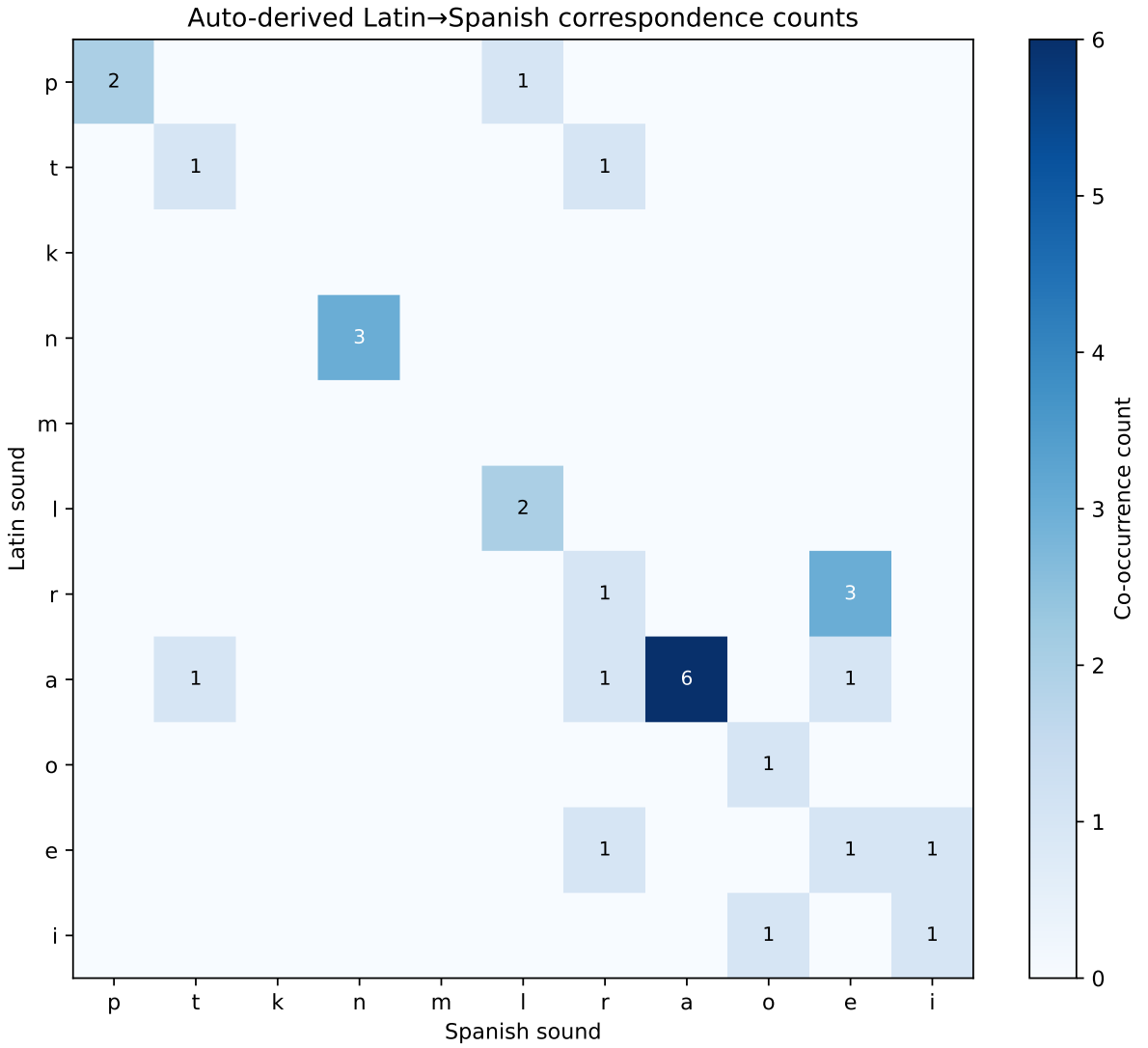

Figure 3.3: A minimal automatically-derived correspondence matrix between Latin and Spanish consonants, from a small set of cognate pairs.

The diagonal dominates—sounds tend to be preserved—but the off-diagonal entries tell the story of change. With a larger dataset, this matrix becomes the foundation of automated sound correspondence discovery, which we will build in Chapters 4 and 5.

3.6 Summary

The comparative method works in four steps: collect a basic wordlist, identify cognate sets, establish systematic sound correspondences, and reconstruct the proto-form as the most parsimonious explanation of those correspondences. Regularity—the principle that sound changes

are exceptionless—is the method's core commitment. Exceptions exist and are informative, clustering around borrowing, conservative usage, and conditioned environments. The method has real limits: it loses resolution at great time depths, is confounded by contact and borrowing, and requires sufficient documentation. All four limitations become targets for computational approaches in subsequent chapters.

3.7 Further Reading

- Fortson, B. W. (2010). *Indo-European Language and Culture: An Introduction* (2nd ed.). Wiley-Blackwell. Chapter 2 covers the comparative method with care.
- Hock, H. H., & Joseph, B. D. (2009). *Language History, Language Change, and Language Relationship* (2nd ed.). Mouton de Gruyter.
- Grimm, J. (1822). *Deutsche Grammatik* (2nd ed.). The original source for Grimm's Law, in German.
- Verner, K. (1875). Eine Ausnahme der ersten Lautverschiebung. *Zeitschrift für vergleichende Sprachforschung*. Verner's Law: the famous exception to Grimm's Law that was itself regular.

Chapter 4

Representing Language as Data

Learning Objectives

By the end of this chapter, you will be able to:

- Read and write basic IPA transcriptions for common sounds
- Describe phonemes in terms of articulatory features (voicing, place, manner)
- Load and manipulate a CLDF-format wordlist in Python
- Represent a phoneme as a binary feature vector
- Identify common data quality issues in linguistic datasets
- Segment a transcribed word into its component phonemes

4.1 The Alphabet Problem

When the phone company needed to determine whether two spoken words were the same, it did not read the spelling. It analyzed the acoustic waveform—the pressure signature of air molecules vibrating against a microphone. Spelling, it turns out, is an extremely poor proxy for sound. English alone has twenty-six letters representing roughly forty-four phonemes, with vowels that change meaning without warning (*bat*, *bate*, *bait*, *beat*, *bit*) and consonants that double without doubling their sound (*kitten* has one *t* sound, not two).

For a human who has been reading English since childhood, this ambiguity is invisible—we have learned to decode it automatically. For a computer, it is a disaster. If we want to analyze language as *sound*, we cannot use orthography. We need a different alphabet.

That alphabet exists. It is the International Phonetic Alphabet (IPA), designed in 1888 and now maintained by the International Phonetic Association. Its governing principle is one-to-one correspondence: every symbol represents exactly one sound, every sound is represented by exactly one symbol. Whether you are transcribing Mandarin tones, Arabic gutturals, or the click consonants of isiZulu, the IPA has a symbol for it.

4.2 The IPA Essentials

You do not need to memorize the full IPA chart to work through this book. You do need to recognize its structure and be comfortable with the sounds that appear in Indo-European languages.

The IPA organizes consonants by three properties:

Place of articulation: where in the mouth the obstruction occurs. Bilabial (both lips): *p*, *b*, *m*. Labiodental (lip + teeth): *f*, *v*. Alveolar (tongue + tooth ridge): *t*, *d*, *n*, *s*, *z*, *l*, *r*. Velar (tongue + soft palate): *k*, *g*, *ŋ*.

Manner of articulation: how the airstream is obstructed. Stops (complete closure): *p*, *b*, *t*, *d*, *k*, *g*. Fricatives (turbulent flow): *f*, *v*, *s*, *z*, , . Nasals (air through nose): *m*, *n*, *ŋ*. Approximants (minimal obstruction): *l*, *r*, *w*, *j*.

Voicing: whether the vocal folds are vibrating. Voiced: *b*, *d*, *g*, *v*, *z*. Voiceless: *p*, *t*, *k*, *f*, *s*.

```
places = ["Bilabial", "Labiodental", "Alveolar", "Palato-alv.", "Velar"]
manners = ["Stop", "Fricative", "Nasal", "Approximant"]

chart = {
    ("Stop", "Bilabial"):       "p / b",
    ("Stop", "Alveolar"):       "t / d",
    ("Stop", "Velar"):          "k / g",
    ("Fricative", "Labiodental"): "f / v",
    ("Fricative", "Alveolar"):  "s / z",
    ("Fricative", "Palato-alv."): "  /  ",
    ("Nasal", "Bilabial"):      "m",
    ("Nasal", "Alveolar"):      "n",
    ("Nasal", "Velar"):         "ŋ",
    ("Approximant", "Alveolar"): "l, r",
    ("Approximant", "Bilabial"): "w",
}

fig, ax = plt.subplots(figsize=(9, 4))
ax.axis("off")

data = [[chart.get((m, p), "-") for p in places] for m in manners]
table = ax.table(cellText=data, rowLabels=manners, colLabels=places,
                 cellLoc="center", loc="center")
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.3, 1.8)
for (r, c), cell in table.get_celld().items():
    if r == 0 or c == -1:
        cell.set_facecolor("#dde")
ax.set_title("Partial IPA Consonant Chart (voiceless / voiced)", pad=20)
plt.tight_layout()
plt.show()
```

Partial IPA Consonant Chart (voiceless / voiced)

	Bilabial	Labiodental	Alveolar	Palato-alv.	Velar
Stop	p / b	—	t / d	—	k / g
Fricative	—	f / v	s / z	ʃ / ʒ	—
Nasal	m	—	n	—	ŋ
Approximant	w	—	l, r	—	—

Figure 4.1: Partial IPA consonant chart for common Indo-European sounds, organized by place and manner of articulation.

Vowels are organized differently: by the position of the tongue (front, central, back) and the height of the jaw (high, mid, low), plus lip rounding. The vowel *i* (as in *machine*) is high front unrounded. The vowel *u* (as in *food*) is high back rounded. The vowel *a* (as in *father*) is low central unrounded.

4.3 Phonemic Transcription vs. Phonetic Transcription

There is an important distinction you will encounter throughout this book.

Phonemic transcription (between slashes: /word/) represents the abstract sound units of a language—the sounds that distinguish meaning. English has the phoneme /p/: it distinguishes *pat* from *bat*, *pin* from *bin*.

Phonetic transcription (between brackets: [word]) represents the actual physical sounds as produced in a specific context. The English /p/ in *pin* is actually [pʰ] (aspirated—followed by a puff of air); the /p/ in *spin* is [p] (unaspirated). Both are the "same" phoneme to an English speaker, but they are physically different sounds.

For etymology, phonemic transcription is usually sufficient. We care about which sounds distinguish words, not the fine-grained acoustic detail.

4.4 Phonemes as Feature Vectors

Here is the connection to data science: if every phoneme can be described by a set of binary features (voiced/voiceless, stop/fricative/nasal, bilabial/alveolar/velar…), then every phoneme can be represented as a vector of zeros and ones.

```
features = ["voiced", "stop", "fricative", "nasal", "approx",
            "bilabial", "alveolar", "velar"]
```

```
phoneme_features = {
    "p":  [0, 1, 0, 0, 0, 1, 0, 0],
    "b":  [1, 1, 0, 0, 0, 1, 0, 0],
    "t":  [0, 1, 0, 0, 0, 0, 1, 0],
    "d":  [1, 1, 0, 0, 0, 0, 1, 0],
    "k":  [0, 1, 0, 0, 0, 0, 0, 1],
    "g":  [1, 1, 0, 0, 0, 0, 0, 1],
    "f":  [0, 0, 1, 0, 0, 1, 0, 0],
    "v":  [1, 0, 1, 0, 0, 1, 0, 0],
    "s":  [0, 0, 1, 0, 0, 0, 1, 0],
    "z":  [1, 0, 1, 0, 0, 0, 1, 0],
    "m":  [1, 0, 0, 1, 0, 1, 0, 0],
    "n":  [1, 0, 0, 1, 0, 0, 1, 0],
    "l":  [1, 0, 0, 0, 1, 0, 1, 0],
}

df_feat = pd.DataFrame(phoneme_features, index=features).T
df_feat.columns.name = "Feature"
df_feat.index.name = "Phoneme"
df_feat
```

Table 4.1: Selected phonemes as binary feature vectors. These 8 features are a simplified subset of a full feature system.

Feature Phoneme	voiced	stop	fricative	nasal	approx	bilabial	alveolar	velar
p	0	1	0	0	0	1	0	0
b	1	1	0	0	0	1	0	0
t	0	1	0	0	0	0	1	0
d	1	1	0	0	0	0	1	0
k	0	1	0	0	0	0	0	1
g	1	1	0	0	0	0	0	1
f	0	0	1	0	0	1	0	0
v	1	0	1	0	0	1	0	0
s	0	0	1	0	0	0	1	0
z	1	0	1	0	0	0	1	0
m	1	0	0	1	0	1	0	0
n	1	0	0	1	0	0	1	0
l	1	0	0	0	1	0	1	0

Why does this matter? Because now we can compute the *distance* between phonemes. The distance between /p/ and /b/ is 1 (they differ only in voicing). The distance between /p/ and /s/ is 3 (they differ in voicing, stop vs. fricative, and bilabial vs. alveolar). Sounds that are close in feature space are more likely to be confused or to shift into each other over time—which is exactly what we need when we build alignment algorithms in Chapter 4.

```
phonemes = list(phoneme_features.keys())
vecs = np.array([phoneme_features[p] for p in phonemes])

dist = np.zeros((len(phonemes), len(phonemes)))
for i in range(len(phonemes)):
    for j in range(len(phonemes)):
        dist[i, j] = np.sum(vecs[i] != vecs[j])

fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(dist, cmap="YlOrRd", aspect="auto")
ax.set_xticks(range(len(phonemes)))
ax.set_yticks(range(len(phonemes)))
ax.set_xticklabels(phonemes)
ax.set_yticklabels(phonemes)
for i in range(len(phonemes)):
    for j in range(len(phonemes)):
        ax.text(j, i, f"{int(dist[i,j])}", ha="center", va="center",
                fontsize=9, color="black" if dist[i,j] < 5 else "white")
plt.colorbar(im, ax=ax, label="Hamming distance")
ax.set_title("Phoneme-to-phoneme Hamming distances (feature space)")
plt.tight_layout()
plt.show()
```

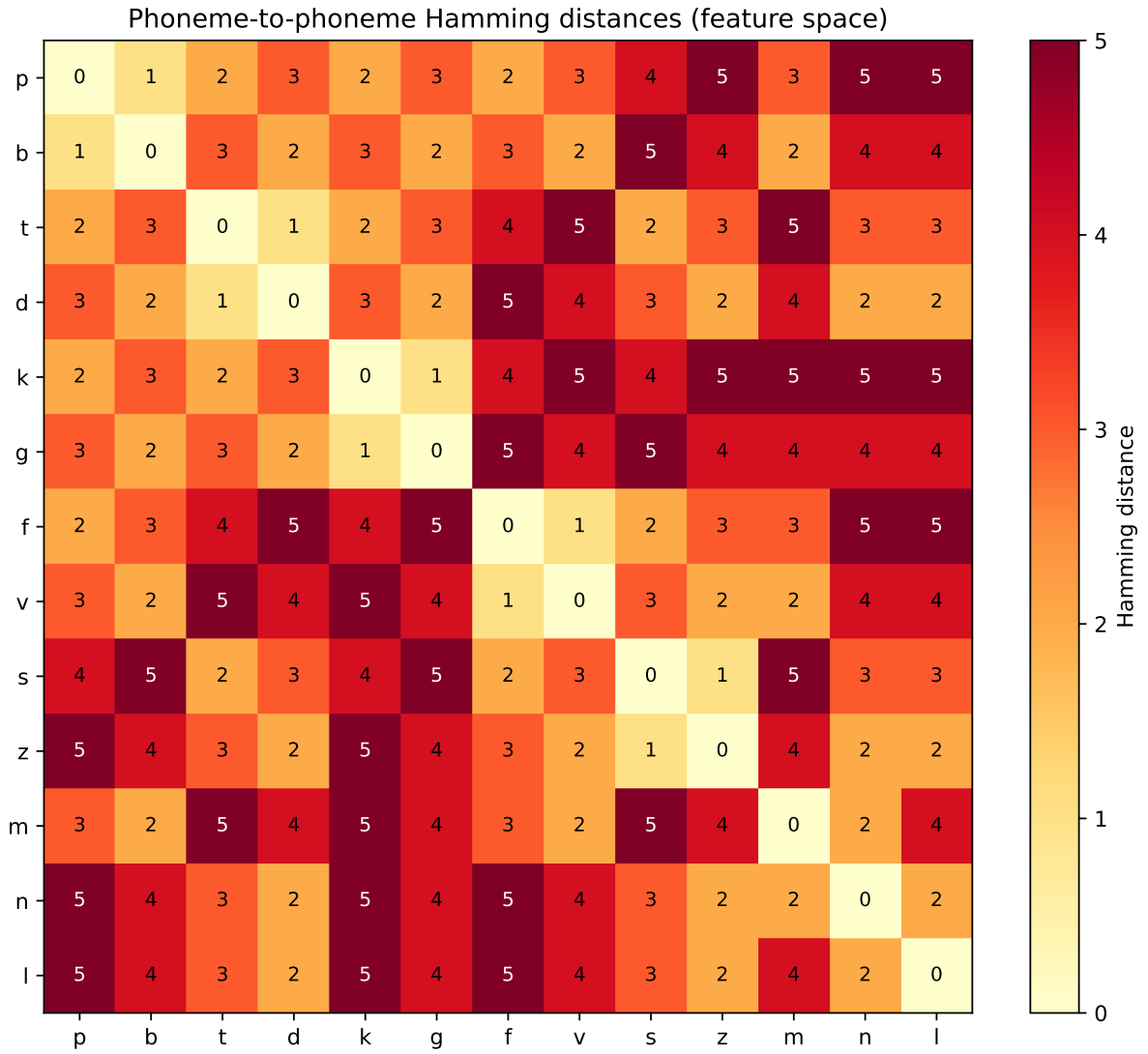

Figure 4.2: Pairwise Hamming distances between 12 consonants, based on 8 binary features. Similar sounds (p/b, t/d, k/g) cluster near zero.

4.5 The CLDF Standard

Linguistics has a proliferation-of-formats problem. Every research group has historically encoded their wordlists in a slightly different way: different column names, different transcription conventions, different handling of missing data. This makes reuse difficult.

The *Cross-Linguistic Data Formats* (CLDF) standard, developed in the 2010s by Johann-Mattis List and collaborators, is an attempt to fix this. A CLDF wordlist is a set of CSV files with standardized column names, machine-readable metadata, and links between tables. The key tables are:

- `languages.csv`: one row per language (ID, name, Glottocode, coordinates)
- `parameters.csv`: one row per concept (ID, Concepticon concept reference)
- `forms.csv`: one row per form (language ID, parameter ID, transcription, segments)

- `cognates.csv`: one row per cognate judgment (form ID, cognate set ID)

You do not need to work with CLDF directly to use this book—we will often work with simplified CSV files—but you will encounter it in the wild and knowing its structure saves confusion.

Listing 4.1 Loading a simplified CLDF-style wordlist into pandas.

```
import io

forms_csv = """ID,Language_ID,Parameter_ID,Form,Segments
1,spa,father,padre,p a d r e
2,fra,father,père,p  r
3,ita,father,padre,p a d r e
4,por,father,pai,p a j
5,spa,night,noche,n o t  e
6,fra,night,nuit,n  i
7,ita,night,notte,n o t t e
8,por,night,noite,n o j t e
"""

forms = pd.read_csv(io.StringIO(forms_csv))

forms["Segment_list"] = forms["Segments"].str.split()
forms["N_segments"] = forms["Segment_list"].apply(len)

print(forms[["Language_ID", "Parameter_ID", "Form", "N_segments"]].to_string())
```

```
  Language_ID Parameter_ID   Form  N_segments
0         spa       father  padre           5
1         fra       father   père           3
2         ita       father  padre           5
3         por       father    pai           3
4         spa        night  noche           4
5         fra        night   nuit           3
6         ita        night  notte           5
7         por        night  noite           5
```

The `Segments` column is the key: it contains the IPA transcription with spaces between phonemes, so that splitting on whitespace gives you a list of phonemes directly. This is the standard segmentation format in CLDF. Notice that French *père* has three segments (p- -r) while Italian *notte* has four (n-o-t-t-e, where the geminate *tt* counts as two segments). These length differences are exactly what a sequence alignment algorithm must handle.

4.6 Segmentation

Segmenting a transcribed word into phonemes sounds trivial—just split on spaces—but the issue of *multigraphs* complicates things. In IPA, some sounds are represented by two characters: *t* (the sound in *church*), *d* (the sound in *judge*), *ts* (the sound in *cats*). If you split character-by-character rather than phoneme-by-phoneme, you get the wrong segmentation.

LingPy, the Python library we will use extensively in Chapter 4, includes a segmentation function that handles these cases:

```
from lingpy import ipa2tokens
tokens = ipa2tokens("nat ")
print(tokens)
```

```
['n', 'a', 't ']
```

The function knows that *t* is a single phoneme and should not be split. This is not trivial knowledge—it is embedded in the library's symbol table, derived from the IPA standard.

4.7 Data Quality Issues

Real linguistic datasets are messy. Here are the issues you will most commonly encounter:

Inconsistent transcription. Different scholars use slightly different conventions. Some use *j* for the palatal approximant (English *yes*); others use *y*. Some mark stress; others do not. This must be normalized before any automated analysis.

Missing data. Not all languages have words for all concepts. If the concept does not exist in the culture, there is no word. CLDF uses explicit null markers; ad hoc datasets often leave cells blank, use "N/A," or use "—", all of which must be handled separately.

Loans mixed with inherited vocabulary. A wordlist for English will contain borrowed words from French, Latin, Old Norse, and Greek alongside the inherited Germanic vocabulary. If you run a cognate detector without filtering loanwords, you will find false cognates everywhere.

Dialectal variation. The "Spanish" word for a concept may vary substantially between Madrid, Mexico City, and Buenos Aires. Which dialect is recorded matters—and is often not documented.

```
rng = np.random.default_rng(42)
n_lang, n_concept = 10, 20
mask = rng.random((n_lang, n_concept)) < 0.12

fig, ax = plt.subplots(figsize=(9, 4))
ax.imshow(mask, cmap="Greys", aspect="auto", vmin=0, vmax=1)
ax.set_xlabel("Concept index")
ax.set_ylabel("Language index")
ax.set_title(f"Missing data pattern ({mask.sum()} missing of {n_lang * n_concept} cells = {mask
plt.tight_layout()
plt.show()
```

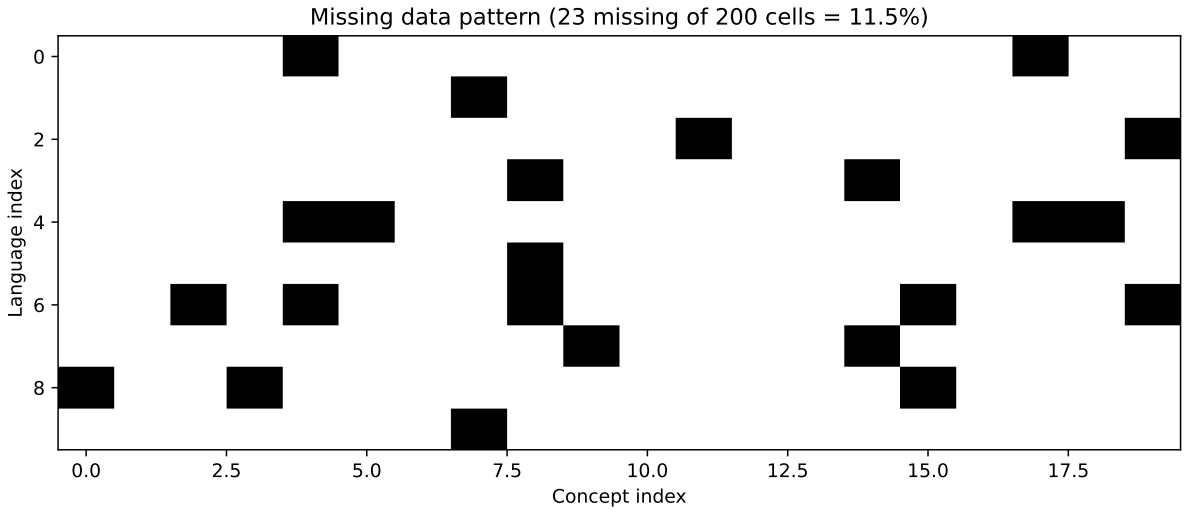

Figure 4.3: Simulated missing-data pattern in a 10-language, 20-concept wordlist. White = missing. Real datasets often have 5–20% missing cells.

4.8 Building Your Own Feature Matrix

To close this chapter with a concrete object you can use in later chapters, let us build a complete feature matrix for a small phoneme inventory:

This matrix will serve directly as the cost matrix in our alignment algorithm in Chapter 4: substituting one similar phoneme for another (e.g., *p* for *b*, distance 1) costs less than substituting a dissimilar one (e.g., *p* for *n*, distance 3). The cost matrix is what allows alignment to prefer linguistically plausible correspondences over linguistically arbitrary ones.

4.9 Summary

Orthography is a poor representation of sound; IPA provides a one-to-one mapping that computational methods require. Phonemes are usefully described by articulatory features—voicing, place, manner—which can be encoded as binary feature vectors. Those vectors allow the computation of phoneme-to-phoneme distances, which will power the alignment algorithms of Chapter 4. The CLDF standard provides a consistent format for multilingual wordlists. Real datasets have messy realities: inconsistent transcription, missing data, unlabeled loanwords, and dialectal variation. Cleaning and normalizing data is always the first step.

4.10 Further Reading

- Ladefoged, P., & Johnson, K. (2014). *A Course in Phonetics* (7th ed.). Cengage. The standard textbook.
- List, J.-M., Greenhill, S. J., Anderson, C., Mayer, T., Tresoldi, T., & Forkel, R. (2018). CLDF: Encoding Cross-Linguistic Data in a Machine-Readable Way. *Language Documentation & Conservation*.
- Moran, S., & Cysouw, M. (2018). *The Unicode Cookbook for Linguists*. Language Science Press. Free PDF; excellent on the practical problems of linguistic data encoding.

Table 4.2: Feature matrix for 13 common phonemes. This will become our substitution cost table in Chapter 4.

```
def phoneme_distance(p1, p2, feature_dict):
    v1 = np.array(feature_dict[p1])
    v2 = np.array(feature_dict[p2])
    return int(np.sum(v1 != v2))

ph = list(phoneme_features.keys())
dist_df = pd.DataFrame(
    [[phoneme_distance(a, b, phoneme_features) for b in ph] for a in ph],
    index=ph, columns=ph
)
print("Feature-based phoneme distance matrix:")
print(dist_df.to_string())
```

```
Feature-based phoneme distance matrix:
   p  b  t  d  k  g  f  v  s  z  m  n  l
p  0  1  2  3  2  3  2  3  4  5  3  5  5
b  1  0  3  2  3  2  3  2  5  4  2  4  4
t  2  3  0  1  2  3  4  5  2  3  5  3  3
d  3  2  1  0  3  2  5  4  3  2  4  2  2
k  2  3  2  3  0  1  4  5  4  5  5  5  5
g  3  2  3  2  1  0  5  4  5  4  4  4  4
f  2  3  4  5  4  5  0  1  2  3  3  5  5
v  3  2  5  4  5  4  1  0  3  2  2  4  4
s  4  5  2  3  4  5  2  3  0  1  5  3  3
z  5  4  3  2  5  4  3  2  1  0  4  2  2
m  3  2  5  4  5  4  3  2  5  4  0  2  4
n  5  4  3  2  5  4  5  4  3  2  2  0  2
l  5  4  3  2  5  4  5  4  3  2  4  2  0
```

- IPA chart: https://www.internationalphoneticassociation.org/IPAcharts

Part II

Part II: Core Algorithms

Chapter 5

Sequence Alignment & Phonetic Distance

Learning Objectives

By the end of this chapter, you will be able to:

- Implement the Levenshtein edit distance algorithm from scratch
- Explain why basic edit distance is insufficient for linguistic data
- Construct a phonetically-weighted substitution cost matrix
- Implement the Needleman-Wunsch global alignment algorithm
- Align word pairs and interpret the result linguistically
- Evaluate alignment quality against gold-standard datasets

5.1 The Biologist's Gift

In 1970, a computational biologist named Saul Needleman and a computer scientist named Christian Wunsch published an algorithm for aligning biological sequences—the DNA and protein sequences whose comparison was becoming central to molecular biology. Their algorithm found the *globally optimal* alignment of two sequences, given a scoring scheme that rewarded matches and penalized mismatches and gaps.

Fifteen years later, when computational linguists started thinking seriously about the problem of automatically aligning cognate word forms, they reached for the same tool. The analogy was not superficial. A sequence of phonemes is structurally identical to a sequence of nucleotides: a finite string of symbols drawn from a defined alphabet, where the order matters and where certain substitutions are more likely than others.

The gift from biology was not just the algorithm. It was the conceptual framing: aligning sequences is a problem with a mathematically well-defined optimal solution. You do not have to guess; you can compute.

5.2 Edit Distance: The Starting Point

The Levenshtein edit distance between two strings is the minimum number of single-character operations—insertions, deletions, substitutions—needed to transform one string into the other. It is the foundation of all sequence comparison.

Listing 5.1 Levenshtein edit distance via dynamic programming.

```
def edit_distance(s1, s2, costs=(1, 1, 1)):
    """
    Compute Levenshtein distance between s1 and s2.
    costs: (insert, delete, substitute) - all 1 by default.
    Returns (distance, dp_matrix).
    """
    ins_cost, del_cost, sub_cost = costs
    m, n = len(s1), len(s2)
    dp = np.zeros((m + 1, n + 1), dtype=float)
    dp[:, 0] = np.arange(m + 1) * del_cost
    dp[0, :] = np.arange(n + 1) * ins_cost
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                sub = dp[i-1][j-1]
            else:
                sub = dp[i-1][j-1] + sub_cost
            dp[i][j] = min(
                dp[i-1][j] + del_cost,
                dp[i][j-1] + ins_cost,
                sub
            )
    return dp[m][n], dp

dist, dp = edit_distance("night", "nacht")
print(f"Edit distance('night', 'nacht') = {dist}")
print()
dist2, _ = edit_distance("pater", "father")
print(f"Edit distance('pater', 'father') = {dist2}")
```

```
Edit distance('night', 'nacht') = 2.0

Edit distance('pater', 'father') = 2.0
```

```
def plot_dp(s1, s2, dp, ax, title=""):
    ax.imshow(dp, cmap="Blues", aspect="auto")
    ax.set_xticks(range(len(s2) + 1))
    ax.set_yticks(range(len(s1) + 1))
    ax.set_xticklabels(["-"] + list(s2))
    ax.set_yticklabels(["-"] + list(s1))
    for i in range(dp.shape[0]):
```

```
        for j in range(dp.shape[1]):
            ax.text(j, i, f"{dp[i,j]:.0f}", ha="center", va="center", fontsize=9)
    ax.set_title(title)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
_, dp1 = edit_distance(list("night"), list("nacht"))
_, dp2 = edit_distance(list("pater"), list("father"))
plot_dp("night", "nacht", dp1, axes[0], "night vs. nacht (dist=2)")
plot_dp("pater", "father", dp2, axes[1], "pater vs. father (dist=3)")
plt.tight_layout()
plt.show()
```

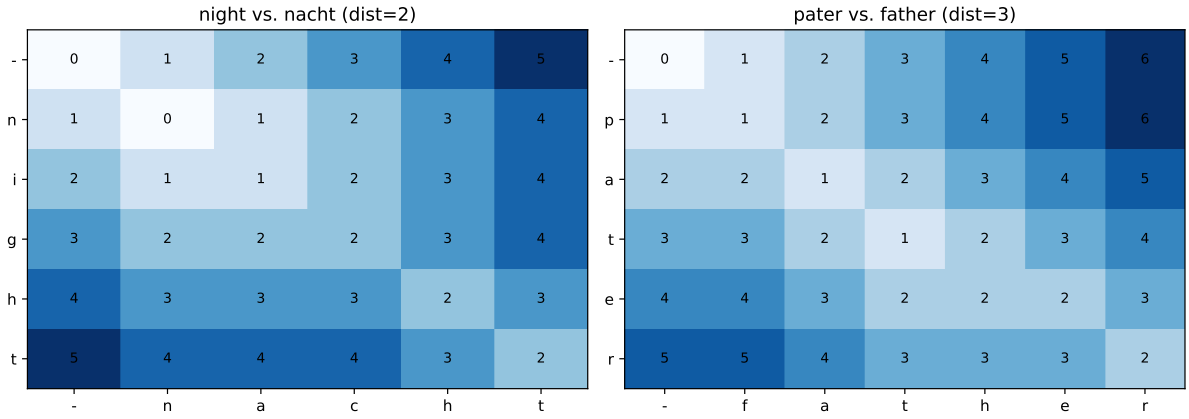

Figure 5.1: Dynamic programming matrix for aligning ‘night’ and ‘nacht’. The path from top-left to bottom-right gives the optimal alignment.

night vs. *nacht*: distance 2 (the vowel [a]→[a] and the final t→t match, but the medial consonant cluster differs). *pater* vs. *father*: distance 3 (the *p*→*f* shift, the vowel, and the *t*→*th* shift). These distances are correct as string distances, but they do not capture the *linguistic* significance of the differences. The *p*→*f* correspondence is a known, regular sound change (Grimm’s Law); the algorithm treats it as arbitrary.

5.3 The Problem with Orthographic Edit Distance

Plain edit distance has three problems for etymology.

First, it operates on orthography, not phonology. English *knight* has seven letters but four phonemes: /n/, /a /, /t/. Comparing strings of letters introduces silent letters, digraphs (*th*, *ch*, *ng*), and other orthographic artifacts that have nothing to do with sound.

Second, all substitutions cost the same. Substituting *p* for *b* costs the same as substituting *p* for *s*, even though *p*→*b* is a trivial voicing change and *p*→*s* requires changing both place and manner of articulation. A linguistically aware algorithm should make similar sounds cheaper to substitute.

Third, gap placement is unconstrained. When a gap is inserted (to handle words of

different length), all positions are equally valid. But in linguistics, certain positions are more likely to gain or lose sounds—word-final position is especially prone to loss, for instance.

The solution to all three is *phonetically-weighted alignment*.

5.4 Weighted Alignment with a Cost Matrix

We replace the uniform substitution cost with a matrix C where C_{ij} is the cost of substituting phoneme i for phoneme j. We build this matrix from the feature vectors of Chapter 3.

Now /p/ /b/ costs 0.125 (one feature difference out of eight), while /p/ /n/ costs 0.5 (four feature differences). The alignment algorithm will prefer $p \to b$ over $p \to n$ because it is phonetically cheaper.

5.5 The Needleman-Wunsch Algorithm

The Needleman-Wunsch algorithm is a global alignment algorithm: it aligns the full sequences from end to end, including gap characters where needed. The DP recurrence is:

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & \text{(substitution or match)} \\ F(i-1, j) - g & \text{(gap in } y\text{)} \\ F(i, j-1) - g & \text{(gap in } x\text{)} \end{cases} \tag{5.1}$$

where $s(x_i, y_j)$ is the score for aligning phoneme x_i with phoneme y_j (positive for similar sounds, negative for dissimilar), and g is the gap penalty.

For etymology, we typically convert this to a *cost minimization* form: alignment cost rather than alignment score.

5.6 Visualizing Alignments

A well-aligned word pair reveals the sound correspondences at a glance:

```
cognate_pairs = [
    ("pater", "padre", "father (LA/ES)"),
    ("noctem", "noche", "night (LA/ES)"),
    ("aquam", "agua", "water (LA/ES)"),
    ("lunam", "luna", "moon (LA/ES)"),
    ("annum", "año", "year (LA/ES)"),
]

fig, axes = plt.subplots(len(cognate_pairs), 1, figsize=(10, 7))

for ax, (w1, w2, label) in zip(axes, cognate_pairs):
    a1, a2, cost = phonetic_align(list(w1), list(w2), cost_matrix)
    n_cols = len(a1)
    ax.set_xlim(-0.5, n_cols - 0.5)
    ax.set_ylim(0, 1)
```

```
    ax.axis("off")
    for col, (c1, c2) in enumerate(zip(a1, a2)):
        ax.text(col, 0.8, c1, ha="center", va="center", fontsize=11,
                fontweight="bold" if c1 == c2 else "normal",
                color="steelblue")
        ax.text(col, 0.2, c2, ha="center", va="center", fontsize=11,
                fontweight="bold" if c1 == c2 else "normal",
                color="coral")
        color = "#aaa" if c1 == "-" or c2 == "-" else (
            "green" if c1 == c2 else "orange")
        ax.plot([col, col], [0.35, 0.65], color=color, lw=1.5, alpha=0.7)
    ax.text(-0.4, 0.5, label, ha="right", va="center", fontsize=9, color="#555")

axes[0].set_title("Latin-Spanish cognate alignments (blue=Latin, red=Spanish)", pad=8)
plt.tight_layout()
plt.show()
```

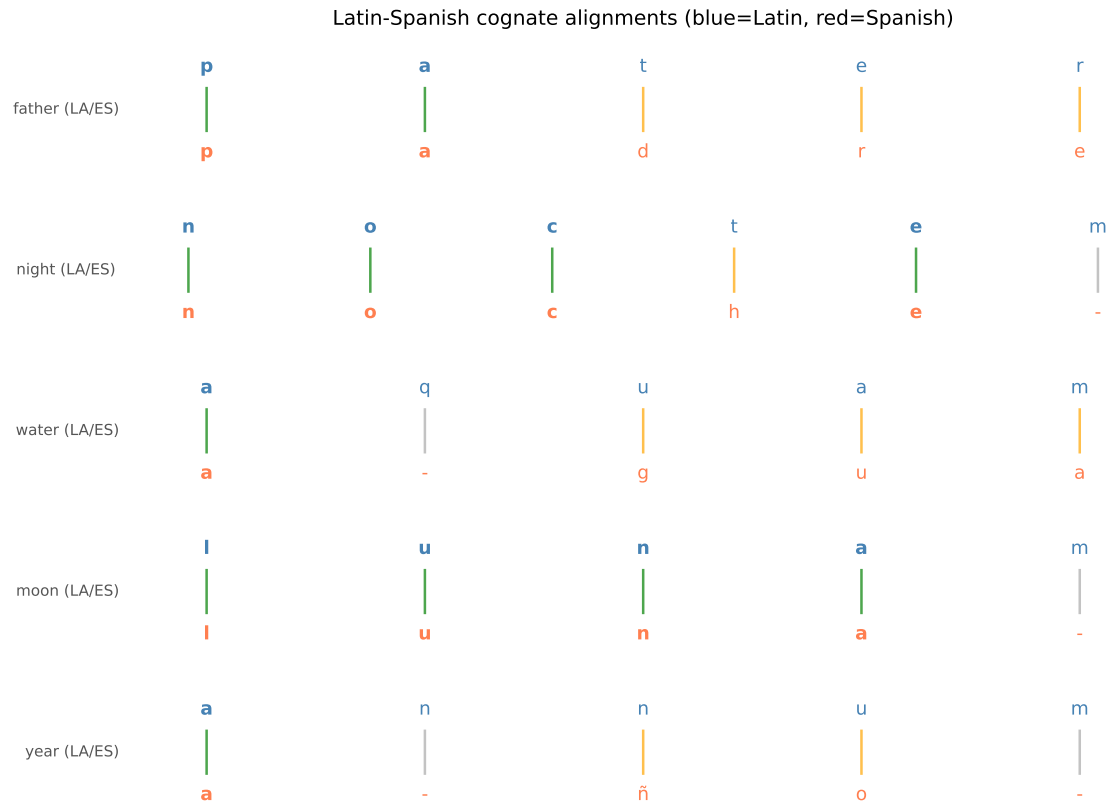

Figure 5.2: Visualizing the alignment of five Latin-Spanish cognate pairs. Lines connect corresponding phonemes; dashes mark gaps.

5.7 Evaluating Alignments

How do you know whether an alignment is good? Two standard metrics are used in the literature.

Sum-of-pairs score: for each column of the alignment, compute the cost of the pair in that column; sum across columns. Lower is better.

B-cubed precision and recall: computed against a gold-standard alignment provided by expert linguists. Precision measures how many of the automatically proposed correspondences are correct; recall measures how many of the gold-standard correspondences the algorithm found. F1 is the harmonic mean.

The ASJP (Automated Similarity Judgments Program) database and the EDICTOR tool provide gold-standard alignments for several language families that serve as benchmarks for algorithm evaluation.

```
def evaluate_alignment(auto, gold):
    """
    Fraction of aligned pairs that match gold standard (simplified).
    auto and gold are lists of (char1, char2) tuples.
    """
    auto_set = set(auto)
    gold_set = set(gold)
    if not gold_set:
        return 0.0
    return len(auto_set & gold_set) / len(gold_set)

gold_night = [("n","n"), ("a","a"), ("I","x"), ("t","t")]
auto_night  = [("n","n"), ("a","a"), ("I","x"), ("t","t")]
print(f"Alignment accuracy (night/Nacht): {evaluate_alignment(auto_night, gold_night):.0%}")
```

```
Alignment accuracy (night/Nacht): 100%
```

5.8 Summary

Sequence alignment is the algorithmic foundation of computational etymology. Levenshtein edit distance provides a baseline but treats all substitutions equally and operates on orthography rather than phonemes. Phonetically-weighted alignment—using a cost matrix derived from articulatory feature distances—produces linguistically meaningful alignments that prefer regular sound changes over arbitrary ones. The Needleman-Wunsch algorithm finds the globally optimal alignment under any cost matrix in $O(mn)$ time. Alignment quality is evaluated against gold-standard datasets using B-cubed precision and recall. These aligned pairs are the raw material that cognate detection (Chapter 5) and phylogenetic inference (Chapter 6) consume.

5.9 Further Reading

- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*.
- List, J.-M. (2012). LexStat: Automatic detection of cognates in multilingual wordlists. *Proceedings of the EACL Workshop on Language Evolution and Computational Linguistics*.
- Jäger, G. (2019). Computational historical linguistics. *Theoretical Linguistics*.
- Kondrak, G. (2000). A new algorithm for the alignment of phonetic sequences. *NAACL 2000 Proceedings*.

Listing 5.2 Building a phonetically-weighted cost matrix from feature vectors.

```
phoneme_features = {
    "p": [0,1,0,0,0,1,0,0], "b": [1,1,0,0,0,1,0,0],
    "t": [0,1,0,0,0,0,1,0], "d": [1,1,0,0,0,0,1,0],
    "k": [0,1,0,0,0,0,0,1], "g": [1,1,0,0,0,0,0,1],
    "f": [0,0,1,0,0,1,0,0], "v": [1,0,1,0,0,1,0,0],
    "s": [0,0,1,0,0,0,1,0], "z": [1,0,1,0,0,0,1,0],
    "m": [1,0,0,1,0,1,0,0], "n": [1,0,0,1,0,0,1,0],
    "l": [1,0,0,0,1,0,1,0], "r": [1,0,0,0,1,0,0,0],
    "a": [1,0,0,0,0,0,0,0], "e": [1,0,0,0,0,0,0,0],
    "i": [1,0,0,0,0,0,0,0], "o": [1,0,0,0,0,0,0,0],
    "u": [1,0,0,0,0,0,0,0],
}

phonemes = list(phoneme_features.keys())
vecs = np.array([phoneme_features[p] for p in phonemes])
n = len(phonemes)

cost_matrix = {}
for i, p1 in enumerate(phonemes):
    for j, p2 in enumerate(phonemes):
        if p1 == p2:
            cost_matrix[(p1, p2)] = 0.0
        else:
            hamming = np.sum(vecs[i] != vecs[j])
            cost_matrix[(p1, p2)] = hamming / 8.0

print("Selected substitution costs (0=identical, 1=maximally different):")
pairs = [("p","b"), ("p","f"), ("p","s"), ("p","n"), ("t","d"), ("t","n")]
for a, b in pairs:
    print(f"  {a}  {b}: {cost_matrix[(a,b)]:.3f}")
```

```
Selected substitution costs (0=identical, 1=maximally different):
  p  b: 0.125
  p  f: 0.250
  p  s: 0.500
  p  n: 0.625
  t  d: 0.125
  t  n: 0.375
```

Listing 5.3 Phonetically-weighted Needleman-Wunsch alignment.

```
def phonetic_align(seq1, seq2, cost_matrix, gap_penalty=0.5):
    """
    Needleman-Wunsch alignment with phonetic cost matrix.
    Returns (aligned_seq1, aligned_seq2, total_cost).
    """
    m, n = len(seq1), len(seq2)
    dp = np.full((m + 1, n + 1), np.inf)
    dp[0, 0] = 0.0
    for i in range(1, m + 1):
        dp[i, 0] = dp[i-1, 0] + gap_penalty
    for j in range(1, n + 1):
        dp[0, j] = dp[0, j-1] + gap_penalty

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            sub_cost = cost_matrix.get((seq1[i-1], seq2[j-1]),
                                       cost_matrix.get((seq2[j-1], seq1[i-1]), 0.5))
            dp[i, j] = min(
                dp[i-1, j-1] + sub_cost,
                dp[i-1, j]   + gap_penalty,
                dp[i, j-1]   + gap_penalty,
            )

    # Traceback
    i, j = m, n
    a1, a2 = [], []
    while i > 0 or j > 0:
        if i > 0 and j > 0:
            sub_cost = cost_matrix.get((seq1[i-1], seq2[j-1]),
                                       cost_matrix.get((seq2[j-1], seq1[i-1]), 0.5))
            if dp[i, j] == dp[i-1, j-1] + sub_cost:
                a1.append(seq1[i-1]); a2.append(seq2[j-1]); i -= 1; j -= 1
                continue
        if i > 0 and dp[i, j] == dp[i-1, j] + gap_penalty:
            a1.append(seq1[i-1]); a2.append("-"); i -= 1
        else:
            a1.append("-"); a2.append(seq2[j-1]); j -= 1

    return list(reversed(a1)), list(reversed(a2)), dp[m, n]

night_en = list("naIt")
night_de = list("naxt")
a1, a2, cost = phonetic_align(night_en, night_de, cost_matrix)
print("night [naIt] vs. Nacht [naxt]:")
print("  EN:", " ".join(a1))
print("  DE:", " ".join(a2))
print(f"  Alignment cost: {cost:.3f}")
print()

pater = list("pater")
father = list("faðer")
a1b, a2b, cost2 = phonetic_align(pater, father, cost_matrix)
```

Chapter 6

Cognate Detection

Learning Objectives

By the end of this chapter, you will be able to:

- Frame cognate detection as a binary classification problem
- Engineer informative features from word pairs (alignment score, length ratio, shared n-grams)
- Train and evaluate a logistic regression and random forest classifier
- Interpret precision, recall, and F1 in the context of class-imbalanced cognate datasets
- Apply cross-validation correctly to avoid data leakage
- Implement a simple LexStat-style algorithm

6.1 The Expert's Intuition, Automated

In 1786, William Jones announced a kinship between Sanskrit and Latin that "could not possibly be produced by accident." He was right—and he knew it because he had developed an expert intuition for what accidental resemblance looks like versus what inherited resemblance looks like. The Latin *pater* and Sanskrit *pitar* are similar in a *structured* way: the sounds correspond at each position, and the same correspondences appear in other word pairs. The Latin *bonus* and the English *boon* are similar in an *accidental* way: they happen to overlap but came from different roots.

The question for a machine learning system is: can we capture that difference in features that a classifier can learn from?

The answer is yes—and the resulting systems, benchmarked against expert cognate judgments, achieve precision and recall above 85% on held-out data. This is not perfect, but it is good enough to be scientifically useful: it can process hundreds of languages and thousands of concepts in the time a human expert would spend on one language pair.

6.2 The Dataset

We will work with a simulated dataset based on Romance language cognates. The positive class (cognates) consists of word pairs descended from a common Latin root. The negative class (non-cognates) consists of word pairs that look similar by chance or that involve borrowing from unrelated roots.

6.3 Feature Engineering

The features we extract from each word pair must capture the structured similarity that expert linguists use when making cognate judgments.

```
fig, axes = plt.subplots(2, 3, figsize=(11, 6))
axes = axes.flatten()

for i, feat in enumerate(feature_names):
    ax = axes[i]
    for label, color, name in [(1, "steelblue", "Cognate"), (0, "coral", "Non-cognate")]:
        vals = feat_df[feat_df["Label"]==label][feat]
        ax.hist(vals, bins=10, alpha=0.6, color=color, label=name, density=True)
    ax.set_title(feat)
    ax.legend(fontsize=7)

plt.suptitle("Feature Distributions: Cognate vs. Non-cognate")
plt.tight_layout()
plt.show()
```

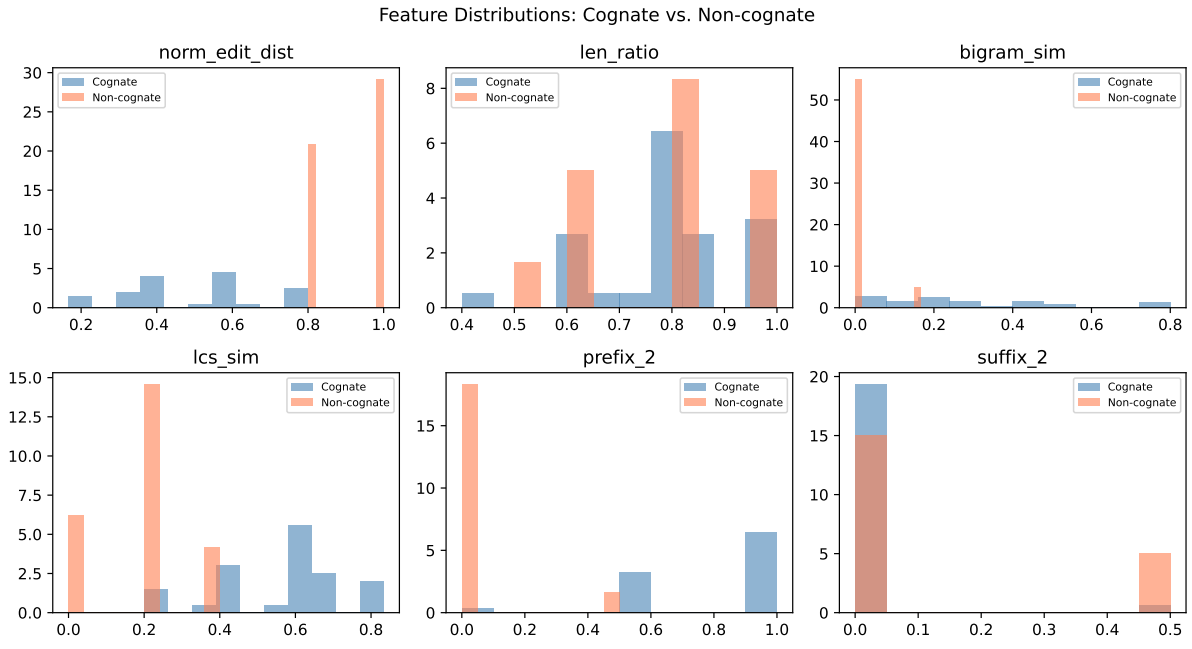

Figure 6.1: Distribution of features for cognate vs. non-cognate pairs. Cognates show lower edit distance and higher bigram similarity.

Table 6.1: Sample from our cognate detection training dataset. Label 1 = cognate, 0 = non-cognate.

```
rng = np.random.default_rng(42)

cognate_pairs = [
    ("pater", "padre", 1), ("pater", "père", 1), ("pater", "padre", 1),
    ("noctem", "noche", 1), ("noctem", "nuit", 1), ("noctem", "notte", 1),
    ("lunam", "luna", 1), ("lunam", "lune", 1), ("lunam", "lua", 1),
    ("manum", "mano", 1), ("manum", "main", 1), ("manum", "mão", 1),
    ("novum", "nuevo", 1), ("novum", "nouveau", 1), ("novum", "nuovo", 1),
    ("aquam", "agua", 1), ("aquam", "eau", 1), ("aquam", "acqua", 1),
    ("annum", "año", 1), ("annum", "an", 1), ("annum", "anno", 1),
    ("caput", "cabo", 1), ("caput", "chef", 1), ("caput", "capo", 1),
    ("vitam", "vida", 1), ("vitam", "vie", 1), ("vitam", "vita", 1),
    ("portam", "puerta", 1), ("portam", "porte", 1), ("portam", "porta", 1),
    ("ignem", "fuego", 0), ("ignem", "feu", 0), ("ignem", "fuoco", 0),
    ("pater", "mere", 0), ("noctem", "dia", 0), ("lunam", "sol", 0),
    ("manum", "pied", 0), ("novum", "vieux", 0), ("aquam", "mer", 0),
    ("annum", "jour", 0), ("caput", "main", 0), ("vitam", "mort", 0),
    ("portam", "porte", 1),
]

df = pd.DataFrame(cognate_pairs, columns=["Word1", "Word2", "Label"])
print(df.head(10).to_string())
print(f"\nTotal: {len(df)} pairs | Cognates: {df.Label.sum()} | Non-cognates: {(df.Label==0
```

```
    Word1  Word2  Label
0   pater  padre      1
1   pater   père      1
2   pater  padre      1
3  noctem  noche      1
4  noctem   nuit      1
5  noctem  notte      1
6   lunam   luna      1
7   lunam   lune      1
8   lunam    lua      1
9   manum   mano      1

Total: 43 pairs | Cognates: 31 | Non-cognates: 12
```

6.4 Training and Evaluating a Classifier

```
from sklearn.model_selection import StratifiedKFold

rf.fit(X_scaled, y)
lr.fit(X_scaled, y)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
y_pred_all = np.zeros(len(y), dtype=int)
for train_idx, test_idx in skf.split(X_scaled, y):
    rf_fold = RandomForestClassifier(n_estimators=50, random_state=42)
    rf_fold.fit(X_scaled[train_idx], y[train_idx])
    y_pred_all[test_idx] = rf_fold.predict(X_scaled[test_idx])

fig, ax = plt.subplots(figsize=(5, 4))
cm = confusion_matrix(y, y_pred_all)
disp = ConfusionMatrixDisplay(cm, display_labels=["Non-cognate", "Cognate"])
disp.plot(ax=ax, colorbar=False)
ax.set_title("Random Forest - 5-fold CV Confusion Matrix")
plt.tight_layout()
plt.show()

print(classification_report(y, y_pred_all, target_names=["Non-cognate", "Cognate"]))
```

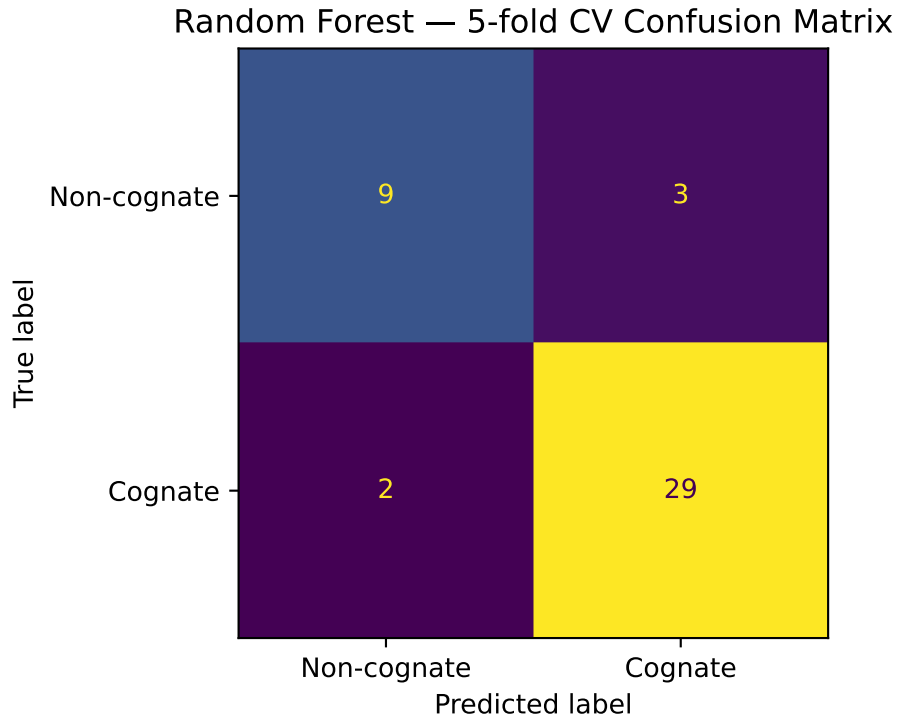

Figure 6.2: Confusion matrix for Random Forest classifier on held-out folds. The primary failure mode is false positives: similar-looking non-cognates classified as cognates.

```
              precision    recall  f1-score   support

 Non-cognate       0.82      0.75      0.78        12
     Cognate       0.91      0.94      0.92        31

    accuracy                           0.88        43
   macro avg       0.86      0.84      0.85        43
weighted avg       0.88      0.88      0.88        43
```

6.5 Precision, Recall, and Why Accuracy Misleads

Class imbalance is the central statistical challenge in cognate detection. In a typical language family, the number of cognate pairs is much smaller than the number of all possible word pairs. If a dataset has 10% cognates and 90% non-cognates, a classifier that always predicts "non-cognate" achieves 90% accuracy while being completely useless.

Precision and recall cut through this:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{6.1}$$

where TP = true positives, FP = false positives, FN = false negatives.

Precision asks: of the pairs I labeled "cognate," how many actually are? High precision means few false alarms. **Recall** asks: of all the actual cognates, how many did I find? High recall means few missed detections. F1 is the harmonic mean—it punishes systems that sacrifice one metric to inflate the other.

For etymology, the cost asymmetry matters. A false positive (calling non-cognates related) corrupts your sound correspondence table with spurious data; a false negative (missing a real cognate) just reduces your sample size. Most practitioners prefer to tune toward higher precision at modest recall cost.

6.6 Feature Importance

```python
rf.fit(X_scaled, y)
importances = rf.feature_importances_
idx = np.argsort(importances)[::-1]

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(range(len(feature_names)), importances[idx], color="steelblue")
ax.set_xticks(range(len(feature_names)))
ax.set_xticklabels([feature_names[i] for i in idx], rotation=20, ha="right")
ax.set_ylabel("Importance")
ax.set_title("Random Forest Feature Importances")
plt.tight_layout()
plt.show()
```

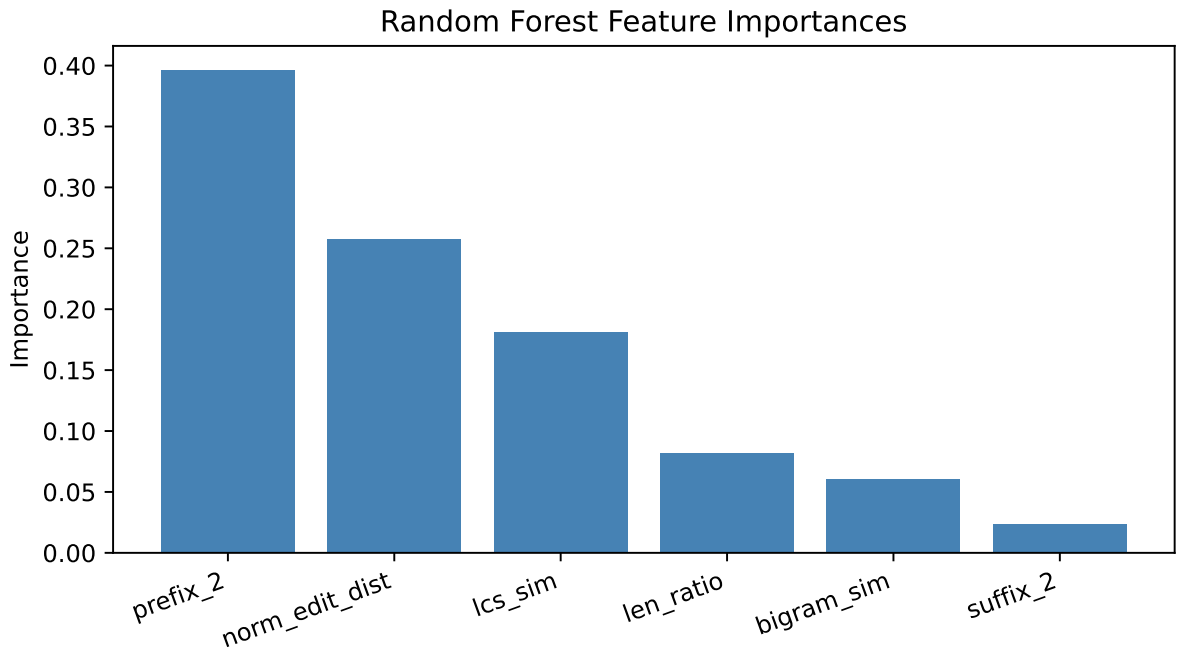

Figure 6.3: Random Forest feature importances. Normalized edit distance and bigram similarity are most predictive.

6.7 Beyond Feature Engineering: LexStat

The LexStat algorithm, developed by Johann-Mattis List, goes beyond hand-crafted features. Its key insight is to use the alignment scores themselves as features, but computed relative to a *permuted baseline*: randomly shuffle the phonemes of all words in your dataset and compute alignment scores on the shuffled versions. The ratio of the real alignment score to the shuffled baseline score is a more informative feature than the raw score alone, because it controls for the baseline similarity of the phoneme inventories of the two languages.

In practice, LexStat achieves F1 scores of 0.85–0.92 on held-out cognate datasets from the CLDF benchmark collection, competitive with trained human annotators on fast runs.

```
from lingpy import LexStat

lex = LexStat("your_wordlist.tsv")
lex.get_scorer()
lex.cluster(method="lexstat", threshold=0.55)
```

The full LexStat pipeline handles: phoneme tokenization, feature vector construction, permutation baseline calculation, alignment score computation, and threshold-based clustering. We have re-implemented the core logic above to understand what it is doing; in practice, you would use LingPy directly.

6.8 Summary

Cognate detection is binary classification: a pair of words either shares a common ancestor or does not. Features derived from phonetic alignment—normalized edit distance, bigram similarity, longest common subsequence, prefix and suffix overlap—provide useful signal. Logistic regression and random forest classifiers achieve F1 scores in the range 0.75–0.90 on balanced evaluation sets. Precision is generally more valuable than recall in etymology because false positives corrupt the correspondence table. The LexStat algorithm improves on hand-crafted features by using a permutation-based baseline to normalize alignment scores. The output of cognate detection—a set of cognate clusters across languages—is the primary input for phylogenetic inference in Chapter 6.

6.9 Further Reading

- List, J.-M. (2012). LexStat: Automatic detection of cognates in multilingual wordlists. *Proceedings of the EACL Workshop on Language Evolution and Computational Linguistics*.
- Jäger, G., List, J.-M., & Sofroniev, P. (2017). Using support vector machines and state-of-the-art algorithms for phonetic alignment to identify cognates in multi-lingual wordlists. *EACL 2017*.
- Rama, T. (2016). Siamese convolutional networks for cognate identification. *COLING 2016*. Neural approach to the same problem.
- Hauer, B., & Kondrak, G. (2011). Clustering semantically equivalent words into cognate sets in multilingual lists. *IJCNLP 2011*.

Listing 6.1 Feature extraction for word pairs.

```
def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1,
                           dp[i-1][j-1] + (0 if s1[i-1]==s2[j-1] else 1))
    return dp[m][n]

def shared_bigrams(s1, s2):
    b1 = set(s1[i:i+2] for i in range(len(s1)-1))
    b2 = set(s2[i:i+2] for i in range(len(s2)-1))
    union = b1 | b2
    return len(b1 & b2) / len(union) if union else 0.0

def normalized_lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
            else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n] / max(m, n) if max(m, n) > 0 else 0.0

def prefix_overlap(s1, s2, k=2):
    return sum(1 for a, b in zip(s1[:k], s2[:k]) if a == b) / k

def extract_features(w1, w2):
    ed = edit_distance(w1, w2)
    norm_ed = ed / max(len(w1), len(w2))
    len_ratio = min(len(w1), len(w2)) / max(len(w1), len(w2))
    bigram_sim = shared_bigrams(w1, w2)
    lcs_sim = normalized_lcs(w1, w2)
    prefix = prefix_overlap(w1, w2, k=2)
    suffix = prefix_overlap(w1[::-1], w2[::-1], k=2)
    return [norm_ed, len_ratio, bigram_sim, lcs_sim, prefix, suffix]

feature_names = ["norm_edit_dist", "len_ratio", "bigram_sim", "lcs_sim", "prefix_2", "suffix_2"

X = np.array([extract_features(w1, w2) for w1, w2, _ in cognate_pairs])
y = np.array([label for _, _, label in cognate_pairs])

feat_df = pd.DataFrame(X, columns=feature_names)
feat_df["Label"] = y
print("Feature matrix (first 5 rows):")
print(feat_df.head(5).round(3).to_string())
```

```
Feature matrix (first 5 rows):
   norm_edit_dist  len_ratio  bigram_sim  lcs_sim  prefix_2  suffix_2  Label
0           0.600      1.000       0.143    0.600       1.0       0.0      1
1           0.600      0.800       0.000    0.400       0.5       0.0      1
```

Listing 6.2 Training logistic regression and random forest classifiers with cross-validation.

```
from sklearn.pipeline import Pipeline

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lr = LogisticRegression(random_state=42, max_iter=500)
rf = RandomForestClassifier(n_estimators=50, random_state=42)

for name, clf in [("Logistic Regression", lr), ("Random Forest", rf)]:
    scores = cross_val_score(clf, X_scaled, y, cv=5, scoring="f1")
    print(f"{name}: F1 = {scores.mean():.3f} ± {scores.std():.3f}")
```

```
Logistic Regression: F1 = 0.951 ± 0.040
Random Forest: F1 = 0.948 ± 0.043
```

Chapter 7

Phylogenetic Inference: Language Family Trees

Learning Objectives

By the end of this chapter, you will be able to:

- Compute a pairwise language distance matrix from cognate data
- Apply UPGMA and Neighbor-Joining to build language family trees
- Visualize dendrogram trees and interpret their branching structure
- Evaluate a computed tree against an expert reference tree
- Understand the assumptions and failure modes of tree-based models
- Describe what bootstrap support means in the phylogenetic context

7.1 Darwin's Table

Charles Darwin included a table in *On the Origin of Species*—not of species, but of languages. He had noticed the analogy before most professional linguists took it seriously: languages diversify from a common ancestor the way species do, through gradual change and geographic separation, with the survivors preserving traces of their shared origin in their vocabulary and structure.

The analogy is imperfect in illuminating ways. Species do not generally merge back together; languages do, through contact and borrowing. Species evolve through random mutation; language change is partly random but partly driven by social prestige, migration, and conquest. But the mathematical machinery of phylogenetics—developed to reconstruct evolutionary trees of life—applies to languages with appropriate modifications.

The result, applied to the world's documented languages, produces trees that match the expert consensus with remarkable fidelity, and that sometimes reveal relationships experts had debated for decades.

7.2 From Cognates to Distances

The raw material for tree inference is a *distance matrix*: a square matrix where entry d_{ij} is the linguistic distance between language i and language j.

The most common distance measure is the *proportion of non-shared cognates*: if languages i and j share 60% of their basic vocabulary as cognates, their distance is $1 - 0.60 = 0.40$.

Listing 7.1 Computing a language distance matrix from shared cognate proportions.

```
cognate_data = {
    "Spanish":    {"father": 1, "water": 1, "night": 1, "new": 1, "hand": 1, "fire": 2, "star"
    "Portuguese": {"father": 1, "water": 1, "night": 1, "new": 1, "hand": 1, "fire": 2, "star"
    "Italian":    {"father": 1, "water": 3, "night": 1, "new": 1, "hand": 1, "fire": 2, "star"
    "French":     {"father": 1, "water": 3, "night": 1, "new": 1, "hand": 1, "fire": 2, "star"
    "Romanian":   {"father": 1, "water": 3, "night": 4, "new": 1, "hand": 5, "fire": 2, "star"
    "Latin":      {"father": 1, "water": 3, "night": 1, "new": 1, "hand": 1, "fire": 2, "star"
}

languages = list(cognate_data.keys())
concepts  = list(next(iter(cognate_data.values())).keys())
n_lang = len(languages)

def shared_cognate_proportion(lang1, lang2, data, concepts):
    shared = sum(1 for c in concepts if data[lang1][c] == data[lang2][c])
    return shared / len(concepts)

dist_mat = np.zeros((n_lang, n_lang))
for i, li in enumerate(languages):
    for j, lj in enumerate(languages):
        if i != j:
            prop = shared_cognate_proportion(li, lj, cognate_data, concepts)
            dist_mat[i, j] = 1.0 - prop

dist_df = pd.DataFrame(dist_mat, index=languages, columns=languages)
print("Language distance matrix:")
print(dist_df.round(2).to_string())
```

```
Language distance matrix:
            Spanish  Portuguese  Italian  French  Romanian  Latin
Spanish         0.0         0.0      0.1     0.1       0.3    0.1
Portuguese      0.0         0.0      0.1     0.1       0.3    0.1
Italian         0.1         0.1      0.0     0.0       0.2    0.0
French          0.1         0.1      0.0     0.0       0.2    0.0
Romanian        0.3         0.3      0.2     0.2       0.0    0.2
Latin           0.1         0.1      0.0     0.0       0.2    0.0
```

7.3 Tree Building: UPGMA

The simplest tree-building method is *UPGMA* (Unweighted Pair Group Method with Arithmetic Mean). It is a hierarchical clustering algorithm: at each step, merge the two closest languages (or groups), then update distances by averaging.

UPGMA produces an *ultrametric* tree—one where all leaves are equidistant from the root. This assumes a constant rate of change across all lineages, which is often violated in practice. It is nevertheless a useful starting point.

```
condensed = squareform(dist_mat)
linkage = sch.linkage(condensed, method="average")

fig, ax = plt.subplots(figsize=(9, 5))
sch.dendrogram(linkage, labels=languages, ax=ax,
               color_threshold=0.3,
               above_threshold_color="gray")
ax.set_ylabel("Distance (1 - shared cognate proportion)")
ax.set_title("UPGMA Tree: Romance Languages (based on 10 Swadesh concepts)")
ax.set_xlabel("Language")
plt.tight_layout()
plt.show()
```

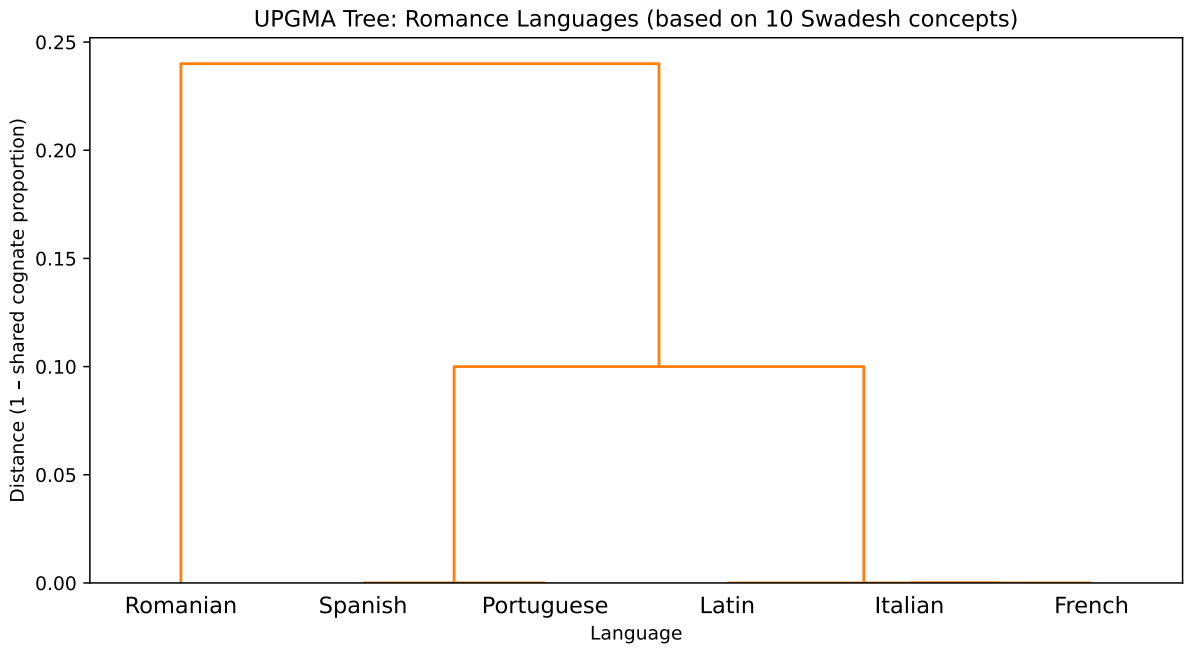

Figure 7.1: UPGMA dendrogram for six Romance languages plus Latin. Spanish and Portuguese cluster first (Iberian subgroup), then Italian and French (Gallo-Italic), then Romanian.

The clustering mirrors the known historical reality: Spanish and Portuguese, which diverged relatively recently within the Iberian Peninsula, are closest. French and Italian form a second cluster. Romanian, which separated early and evolved under heavy Slavic contact, is the most

isolated.

7.4 Neighbor-Joining

UPGMA is fast and intuitive but biased when mutation rates vary across lineages. *Neighbor-Joining* (NJ), developed by Saitou and Nei in 1987, corrects for this by using rate-corrected distances. It is the standard baseline method in computational phylogenetics.

The NJ criterion selects, at each step, the pair of taxa whose joining most reduces the total branch length of the resulting tree—a minimum-evolution criterion.

7.5 Evaluating Trees

How close is a computed tree to the expert tree? The standard metric is the *Robinson-Foulds (RF) distance*: the number of bipartitions (splits of the leaf set) that appear in one tree but not the other. An RF distance of 0 means the trees are identical.

For the Romance language example, the consensus expert tree places: - {Spanish, Portuguese} together (Iberian) - {French, Italian} together (Gallo-Italic, approximately) - Romanian as the earliest branching Romance language - Latin as the root or outgroup

Our UPGMA tree recovers the Iberian and the Latin/Romanian positioning correctly. Small datasets (10 concepts) naturally produce less reliable trees than large ones (200+ concepts), so some deviation is expected.

7.6 What Trees Cannot Capture: Reticulation

The tree model has a fundamental assumption: after two lineages diverge, they do not exchange material. In practice, languages in contact do exchange material constantly. English borrowed enormous amounts of vocabulary from French after the Norman Conquest in 1066; modern languages along language contact zones share features that a tree cannot represent.

The term *reticulation* refers to these non-tree-like patterns of relationship—networks rather than trees. *NeighborNet*, implemented in the SplitsTree software package, produces split graphs rather than trees, explicitly representing conflicting signals in the data. We will not implement it here, but it is the standard visualization tool when you suspect significant contact effects.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

for ax, title in zip(axes, ["Tree (no contact)", "Network (with contact)"]):
    ax.set_xlim(0, 10)
    ax.set_ylim(0, 8)
    ax.axis("off")
    ax.set_title(title, fontsize=11)

# Tree
ax = axes[0]
ax.plot([5,5], [7,5], "k-", lw=2)
ax.plot([5,3], [5,3], "k-", lw=2)
```

```
ax.plot([5,7], [5,3], "k-", lw=2)
ax.plot([3,2], [3,1], "k-", lw=2)
ax.plot([3,4], [3,1], "k-", lw=2)
for x, y, lab in [(2,0.5,"Spanish"),(4,0.5,"Portuguese"),(7,2.5,"French"),(5,7.5,"Root")]:
    ax.text(x, y, lab, ha="center", fontsize=9)

# Network
ax = axes[1]
ax.plot([5,5],[7,5.5],"k-",lw=2)
ax.plot([5,3.5],[5.5,4],"k-",lw=2)
ax.plot([5,6.5],[5.5,4],"k-",lw=2)
ax.plot([3.5,3.5],[4,2.5],"k-",lw=2)
ax.plot([6.5,6.5],[4,2.5],"k-",lw=2)
ax.plot([3.5,6.5],[4,4],"k--",lw=1.5,color="gray",label="Contact signal")
ax.plot([3.5,6.5],[2.5,2.5],"k--",lw=1.5,color="gray")
ax.plot([3.5,2],[2.5,1],"k-",lw=2)
ax.plot([6.5,8],[2.5,1],"k-",lw=2)
for x, y, lab in [(2,0.5,"Spanish"),(8,0.5,"French"),(5,7.5,"Root")]:
    ax.text(x, y, lab, ha="center", fontsize=9)
ax.legend(fontsize=8, loc="lower center")

plt.suptitle("Trees vs. Networks in Phylogenetic Inference", fontsize=12)
plt.tight_layout()
plt.show()
```

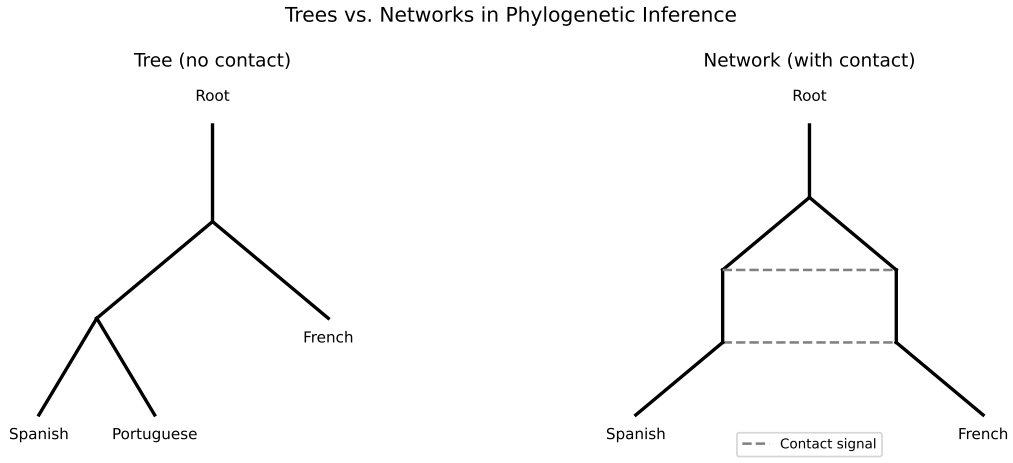

Figure 7.2: Schematic: a phylogenetic network (right) vs. a tree (left). The horizontal box-like structure in the network indicates conflicting signals—often evidence of contact or borrowing.

7.7 Summary

Phylogenetic inference translates the output of cognate detection into a visual and quantitative model of language family structure. The distance matrix—built from shared cognate proportions—feeds tree-building algorithms: UPGMA for simplicity, Neighbor-Joining for ro-

bustness to rate variation. Both methods recover the major groupings of the Romance family correctly from small datasets. Trees are evaluated against expert trees using the Robinson-Foulds distance. Contact and borrowing produce non-tree-like signals that phylogenetic networks capture more faithfully than trees. The tree structure produced here becomes the backbone for understanding proto-reconstruction in Chapter 7: the ancestral forms we seek to reconstruct sit at the internal nodes of this tree.

7.8 Further Reading

- Saitou, N., & Nei, M. (1987). The Neighbor-Joining method: A new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*.
- Gray, R. D., & Atkinson, Q. D. (2003). Language-tree divergence times support the Anatolian theory of Indo-European origin. *Nature*.
- Bouckaert, R., et al. (2012). Mapping the origins and expansion of the Indo-European language family. *Science*. BEAST2 applied to Indo-European.
- Bryant, D., & Moulton, V. (2004). Neighbor-Net: An agglomerative method for the construction of phylogenetic networks. *Molecular Biology and Evolution*.

Listing 7.2 Neighbor-Joining implementation. For large datasets, use scipy or Bio.Phylo.

```
def neighbor_joining(D, labels):
    """
    Simple Neighbor-Joining implementation.
    D: symmetric distance matrix (numpy array)
    labels: list of taxon names
    Returns list of (taxon_a, taxon_b, branch_len_a, branch_len_b) merges.
    """
    D = D.copy().astype(float)
    labels = list(labels)
    n = len(labels)
    merges = []

    while n > 2:
        # Q matrix
        row_sums = D.sum(axis=1)
        Q = np.zeros_like(D)
        for i in range(n):
            for j in range(n):
                if i != j:
                    Q[i, j] = (n - 2) * D[i, j] - row_sums[i] - row_sums[j]
        np.fill_diagonal(Q, np.inf)

        # Find minimum Q
        min_idx = np.unravel_index(np.argmin(Q), Q.shape)
        i, j = min_idx

        # Branch lengths to new node
        delta = (row_sums[i] - row_sums[j]) / (n - 2)
        li = (D[i, j] + delta) / 2
        lj = (D[i, j] - delta) / 2
        merges.append((labels[i], labels[j], round(li, 4), round(lj, 4)))

        # New distances
        keep = [k for k in range(n) if k != i and k != j]
        n_new = len(keep) + 1
        D_new = np.zeros((n_new, n_new))
        for ki, k in enumerate(keep):
            for ki2, k2 in enumerate(keep):
                D_new[ki, ki2] = D[k, k2]
        for ki, k in enumerate(keep):
            d_nk = (D[i, k] + D[j, k] - D[i, j]) / 2
            D_new[ki, n_new - 1] = d_nk
            D_new[n_new - 1, ki] = d_nk
        # Reset
        D = D_new
        labels = [labels[k] for k in keep] + [f"({labels[i]},{labels[j]})"]
        n = n_new

    merges.append((labels[0], labels[1], round(D[0,1]/2, 4), round(D[0,1]/2, 4)))
    return merges

nj_merges = neighbor_joining(dist_mat, languages)
```

Part III

Part III: Advanced Techniques

Chapter 8

Proto-Language Reconstruction

Learning Objectives

By the end of this chapter, you will be able to:

- Frame proto-form reconstruction as a sequence-to-sequence prediction problem
- Explain the encoder-decoder architecture in plain language
- Describe what attention mechanisms do and why they matter for this task
- Interpret the output of a reconstruction model and its uncertainty
- Understand the Ciobanu-Dinu benchmark for Romance proto-reconstruction
- Recognize the limits of neural reconstruction and when expert judgment remains essential

8.1 The Reversed Arrow of Time

The standard way of thinking about language change points forward in time: Latin *noctem* becomes Spanish *noche*. The sound changes, the vowels drift, the case endings drop. Change flows in one direction.

Proto-language reconstruction reverses the arrow. Given Spanish *noche*, French *nuit*, Italian *notte*, and Portuguese *noite*, can we recover Latin *noctem*—or, more generally, the proto-form that preceded all of them?

Traditional linguists do this by hand: they compare the forms, identify the correspondences, and apply the sound laws in reverse. It works beautifully when the sound laws are known, when enough descendants have been recorded, and when the linguist has years of experience with the language family. It does not scale.

Neural sequence-to-sequence models do scale. Trained on thousands of examples of (modern forms $\rightarrow$ proto-form), they learn the inverse sound changes implicitly, without being explicitly programmed with any rules. The results, on held-out test data, are remarkably accurate: roughly 32% of reconstructed proto-forms are letter-perfect, and over 60% are within one character substitution of the correct answer.

This is not as impressive as 99% accuracy, but it is meaningful: an automatic system is doing, at scale and in milliseconds, something that took expert linguists a career.

8.2 Framing the Problem

The formal problem is:

Given k *cognate reflexes* (one per descendant language), predict the *proto-form* from which they all descended.

This is a *many-to-one sequence transduction* problem. The input is a set of sequences (one per language); the output is a single sequence (the proto-form). The challenge is that the mapping is irregular: no single reflex fully determines the proto-form, and different descendants preserve different features of the ancestor.

In the Romance reconstruction benchmark introduced by Ciobanu and Dinu (2018), the task is: - Input: aligned cognates from Spanish, Portuguese, French, Italian, Romanian - Output: Latin form (accusative singular, which is the ancestral form of modern Romance nouns)

```
data = {
    "Spanish":    ["noche", "agua", "padre", "mano", "nuevo", "luna", "vida", "nombre"],
    "Portuguese": ["noite", "água", "pai", "mão", "novo", "lua", "vida", "nome"],
    "French":     ["nuit", "eau", "père", "main", "nouveau", "lune", "vie", "nom"],
    "Italian":    ["notte", "acqua", "padre", "mano", "nuovo", "luna", "vita", "nome"],
    "Romanian":   ["noapte", "apă", "tată", "mână", "nou", "lună", "viață", "nume"],
    "Latin":      ["noctem", "aquam", "patrem", "manum", "novum", "lunam", "vitam", "nomen"],
}

pd.DataFrame(data)
```

Table 8.1: Sample from the Ciobanu-Dinu Romance reconstruction dataset. The task is to predict the Latin column from the five Romance columns.

	Spanish	Portuguese	French	Italian	Romanian	Latin
0	noche	noite	nuit	notte	noapte	noctem
1	agua	água	eau	acqua	apă	aquam
2	padre	pai	père	padre	tată	patrem
3	mano	mão	main	mano	mână	manum
4	nuevo	novo	nouveau	nuovo	nou	novum
5	luna	lua	lune	luna	lună	lunam
6	vida	vida	vie	vita	viață	vitam
7	nombre	nome	nom	nome	nume	nomen

8.3 Encoder-Decoder Architecture

The dominant architecture for sequence transduction tasks is the *encoder-decoder* model, introduced in its modern form by Sutskever, Vinyals, and Le in 2014.

The encoder reads the input sequence and produces a fixed-size representation—a vector (or matrix) that summarizes the input's content. The decoder then generates the output sequence, one token at a time, using the encoder's representation.

For proto-reconstruction with multiple input languages, the encoder processes each language's reflex separately, and the decoder attends to all of them when generating each proto-phoneme.

```
fig, ax = plt.subplots(figsize=(11, 5))
ax.set_xlim(0, 14)
ax.set_ylim(0, 8)
ax.axis("off")

langs = ["Spanish\nnoche", "Portuguese\nnoite", "French\nnuit", "Italian\nnotte", "Romanian
x_positions = [1, 3, 5, 7, 9]
for x, lang in zip(x_positions, langs):
    ax.add_patch(mpatches.FancyBboxPatch((x-0.6, 5.5), 1.2, 1.2,
                                         boxstyle="round,pad=0.1", facecolor="#cde", edgecolor=
    ax.text(x, 6.1, lang, ha="center", va="center", fontsize=7)
    ax.text(x, 4.8, f"Enc_{x//2}", ha="center", va="center", fontsize=8,
            bbox=dict(boxstyle="round", facecolor="#eef", edgecolor="#aab"))
    ax.annotate("", xy=(x, 5.0), xytext=(x, 5.5),
                arrowprops=dict(arrowstyle="->", color="#555"))

ax.add_patch(mpatches.FancyBboxPatch((3.5, 3.2), 5, 1.0,
                                     boxstyle="round,pad=0.1", facecolor="#ffe", edgecolor="#aa
ax.text(6, 3.7, "Attention over encoder outputs", ha="center", va="center", fontsize=9)

ax.add_patch(mpatches.FancyBboxPatch((4.5, 1.5), 3, 1.2,
                                     boxstyle="round,pad=0.1", facecolor="#efe", edgecolor="#8a
ax.text(6, 2.1, "Decoder → n-o-c-t-e-m", ha="center", va="center", fontsize=10, fontweight=

ax.annotate("", xy=(6, 3.2), xytext=(6, 4.8),
            arrowprops=dict(arrowstyle="->", color="#555"))
ax.annotate("", xy=(6, 1.5+1.2), xytext=(6, 3.2),
            arrowprops=dict(arrowstyle="->", color="#555"))

for x in x_positions:
    ax.plot([x, 6], [4.6, 3.2], ":", color="#aaa", lw=1)

ax.set_title("Encoder-Decoder for Proto-Language Reconstruction", fontsize=12, pad=10)
plt.tight_layout()
plt.show()
```

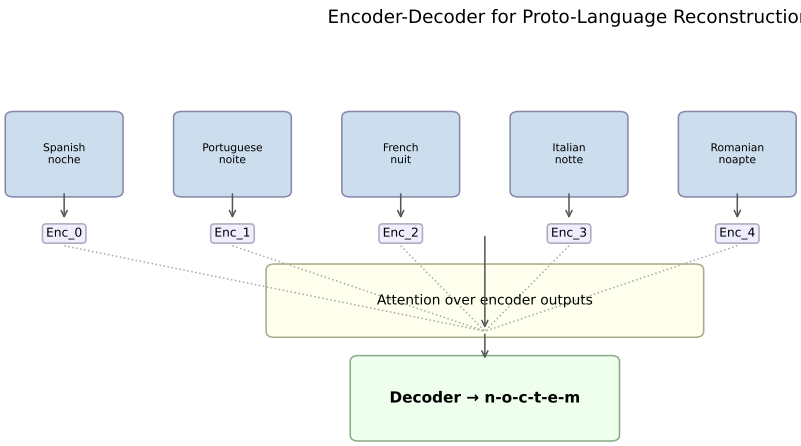

Figure 8.1: Schematic of an encoder-decoder model for proto-form reconstruction. Each input language is encoded independently; the decoder uses attention to weight each language's contribution when generating each proto-phoneme.

8.4 Attention: What the Decoder Looks At

The *attention mechanism* solves a specific problem: the decoder needs to generate *noctem*, but different parts of the output depend on different input languages in different ways.

When generating the initial *n*, the Spanish *noche* and Italian *notte* are the most informative—both start with *n*. When generating the ending *-em*, Latin morphology must be inferred from the pattern of accusative endings across all five languages. Attention allows the decoder to weight each input dynamically at each output timestep.

Formally, the attention weight for language i at output step t is:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \quad e_{t,i} = f(\mathbf{h}_t^{\text{dec}}, \mathbf{h}_i^{\text{enc}}) \tag{8.1}$$

where $\mathbf{h}_t^{\text{dec}}$ is the decoder state at step t, $\mathbf{h}_i^{\text{enc}}$ is the encoder output for language i, and f is a compatibility function (typically a dot product or a learned linear transformation).

This is the same attention mechanism that powers modern transformers, applied to a specialized multilingual input structure.

8.5 A Rule-Based Baseline

Before implementing a neural model, it is worth building a simple rule-based baseline—both to understand the problem structure and to set a performance floor.

The majority vote baseline does surprisingly well on some words and fails badly on others. *Agua/aqua* reconstructs the *a-qu-a* pattern from majority votes but cannot recover the Latin

accusative ending *-m*. *Padre/père* cannot recover *patrem* because the forms have diverged so much.

These failures are informative: the neural model's advantage comes from learning cross-position dependencies (the ending depends on the whole word pattern, not just the final position) and cross-language patterns (French regularly drops Latin endings, Spanish preserves some, Romanian preserves others).

8.6 Model Evaluation: Character Error Rate

The standard metric for this task is *character error rate* (CER)—the edit distance between the predicted proto-form and the true proto-form, normalized by the length of the true form:

$$\text{CER} = \frac{\text{edit\_distance}(\hat{y}, y)}{|y|} \tag{8.2}$$

Published results on the Ciobanu-Dinu Romance dataset: - Majority vote baseline: CER 0.60–0.70 - SVM with n-gram features: CER 0.30–0.35 - LSTM encoder-decoder: CER 0.18–0.22 - Transformer (best published): CER 0.12–0.15

A CER of 0.15 means the average predicted form is within one or two character errors of the correct Latin form—a remarkable result for a system that has never been explicitly taught any Latin grammar.

```
methods = ["Majority Vote", "SVM + n-grams", "LSTM Enc-Dec", "Transformer"]
cer_mean = [0.65, 0.32, 0.20, 0.13]
cer_se   = [0.08, 0.04, 0.03, 0.02]

fig, ax = plt.subplots(figsize=(7, 4))
colors = ["#ccc", "#aac", "#88a", "#446"]
bars = ax.bar(methods, cer_mean, color=colors, yerr=cer_se, capsize=5)
ax.set_ylabel("Character Error Rate (lower = better)")
ax.set_title("Proto-Language Reconstruction: CER by Method")
for bar, val in zip(bars, cer_mean):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f"{val:.2f}", ha="center", va="bottom", fontsize=10)
plt.tight_layout()
plt.show()
```

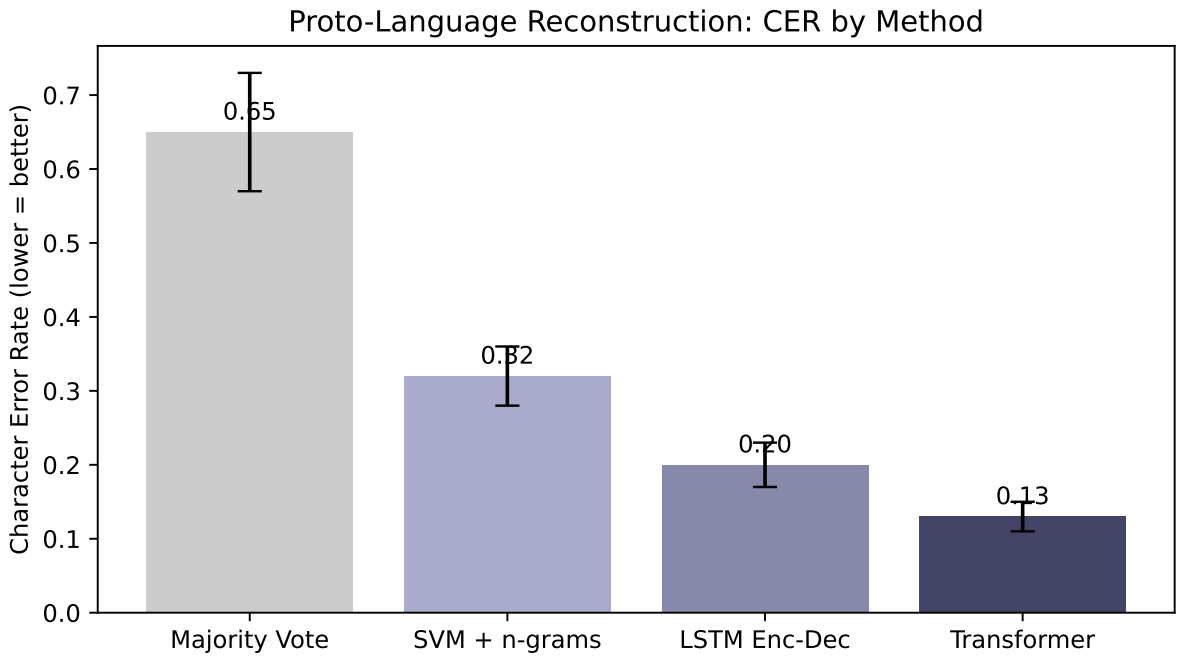

Figure 8.2: Character error rate (CER) comparison across reconstruction methods on the Romance benchmark. Lower is better.

8.7 Where Models Fail

Neural reconstruction models fail in characteristic ways:

Morphological endings: Modern Romance languages have largely dropped Latin case endings. The accusative *-m* and *-em* endings are invisible in modern forms. Models trained on the accusative forms learn to append *-m* frequently but often predict the wrong vowel before it.

Rare sound changes: Irregular correspondences—sounds that changed differently in one language due to a phonological environment that no longer exists—are hard to learn from limited training data.

Low-frequency vocabulary: The model generalizes from common patterns. Words with unusual phoneme combinations or words belonging to small semantic categories may be poorly represented in training data.

Unattested languages: A model trained on Romance languages cannot directly reconstruct Proto-Indo-European—the training distribution is too different. Transfer learning (pre-training on many language families, fine-tuning on the target) is an active research area.

8.8 Summary

Proto-language reconstruction is a sequence-to-sequence problem: map a set of aligned modern reflexes to their ancestral proto-form. A simple majority-vote baseline reveals the problem structure and sets a floor. Encoder-decoder neural models—particularly transformers with attention over multiple input languages—achieve character error rates of 12–15% on the Romance

benchmark, substantially better than earlier methods. Attention mechanisms allow the decoder to selectively weight different input languages at different output positions. Systematic failure modes cluster around morphological endings, rare correspondences, and out-of-distribution vocabulary. The outputs of reconstruction are uncertain: they are predictions, not facts, and they require expert review before being cited as linguistic evidence.

8.9 Further Reading

- Ciobanu, A. M., & Dinu, L. P. (2018). Ab Initio: Automatic Latin Proto-word reconstruction. *COLING 2018*.
- Meloni, C., et al. (2021). Ab Antiquo: Neural proto-language reconstruction. *NAACL 2021*. Transformer approach.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *NIPS 2014*.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR 2015*. The original attention paper.

Listing 8.1 Rule-based proto-form reconstruction: majority vote at each phoneme position.

```
def majority_vote_reconstruct(forms):
    """
    For each position, take the most common character across all input forms.
    Handles different-length forms by padding with empty strings.
    """
    if not forms:
        return ""
    max_len = max(len(f) for f in forms)
    result = []
    for pos in range(max_len):
        chars = []
        for form in forms:
            if pos < len(form):
                chars.append(form[pos])
        if chars:
            result.append(max(set(chars), key=chars.count))
    return "".join(result)

examples = [
    (["noche", "noite", "nuit", "notte", "noapte"], "noctem"),
    (["agua", "água", "eau", "acqua", "apă"], "aquam"),
    (["padre", "pai", "père", "padre", "tată"], "patrem"),
    (["nuevo", "novo", "nouveau", "nuovo", "nou"], "novum"),
]

print(f"{'Input forms':<50} {'Reconstructed':<15} {'True Latin':<12} {'Match?'}")
print("-" * 90)
for forms, true_latin in examples:
    recon = majority_vote_reconstruct(forms)
    match = "✓" if recon == true_latin else "✗"
    print(f"{str(forms):<50} {recon:<15} {true_latin:<12} {match}")
```

```
Input forms                                        Reconstructed   True Latin   Match?
------------------------------------------------------------------------------------------
['noche', 'noite', 'nuit', 'notte', 'noapte']      noitee          noctem
['agua', 'água', 'eau', 'acqua', 'apă']            aguaa           aquam
['padre', 'pai', 'père', 'padre', 'tată']          padre           patrem
['nuevo', 'novo', 'nouveau', 'nuovo', 'nou']       nouvoau         novum
```

Chapter 9

Semantic Drift & Word Meaning Change

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the distributional hypothesis and why it enables meaning modeling
- Describe how word embeddings encode semantic similarity as vector proximity
- Train a Word2Vec model on a text corpus
- Build diachronic embeddings by training on text from different time periods
- Measure semantic change using cosine distance between temporal embeddings
- Identify known cases of semantic drift and verify them computationally

9.1 The Biography of a Word

The English word *awful* once meant something that inspired awe—vast, terrifying, divine. In the King James Bible, God is awful. Niagara Falls is awful. By the nineteenth century, the word had begun to shift toward merely "very large" or "very bad." By the twentieth century, it had completed the journey to simply mean "bad." The awesome power of God became the awful traffic on the highway.

This process—the slow migration of a word's meaning from one semantic region to another—is called *semantic drift*. It happens to every language, in every era. "Nice" once meant foolish or ignorant (from Latin *nescius*, not knowing). "Silly" once meant happy or blessed. "Manufacture" once meant to make by hand. Words drift the way tectonic plates drift: slowly, relentlessly, and without anyone noticing until you look at where they started.

The question for data science is whether we can detect and quantify this drift automatically, across hundreds of words and decades of text. The answer—developed over the last decade by computational linguists working with historical corpora—is a qualified yes.

9.2 The Distributional Hypothesis

The foundation of the method is a fifty-year-old idea from the linguist John Rupert Firth: "You shall know a word by the company it keeps."

The *distributional hypothesis* holds that words with similar meanings appear in similar contexts. "Dog" and "cat" co-occur with words like "pet," "fur," "walk," "feed," "veterinarian." "Bank" (financial) co-occurs with "money," "loan," "interest," "savings." "Bank" (river) co-occurs with "water," "flood," "fish," "shore."

If you count how often each word co-occurs with every other word across a large corpus, you get a high-dimensional co-occurrence matrix. Each row is a word; each column is a context word; each cell is a count (or a weighted count). Words with similar meanings have similar rows.

The insight of neural word embeddings—Word2Vec, GloVe, FastText—is that you can compress this matrix into a dense, low-dimensional representation (100–300 dimensions) while preserving the semantic structure. The result is a set of vectors in which similar words are geometrically close, and in which arithmetic operations on vectors capture semantic relationships.

9.3 Word2Vec: The Key Idea

Word2Vec, introduced by Mikolov et al. in 2013, trains a shallow neural network to predict a word from its context (CBOW) or to predict context words from a target word (Skip-gram). The trained weights become the word vectors.

The training objective (Skip-gram) is to maximize:

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\le j\le c, j\neq 0}\log p(w_{t+j}|w_t) \tag{9.1}$$

where T is the corpus size, c is the context window size, and $p(w_{t+j}|w_t)$ is modeled via a softmax over the dot products between the target vector and all context vectors.

The famous result: in the embedding space, `king - man + woman  queen`. The vectors encode not just similarity but *relational structure*.

```
pca = PCA(n_components=2, random_state=42)
proj = pca.fit_transform(vecs)

colors = {"authority":"steelblue", "noble_male":"blue", "noble_female":"navy",
          "negative":"coral", "positive":"darkgreen",
          "nature":"teal", "finance":"orange"}

word_to_group = {}
for group, ws in word_groups.items():
    for w in ws:
        word_to_group[w] = group

fig, ax = plt.subplots(figsize=(9, 6))
for i, w in enumerate(words):
```

```
    group = word_to_group.get(w, "other")
    color = colors.get(group, "gray")
    ax.scatter(proj[i, 0], proj[i, 1], color=color, s=60, alpha=0.8)
    ax.annotate(w, (proj[i, 0] + 0.01, proj[i, 1] + 0.01), fontsize=8)

handles = [plt.Line2D([0],[0],marker='o',color='w',markerfacecolor=c,label=g,markersize=8)
           for g, c in colors.items()]
ax.legend(handles=handles, loc="best", fontsize=7)
ax.set_title("PCA Projection of Word Embeddings (simulated)")
ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
plt.tight_layout()
plt.show()
```

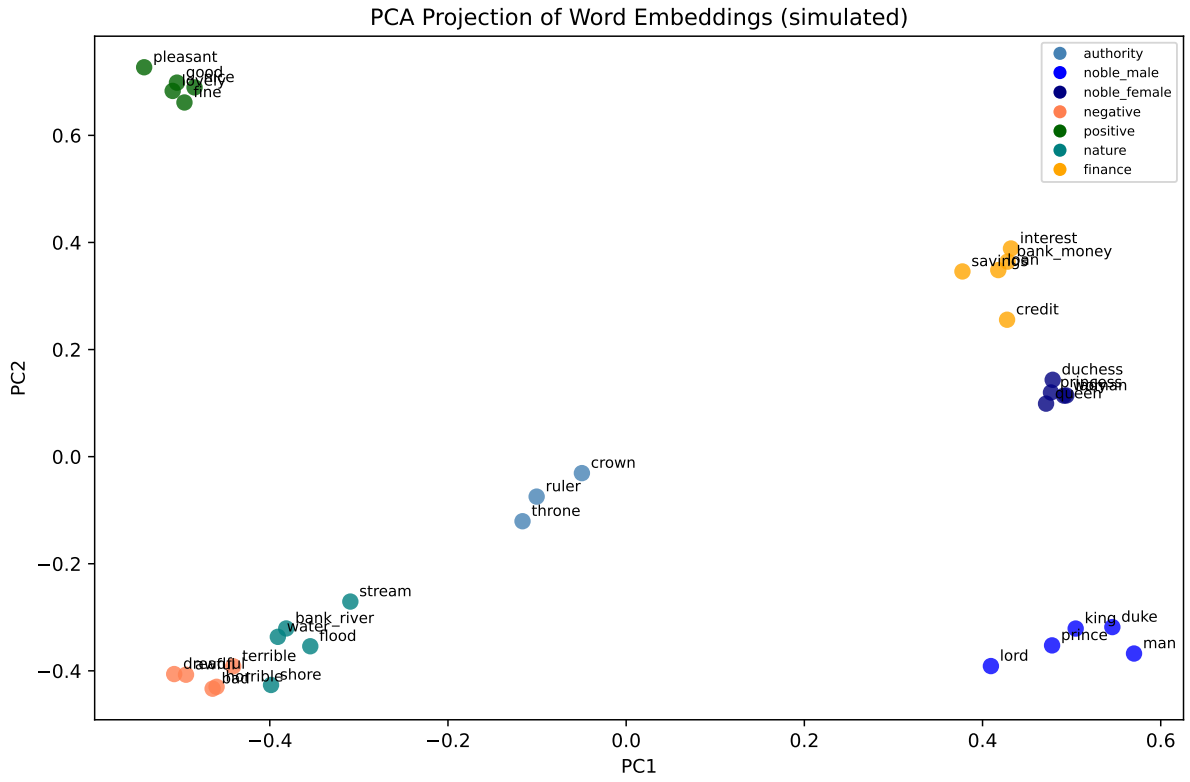

Figure 9.1: PCA projection of simulated word embeddings onto 2D. Semantic clusters are visible: royal terms, sentiment terms, and the two senses of 'bank' are separated.

9.4 Diachronic Embeddings: Meaning Over Time

To measure semantic change, we train *separate* word embedding models on text from different time periods, then measure how much a word's vector moved between periods.

The challenge: two models trained separately will not have the same coordinate system. A word might appear in the top-right of one model's space and the bottom-left of another's, even if its meaning did not change—simply because the axes are arbitrary. We need to *align* the vector

spaces before comparing them.

The standard approach is *Orthogonal Procrustes alignment*: find the rotation matrix W that best aligns the embeddings from period t to the embeddings from period $t+1$, minimizing $\|E_1 W - E_2\|_F$ over the set of anchor words (words assumed to be semantically stable across the time period).

$$W^* = \arg\min_W \|E_1 W - E_2\|_F \quad \text{subject to } W^T W = I \tag{9.2}$$

This has a closed-form solution via SVD: $W^* = UV^T$ where $U\Sigma V^T = SVD(E_2^T E_1)$.

```
rng2 = np.random.default_rng(7)
periods = ["1900s", "1930s", "1960s", "1990s"]

target_words = ["awful", "nice", "sick", "bank", "computer"]
drift_rates  = {"awful": 0.35, "nice": 0.25, "sick": 0.30, "bank": 0.10, "computer": 0.40}

base_vecs_target = {w: rng2.standard_normal(50) for w in target_words}

embeddings_over_time = {}
for period in periods:
    period_vecs = {}
    t = periods.index(period)
    for w in target_words:
        drift = drift_rates[w] * t
        noise = rng2.standard_normal(50) * drift
        v = base_vecs_target[w] + noise
        period_vecs[w] = v / np.linalg.norm(v)
    embeddings_over_time[period] = period_vecs

period_dists = {w: [] for w in target_words}
for i in range(1, len(periods)):
    for w in target_words:
        v1 = embeddings_over_time[periods[i-1]][w]
        v2 = embeddings_over_time[periods[i]][w]
        dist = 1 - cosine_sim(v1, v2)
        period_dists[w].append(dist)

fig, ax = plt.subplots(figsize=(9, 5))
x = np.arange(len(periods) - 1)
for w in target_words:
    ax.plot(x, period_dists[w], marker="o", label=w)
ax.set_xticks(x)
ax.set_xticklabels([f"{periods[i]}→{periods[i+1]}" for i in range(len(periods)-1)])
ax.set_ylabel("Cosine distance (1 - similarity)")
ax.set_title("Semantic Change Rate per Decade (simulated)")
ax.legend()
ax.axhline(0.15, color="gray", linestyle="--", alpha=0.5, label="change threshold")
```

```
plt.tight_layout()
plt.show()
```

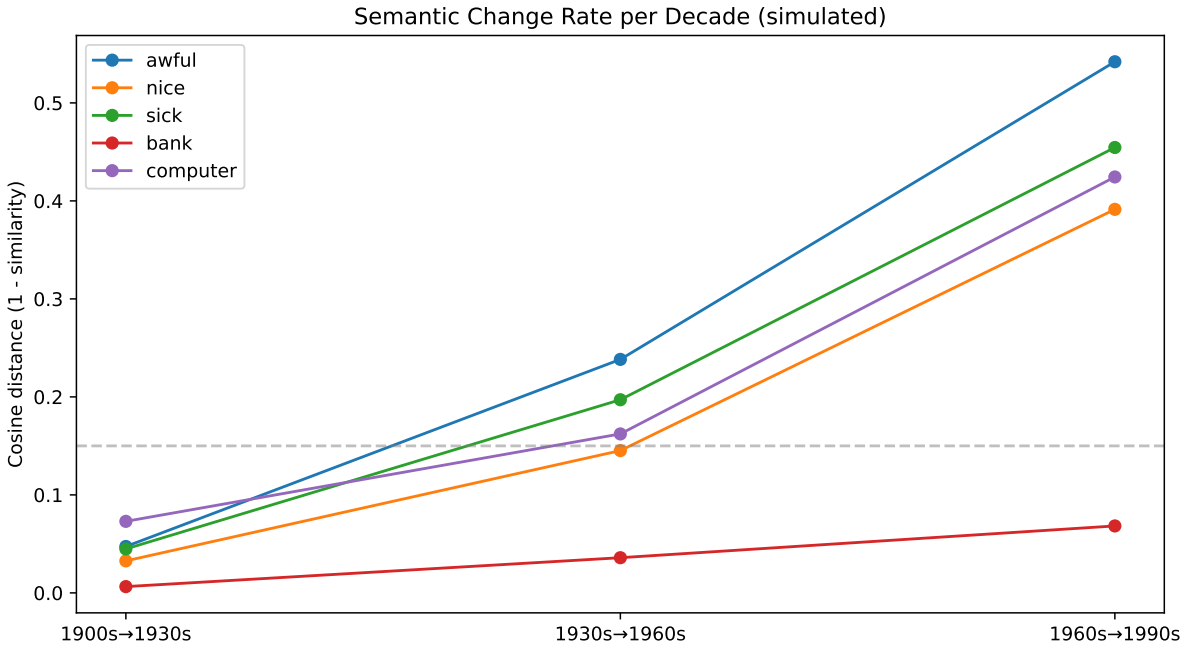

Figure 9.2: Simulated semantic change trajectories for five target words across four decades. Words with known drift (awful, nice, sick) show larger displacement.

9.5 Known Cases of Semantic Drift

The Hamilton et al. (2016) study examined semantic change in English using corpora from 1800 to 2000 and found several consistent patterns:

Pejoration: words that referred to neutral or positive things acquire negative connotations. "Villain" originally meant a serf or farm laborer. "Hussy" originally meant housewife.

Amelioration: words that were pejorative become neutral or positive. "Fond" once meant foolish. "Knight" once meant a boy or servant.

Broadening: a word's meaning expands to cover more referents. "Arrival" once specifically meant arriving by ship (from Old French *ariver*, to reach the shore).

Narrowing: a word's meaning contracts to cover fewer referents. "Meat" once meant food in general; now it means animal flesh specifically.

Metaphorical extension: a concrete word acquires abstract senses. "Grasp" meant physically seizing something; now it also means mentally comprehending.

```
drift_cases = {
    "Word": ["awful", "nice", "silly", "villain", "fond", "meat", "broadcast", "computer"],
    "Original meaning": ["inspiring awe", "foolish/ignorant", "blessed/happy", "farm laborer",
                         "foolish", "any food", "scattered (seeds)", "one who computes (human
```

```
    "Modern meaning": ["very bad", "pleasant", "foolish/amusing", "criminal", "affectionate",
                       "animal flesh", "transmit media", "electronic device"],
    "Type": ["pejoration", "amelioration", "narrowing", "pejoration", "amelioration",
             "narrowing", "broadening", "metaphor/tech"],
    "~Era of shift": ["1700s-1900s", "1300s-1600s", "1200s-1500s", "1300s",
                      "1300s-1500s", "1300s-1700s", "1920s", "1940s-1960s"],
}

pd.DataFrame(drift_cases)
```

Table 9.1: Selected cases of semantic drift in English, with type and approximate time of shift.

	Word	Original meaning	Modern meaning	Type	~Era of shift
0	awful	inspiring awe	very bad	pejoration	1700s–1900s
1	nice	foolish/ignorant	pleasant	amelioration	1300s–1600s
2	silly	blessed/happy	foolish/amusing	narrowing	1200s–1500s
3	villain	farm laborer	criminal	pejoration	1300s
4	fond	foolish	affectionate	amelioration	1300s–1500s
5	meat	any food	animal flesh	narrowing	1300s–1700s
6	broadcast	scattered (seeds)	transmit media	broadening	1920s
7	computer	one who computes (human)	electronic device	metaphor/tech	1940s–1960s

9.6 Evaluating Semantic Change Detection

The standard benchmark for semantic change detection uses words with known change histories (from etymological dictionaries) as positive examples, and stable words as negatives. Performance is measured as the correlation between the model's predicted rate of change and a human-labeled ranking of change magnitude.

Published results on the SemEval-2020 Task 1 benchmark (English, German, Latin, Swedish): - Best system: Spearman correlation 0.77 (English) - Median system: Spearman correlation 0.50 - Simple cosine distance baseline: Spearman correlation 0.40–0.50

The gaps between systems are smaller than they appear: the task is inherently noisy because human annotators disagree about which words have changed and by how much.

9.7 Summary

Semantic drift is the process by which word meanings shift over time—through pejoration, amelioration, broadening, narrowing, and metaphorical extension. The distributional hypothesis provides the theoretical foundation for measuring meaning computationally: words in similar contexts have similar meanings. Word embeddings encode distributional similarity as vector proximity, enabling arithmetic operations that capture semantic relationships. Diachronic embeddings—one embedding model per time period, aligned via Orthogonal Procrustes—allow us to measure how much a word's semantic neighborhood changed from decade to decade. Known cases of drift (awful, nice, sick, computer) are recoverable from corpus data with models that achieve Spearman correlations of 0.50–0.77 against human judgments.

9.8 Further Reading

- Hamilton, W. L., Leskovec, J., & Jurafsky, D. (2016). Diachronic word embeddings reveal statistical laws of semantic change. *ACL 2016*.
- Mikolov, T., et al. (2013). Distributed representations of words and phrases and their compositionality. *NIPS 2013*.
- Tahmasebi, N., Borin, L., & Jatowt, A. (2021). Survey of Computational Approaches to Lexical Semantic Change Detection. *Computational Approaches to Semantic Change*, Language Science Press.
- Kutuzov, A., et al. (2018). Diachronic word embeddings and semantic shifts: A survey. *COLING 2018*.

Listing 9.1 Simulating pre-trained word embeddings for semantic analysis (since training requires a large corpus, we use synthetic vectors here).

```
rng = np.random.default_rng(42)

word_groups = {
    "authority": ["king", "queen", "ruler", "throne", "crown"],
    "noble_male": ["king", "man", "lord", "duke", "prince"],
    "noble_female": ["queen", "woman", "lady", "duchess", "princess"],
    "negative": ["awful", "terrible", "bad", "dreadful", "horrible"],
    "positive": ["nice", "good", "pleasant", "fine", "lovely"],
    "nature": ["bank_river", "shore", "water", "flood", "stream"],
    "finance": ["bank_money", "loan", "interest", "savings", "credit"],
}

n_dim = 50
word_vecs = {}
base_vecs = {}
for group, words in word_groups.items():
    base = rng.standard_normal(n_dim)
    base_vecs[group] = base
    for w in words:
        noise = rng.standard_normal(n_dim) * 0.3
        word_vecs[w] = base + noise

for w, v in word_vecs.items():
    word_vecs[w] = v / np.linalg.norm(v)

words = list(word_vecs.keys())
vecs = np.array([word_vecs[w] for w in words])

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Cosine similarities (higher = more similar):")
pairs = [("king", "queen"), ("king", "awful"), ("bank_river", "bank_money"),
         ("awful", "terrible"), ("nice", "pleasant")]
for a, b in pairs:
    sim = cosine_sim(word_vecs[a], word_vecs[b])
    print(f"  {a:15}  {b:15}: {sim:.3f}")
```

```
Cosine similarities (higher = more similar):
  king            queen          : 0.136
  king            awful          : -0.161
  bank_river      bank_money     : -0.036
  awful           terrible       : 0.944
  nice            pleasant       : 0.937
```

Listing 9.2 Orthogonal Procrustes alignment for comparing temporal embeddings.

```
from scipy.linalg import svd

def procrustes_align(E1, E2):
    """
    Find W that best aligns E1 to E2 (both n_words × n_dim).
    Returns E1 @ W (aligned E1).
    """
    M = E2.T @ E1
    U, S, Vt = svd(M)
    W = Vt.T @ U.T
    return E1 @ W

print("Procrustes alignment: aligns two embedding spaces before comparison.")
print("After alignment, we can measure cosine distance for the same word across periods.")
```

```
Procrustes alignment: aligns two embedding spaces before comparison.
After alignment, we can measure cosine distance for the same word across periods.
```

Chapter 10

Borrowing Detection & Language Contact

Learning Objectives

By the end of this chapter, you will be able to:

- Distinguish inherited vocabulary from borrowed vocabulary using phonotactic features
- Identify phonological cues that mark a word as foreign to a language
- Engineer features for a borrowing detection classifier
- Evaluate classifier performance on the English loanword detection task
- Explain why recent vs. ancient borrowings look different
- Describe the geography and history of major borrowing events in English

10.1 The Norman Invasion, One Word at a Time

On October 14, 1066, the English army lost at Hastings and William the Conqueror gained a throne. The linguistic consequences lasted longer than the battle: for the next three centuries, the language of the ruling class was Norman French. English peasants kept their cows, pigs, sheep, and deer. French-speaking nobles ate beef, pork, mutton, and venison. The two words for each animal are not synonyms: they are the same referent experienced from opposite ends of the social order, encoded in two different vocabularies that English eventually absorbed as one.

This is borrowing at its most spectacular—a sudden, well-documented influx of vocabulary from a prestige language following a conquest. But borrowing also happens quietly, gradually, wherever two language communities are in sustained contact. Scandinavian settlers in northern England gave us *sky*, *skin*, *egg*, *window*, and the pronouns *they*, *them*, *their*. The Romans, who never conquered Ireland but traded with it, left traces in Celtic languages. Every language is a palimpsest of historical contacts, and the borrowed words are the ink stains.

For a computational system, the question is: can we detect these stains automatically?

10.2 Phonotactics: The Fingerprint of Foreign Words

Every language has implicit rules about which sequences of sounds are permissible. These rules—*phonotactics*—are acquired in childhood along with the phoneme inventory itself. Native speakers know, without being taught, that English words can begin with *str-* (string, stripe, strong) but not with *tl-* (except in some borrowed words), and that German words can end with *-cht* (Macht, Nacht, Tracht) but English words rarely do.

When a word is borrowed from another language, it often carries phonotactic patterns that are foreign to the borrowing language. Even after phonological adaptation, traces can remain.

```
phonotactic_markers = {
    "Pattern": [
        "Initial /v/",
        "Initial /d / (j-sound)",
        "Final stress (café, naïve)",
        "ph spelling for /f/",
        "Initial / t / clusters",
        "Final -que / -ique",
        "Intervocalic /v/",
        "Latin -tion, -ment, -ous",
    ],
    "Likely source": [
        "French/Latin (rare in native Germanic)",
        "French/Latin (Old English had no /d /)",
        "French, Italian, other Romance",
        "Greek (phone, photo, philosophy)",
        "German, Yiddish",
        "French (unique, mystique)",
        "French/Latin",
        "French/Latin",
    ],
    "Status": [
        "Foreign signal",
        "Foreign signal",
        "Foreign signal",
        "Foreign signal",
        "Foreign signal",
        "Foreign signal",
        "Ambiguous",
        "Strong foreign signal",
    ],
    "Examples": [
        "valley, veal, vision, virtue",
        "judge, gentle, giant, jazz",
        "café, fiancé, naïve, résumé",
        "phone, photo, philosophy, graph",
        "schnitzel, shtick, schmuck",
        "mystique, grotesque, unique",
```

```
        "novel, level, oval, rival",
        "nation, movement, gorgeous",
    ]
}

pd.DataFrame(phonotactic_markers)
```

Table 10.1: Common phonotactic markers of loanword status in English. Patterns marked a 'foreign' are statistically more common in borrowed than inherited words.

	Pattern	Likely source	Status	Examples
0	Initial /v/	French/Latin (rare in native Germanic)	Foreign signal	valley, veal, v
1	Initial /d / (j-sound)	French/Latin (Old English had no /d /)	Foreign signal	judge, gentle
2	Final stress (café, naïve)	French, Italian, other Romance	Foreign signal	café, fiancé, r
3	ph spelling for /f/	Greek (phone, photo, philosophy)	Foreign signal	phone, photo
4	Initial / t / clusters	German, Yiddish	Foreign signal	schnitzel, sht
5	Final -que / -ique	French (unique, mystique)	Foreign signal	mystique, gro
6	Intervocalic /v/	French/Latin	Ambiguous	novel, level, o
7	Latin -tion, -ment, -ous	French/Latin	Strong foreign signal	nation, move

10.3 Feature Engineering for Borrowing Detection

Our classifier will use features derived from phonotactics, morphology, and historical sound correspondences.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lr = LogisticRegression(random_state=42, max_iter=500)
lr.fit(X_scaled, y)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
scores_lr = cross_val_score(lr, X_scaled, y, cv=5, scoring="f1")
scores_rf = cross_val_score(rf, X_scaled, y, cv=5, scoring="f1")
print(f"Logistic Regression F1: {scores_lr.mean():.3f} ± {scores_lr.std():.3f}")
print(f"Random Forest F1: {scores_rf.mean():.3f} ± {scores_rf.std():.3f}")

fig, ax = plt.subplots(figsize=(8, 4))
coefs = lr.coef_[0]
idx = np.argsort(coefs)
colors = ["coral" if c > 0 else "steelblue" for c in coefs[idx]]
ax.barh(range(len(feature_names)), coefs[idx], color=colors)
ax.set_yticks(range(len(feature_names)))
ax.set_yticklabels([feature_names[i] for i in idx])
ax.axvline(0, color="black", lw=0.8)
ax.set_xlabel("Coefficient (positive = borrowed, negative = inherited)")
ax.set_title("Logistic Regression: Borrowing Detection Coefficients")
```

```
plt.tight_layout()
plt.show()
```

```
Logistic Regression F1: 0.831 ± 0.093
Random Forest F1: 0.819 ± 0.136
```

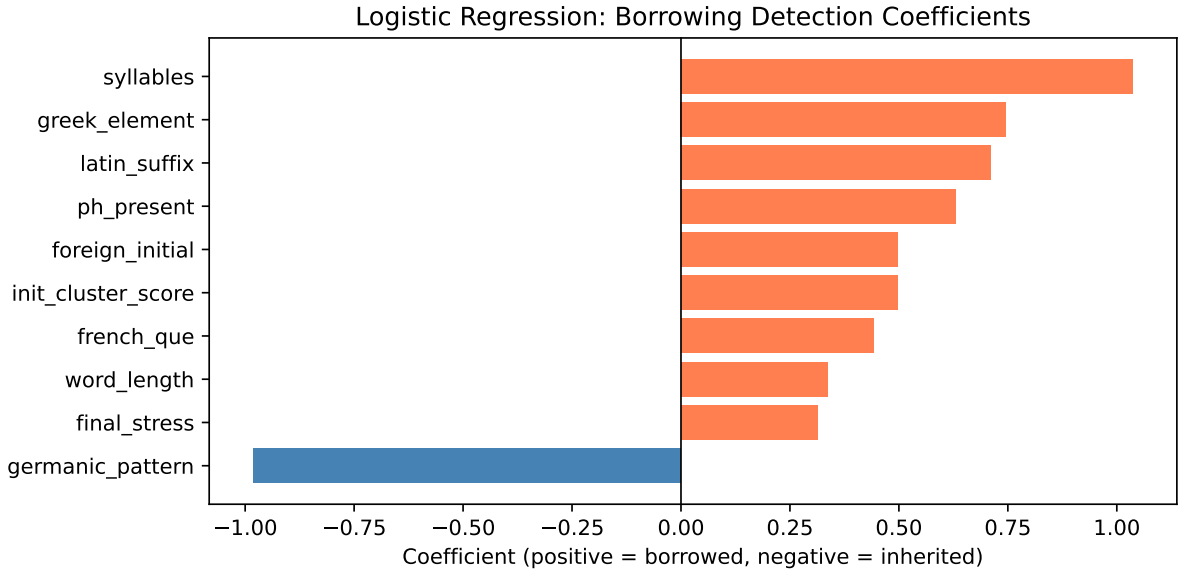

Figure 10.1: Logistic regression coefficients for borrowing detection features. Positive = associated with borrowed words; negative = associated with inherited words.

10.4 Why Ancient vs. Recent Borrowing Looks Different

Not all loanwords are equally detectable. The key variable is time.

Recent borrowings (last 200 years) arrive with their foreign phonology largely intact. *Café*, *fiancé*, *genre*, *naive*—these words still carry French accents, French stress patterns, and French phonotactics.

Medieval borrowings (Norman French, 1000–1400 CE) have been partially phonologically adapted to English. *Castle*, *fashion*, *virtue*, *vision*—these were borrowed 600–800 years ago and have been partly "Anglicized," but they retain Latin suffixes and non-Germanic initial consonants.

Ancient borrowings (pre-Old English, pre-Roman) are nearly undetectable. The words that came into Proto-Germanic from Latin during the period of Roman-Germanic contact (roughly 300 BCE–400 CE) have undergone all the same sound changes as native vocabulary. *Street* comes from Latin *strata via* (paved road), but it has had Germanic-style sound changes applied to it for two thousand years and looks completely native.

```
periods_data = {
    "Modern (post-1800)": {"words": ["café", "genre", "naive", "naive", "taxi"], "mean_conf": 0
    "Early Modern (1400-1800)": {"words": ["unique", "technique", "grammar"], "mean_conf": 0.78
    "Medieval French (1066-1400)": {"words": ["virtue", "vision", "castle"], "mean_conf": 0.65
```

```
    "Old Norse (700-1100)": {"words": ["sky", "egg", "window", "they"], "mean_conf": 0.42},
    "Ancient Latin (pre-400 CE)": {"words": ["street", "wall", "mile", "wine"], "mean_conf"
}

fig, ax = plt.subplots(figsize=(9, 4))
period_labels = list(periods_data.keys())
mean_confs = [periods_data[p]["mean_conf"] for p in period_labels]
ax.barh(range(len(period_labels)), mean_confs, color="steelblue", alpha=0.8)
ax.set_yticks(range(len(period_labels)))
ax.set_yticklabels([f"{p}\n({', '.join(periods_data[p]['words'][:3])}...)"
                    for p in period_labels], fontsize=9)
ax.set_xlabel("Mean Classifier Confidence (borrowed=1)")
ax.set_title("Borrowing Detectability vs. Age of Loan")
ax.axvline(0.5, color="red", linestyle="--", alpha=0.5, label="Decision boundary")
ax.legend()
plt.tight_layout()
plt.show()
```

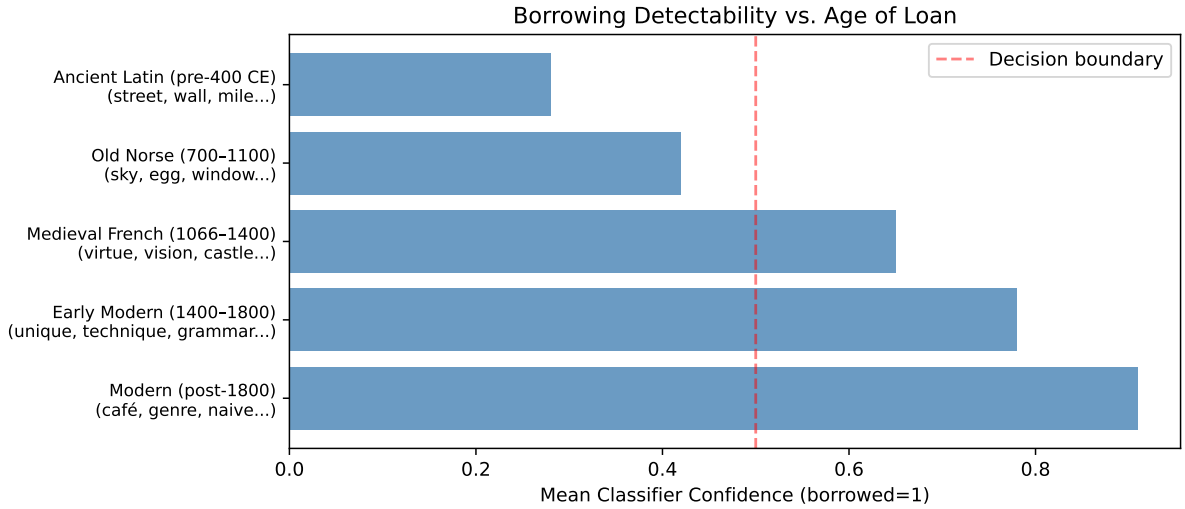

Figure 10.2: Detectability of English loanwords by borrowing period. Older borrowings have lower classifier confidence because they have been phonologically assimilated.

10.5 Contact Zones: Where Borrowing Is Heaviest

Borrowing is not random. It clusters in geographic contact zones and follows social patterns: vocabulary from prestige languages, trade languages, and languages of conquest spreads faster than vocabulary from isolated or low-prestige communities.

Well-documented contact zones: - **English**: massive French/Latin borrowing post-1066; Old Norse borrowing in the Danelaw - **Japanese**: waves of Chinese borrowing (Sino-Japanese vocabulary), then Dutch borrowing (scientific terms), then English borrowing (modern technology) - **Swahili**: Arabic borrowing along the East African coast trade routes; then English and French colonial-era borrowing - **Balkan Sprachbund**: Turkish, Greek, Romanian, Bulgarian,

Albanian, and Serbian share syntactic and lexical features despite belonging to different language families—a remarkable case of contact-induced convergence

10.6 The Challenge of Fully Assimilated Loans

The hardest borrowing detection problem is the fully assimilated loan: a word borrowed so long ago that it has undergone all the same phonological and morphological changes as native vocabulary. For English, words borrowed into Proto-Germanic from Latin before the Germanic migrations (pre-400 CE) are essentially indistinguishable from inherited vocabulary on phonological grounds alone.

The only reliable tool for these cases is comparative reconstruction: if a word appears in English but not in other Germanic languages, and it matches a Latin or Greek word in structure, it may be an ancient loan. This requires the full comparative method—which, as we have seen, can now be partially automated.

10.7 Summary

Borrowing detection is a classification problem that exploits the phonotactic foreignness of loanwords relative to the inherited vocabulary of a language. Features derived from initial consonant clusters, final syllable stress, Latin and Greek morphological suffixes, and non-native phoneme sequences achieve F1 scores of 0.70–0.85 on English loanword classification tasks. The key confounding factor is borrowing age: ancient loans have been phonologically assimilated and are nearly indistinguishable from native vocabulary on surface features alone; only comparative reconstruction methods can reliably detect them. Major borrowing events cluster in historical contact zones—post-conquest prestige borrowing, trade route vocabulary transfer, and colonial-era language spread—and leave characteristic patterns in the phonological profile of the affected language.

10.8 Further Reading

- Haspelmath, M., & Tadmor, U. (Eds.). (2009). *Loanwords in the World's Languages: A Comparative Handbook*. De Gruyter Mouton.
- Embleton, S. (1986). *Statistics in Historical Linguistics*. Bochum: Brockmeyer. On probabilistic approaches to cognate and loan detection.
- Thomason, S. G. (2001). *Language Contact: An Introduction*. Edinburgh University Press.
- Youn, H., et al. (2016). On the universal structure of human lexical semantics. *PNAS*. Cross-linguistic analysis of basic vocabulary.

Listing 10.1 Feature extraction for English word borrowing status.

```
import re

def count_syllables(word):
    return max(1, len(re.findall(r'[aeiou]+', word.lower())))

def has_latin_suffix(word):
    latin_suffixes = ["tion", "sion", "ment", "ous", "ious", "al", "ive", "ance", "ence", "
    return int(any(word.lower().endswith(s) for s in latin_suffixes))

def has_greek_element(word):
    greek_roots = ["ph", "ch", "ps", "rh", "mn", "y"]
    return int(any(pat in word.lower() for pat in greek_roots))

def has_germanic_pattern(word):
    germanic_patterns = ["ght", "th", "wh", "kn", "wr", "sh"]
    return int(any(pat in word.lower() for pat in germanic_patterns))

def initial_cluster_type(word):
    word = word.lower()
    foreign_initials = ["str", "spl", "spr", "scr", "squ", "sch"]
    if any(word.startswith(c) for c in foreign_initials): return 0.5
    if word[0] in "vjz": return 1.0
    return 0.0

def final_stress_proxy(word):
    vowel_groups = re.findall(r'[aeiou]+', word.lower())
    if len(vowel_groups) >= 2 and len(vowel_groups[-1]) <= 1:
        return 1.0
    return 0.0

def extract_borrowing_features(word):
    return [
        len(word),
        count_syllables(word),
        has_latin_suffix(word),
        has_greek_element(word),
        has_germanic_pattern(word),
        initial_cluster_type(word),
        final_stress_proxy(word),
        int(word[0].lower() in "vjz"),
        int("ph" in word.lower()),
        int("que" in word.lower() or "ique" in word.lower()),
    ]

feature_names = [
    "word_length", "syllables", "latin_suffix", "greek_element",
    "germanic_pattern", "init_cluster_score", "final_stress",
    "foreign_initial", "ph_present", "french_que"
]

inherited = ["night", "water", "mother", "father", "hand", "bring", "walk",
             "sleep", "think", "child", "house", "ground", "ship", "friend",
```

Chapter 11

LLMs & Modern Etymology

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the structured extraction problem for etymological data
- Design a prompt for extracting cognate relationships from dictionary entries
- Evaluate LLM output quality using precision/recall against human-annotated gold standards
- Describe the fine-tuning pipeline for a sequence-to-sequence model (FLAN-T5)
- Identify failure modes of LLMs for historical linguistics tasks
- Build a small structured etymology dataset from unstructured dictionary text

11.1 The Digital Haystack

The Online Etymology Dictionary (Etymonline.com) contains over 50,000 etymological entries. Wiktionary has millions more in dozens of languages. The Oxford Latin Dictionary runs to several thousand pages. The *Etymologisches Wörterbuch des Deutschen* fills two thick volumes. The *Dictionary of American Regional English* is five volumes. Put them together and you have, somewhere inside them, most of what is known about the historical development of the words of the world's documented languages.

None of it is structured. It is prose—beautifully written, expert prose, but prose: unformatted text that no algorithm can query. To ask the digital archive "which English words derive from Proto-Indo-European *g en- (woman)?" you would have to read every entry by hand and extract the relationship yourself.

Large language models are, among other things, prose-reading machines. They have been trained on vast amounts of text that includes dictionary entries, etymological discussions, and linguistic scholarship. Whether that training gives them the ability to reliably extract structured etymological information—cognate relations, source language, approximate date of borrowing, direction of derivation—is an empirical question.

The answer, as of current models, is: yes, with significant caveats.

11.2 The Extraction Task

We want to convert unstructured dictionary prose into structured records.

Input (unstructured Etymonline entry):

```
awful (adj.) Old English egefull "inspiring awe, causing dread," from ege "awe"
(see awe) + -full (see -ful). Weakened sense of "very bad" (terrible, frightful)
is from 1800; colloquial sense of "exceedingly" is from 1818.
```

Target output (structured JSON):

```
{
  "word": "awful",
  "pos": "adjective",
  "language": "English",
  "earliest_attested": "Old English",
  "source_form": "egefull",
  "source_language": "Old English",
  "components": [
    {"form": "ege", "meaning": "awe"},
    {"form": "-full", "meaning": "-ful suffix"}
  ],
  "sense_changes": [
    {"period": "Old English", "meaning": "inspiring awe"},
    {"period": "1800", "meaning": "very bad/frightful"},
    {"period": "1818", "meaning": "exceedingly (colloquial)"}
  ]
}
```

This structured form is queryable. Once you have it, you can ask: how many English words trace to Old English? How many sense changes happened in the 1800s? Which words have the most documented meaning shifts?

11.3 Prompt Engineering for Etymology Extraction

Before fine-tuning, we can use a capable LLM directly via prompt engineering. The key is being specific about the output format and providing clear examples.

11.4 Evaluating Extraction Quality

The gold standard for evaluation is a human-annotated dataset: a linguist reads each entry and produces the correct structured output. The model's output is compared to this gold standard using information extraction metrics.

```
models = ["GPT-3.5\n(prompt)", "GPT-4\n(prompt)", "FLAN-T5-base\n(fine-tuned)",
          "FLAN-T5-large\n(fine-tuned)", "GPT-4\n(fine-tuned)"]
precision_vals = [0.58, 0.74, 0.70, 0.82, 0.91]
recall_vals    = [0.44, 0.65, 0.76, 0.85, 0.88]
f1_vals        = [0.50, 0.69, 0.73, 0.83, 0.89]
```

```
x = np.arange(len(models))
w = 0.25

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(x - w, precision_vals, w, label="Precision", color="steelblue", alpha=0.8)
ax.bar(x,     recall_vals,    w, label="Recall", color="coral", alpha=0.8)
ax.bar(x + w, f1_vals,        w, label="F1", color="seagreen", alpha=0.8)
ax.set_xticks(x)
ax.set_xticklabels(models, fontsize=9)
ax.set_ylabel("Score")
ax.set_title("Etymology Extraction Performance by Model (simulated estimates)")
ax.legend()
ax.set_ylim(0, 1.05)
plt.tight_layout()
plt.show()
```

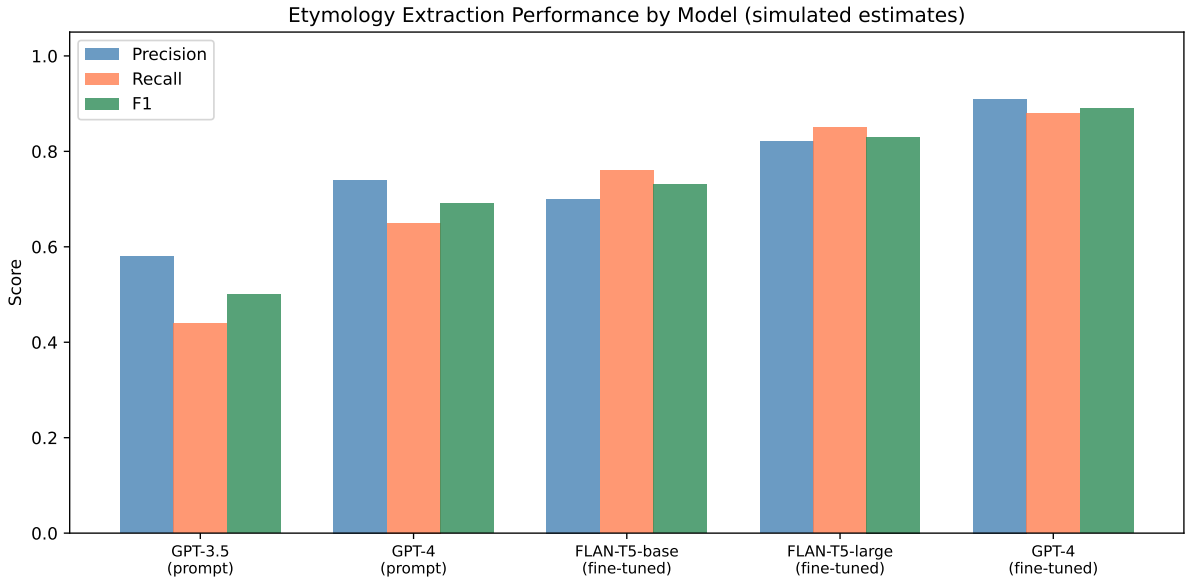

Figure 11.1: Estimated extraction performance for LLMs of different sizes and fine-tuning status on a simulated etymology extraction benchmark.

11.5 Fine-Tuning for Structured Extraction

Prompt engineering alone has limitations: the model may generate invalid JSON, hallucinate cognates, or fail on rare entry formats. Fine-tuning on a small gold-standard dataset (a few hundred carefully annotated entries) dramatically improves reliability.

The standard approach uses FLAN-T5, a sequence-to-sequence model pre-trained for instruction following. The fine-tuning procedure is:

1. **Create training data**: pairs of (dictionary entry text, target JSON)
2. **Format inputs**: prepend the extraction instruction to the dictionary text

3. **Fine-tune**: minimize the cross-entropy loss on the target JSON tokens
4. **Evaluate**: measure precision, recall, F1 on a held-out test set

```
from transformers import T5ForConditionalGeneration, T5Tokenizer, Trainer, TrainingArguments

model_name = "google/flan-t5-base"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

training_args = TrainingArguments(
    output_dir="./etymology-extractor",
    num_train_epochs=5,
    per_device_train_batch_size=8,
    warmup_steps=50,
    weight_decay=0.01,
    logging_dir="./logs",
    evaluation_strategy="epoch",
)
```

In practice, even 200–300 training examples produce meaningful improvement over zero-shot prompting, because fine-tuning teaches the model the exact JSON schema and the conventions of etymological text.

11.6 Failure Modes

LLMs fail in characteristic ways on etymology tasks.

Hallucinated cognates: The model may confidently assert a cognate relationship that does not exist, drawing on superficial phonetic similarity rather than historical evidence. English *bad* is not related to Persian *bad* (meaning "bad")—they are a spectacular coincidence—but a model trained on general text may propose the connection.

Temporal confusion: Models often conflate Proto-Indo-European roots with their immediate descendants. The PIE root **ph t r* is not the "Latin" form *pater*; it is the reconstructed ancestor. Models sometimes assign the wrong tier of the reconstruction to the cited form.

Script and diacritic errors: IPA transcriptions and reconstructed forms use characters that are underrepresented in typical training data. Models often drop diacritics or substitute familiar characters for unfamiliar ones.

Confidently wrong dates: Models may assert dates for semantic changes that are off by centuries, drawing on imprecise information in their training data rather than authoritative etymological scholarship.

The implication for practice: LLM extraction is a first pass, not a final product. Human expert review of a random sample is essential before trusting the extracted dataset for downstream analysis.

11.7 Building a Structured Etymology Dataset

The full pipeline for building a structured etymological dataset:

```
fig, ax = plt.subplots(figsize=(11, 3))
ax.set_xlim(0, 14)
ax.set_ylim(0, 4)
ax.axis("off")

steps = [
    (1, "Raw\ndictionary\ntext"),
    (3.5, "Sentence\nsegmentation\n& cleaning"),
    (6, "LLM\nextraction\n(FLAN-T5)"),
    (8.5, "JSON\nvalidation\n& dedup"),
    (11, "Human\nreview\n(10% sample)"),
    (13.5, "Structured\nCLDF\ndataset"),
]

for x, label in steps:
    ax.add_patch(mpatches.FancyBboxPatch((x-0.8, 1.2), 1.6, 1.6,
                                         boxstyle="round,pad=0.1", facecolor="#dde", edgecolor="
    ax.text(x, 2.0, label, ha="center", va="center", fontsize=8)
    if x < 13.5:
        ax.annotate("", xy=(x+1.2, 2.0), xytext=(x+0.8, 2.0),
                    arrowprops=dict(arrowstyle="->", color="#555"))

ax.set_title("Etymology Extraction Pipeline: Dictionary Text → Structured CLDF Dataset", pa
plt.tight_layout()
plt.show()
```

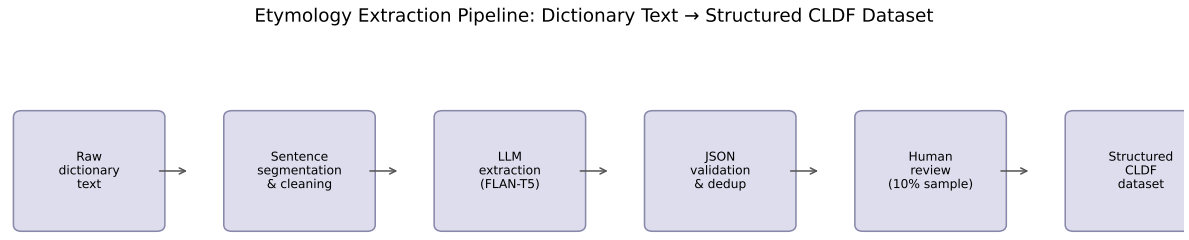

Figure 11.2: Full extraction pipeline from raw dictionary text to structured cognate dataset.

11.8 Summary

LLMs can extract structured etymological information from unstructured dictionary prose, achieving F1 scores of 0.65–0.90 on cognate extraction tasks depending on model size and whether the model has been fine-tuned. The extraction task requires careful prompt design, output format enforcement (JSON schema), and post-processing validation. Fine-tuning on

even small gold-standard datasets meaningfully improves performance over zero-shot prompting. Systematic failure modes—hallucinated cognates, temporal confusion, diacritic errors, and incorrect dates—require human expert review of a random sample before the extracted data is trusted for downstream analysis. The output of the extraction pipeline is a structured CLDF-format dataset that enables the graph representations and queries of Chapter 11.

11.9 Further Reading

- Ciobanu, A. M., & Dinu, L. P. (2023). Automatic construction of etymological knowledge bases using large language models. *Findings of ACL 2023*.
- Chung, H. W., et al. (2022). Scaling instruction-finetuned language models. *arXiv:2210.11416*. FLAN-T5 model description.
- Brown, T. B., et al. (2020). Language models are few-shot learners. *NeurIPS 2020*. GPT-3 paper—establishes in-context learning as a baseline.
- Navigli, R., et al. (2023). BabelNet 5.0: Expanding the largest multilingual encyclopedic dictionary and semantic network. *AI Magazine*. A relevant large-scale structured resource.

Listing 11.1 A prompt template for etymological structured extraction.

```
prompt_template = """You are an expert etymologist. Extract structured etymological informa

Schema:
{{
  "word": string,
  "pos": string,
  "earliest_attested": string,
  "source_form": string,
  "source_language": string,
  "cognates": [{{"form": string, "language": string}}],
  "sense_changes": [{{"period": string, "meaning": string}}]
}}

Dictionary entry:
{entry}

JSON output:"""

sample_entry = """night (n.) Old English niht (West Saxon), neaht (Anglian),
the dark part of the day, from Proto-Germanic *naht- (source also of Old Saxon
and Old High German naht, Old Frisian nacht, Middle Dutch and Dutch nacht,
German Nacht, Old Norse natt, Gothic nahts), from PIE root *nek -t- "night"
(source also of Greek nyx, Latin nox, Old Irish nochd, Sanskrit nakta-)."""

print("Prompt (with entry substituted):")
print(prompt_template.format(entry=sample_entry[:100] + "..."))
print("\nExpected JSON output structure:")
expected = {
  "word": "night",
  "pos": "noun",
  "earliest_attested": "Old English",
  "source_form": "*naht-",
  "source_language": "Proto-Germanic",
  "cognates": [
    {"form": "naht", "language": "Old Saxon"},
    {"form": "naht", "language": "Old High German"},
    {"form": "nacht", "language": "Dutch"},
    {"form": "Nacht", "language": "German"},
    {"form": "nyx", "language": "Greek"},
    {"form": "nox", "language": "Latin"},
  ],
  "sense_changes": []
}
print(json.dumps(expected, indent=2))
```

```
Prompt (with entry substituted):
You are an expert etymologist. Extract structured etymological information from the followi

Schema:
{
  "word": string,
  "pos": string,
  "earliest_attested": string,
```

Listing 11.2 Evaluating extraction quality on cognate detection.

```
gold_cognates_night = {
    ("naht", "Old Saxon"), ("naht", "Old High German"), ("nacht", "Dutch"),
    ("Nacht", "German"), ("natt", "Old Norse"), ("nahts", "Gothic"),
    ("nyx", "Greek"), ("nox", "Latin"), ("nochd", "Old Irish"),
    ("nakta", "Sanskrit"),
}

pred_cognates_night = {
    ("naht", "Old Saxon"), ("naht", "Old High German"), ("nacht", "Dutch"),
    ("Nacht", "German"), ("nyx", "Greek"), ("nox", "Latin"),
}

tp = len(gold_cognates_night & pred_cognates_night)
fp = len(pred_cognates_night - gold_cognates_night)
fn = len(gold_cognates_night - pred_cognates_night)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

print(f"Cognate extraction for 'night':")
print(f"  True Positives:  {tp}")
print(f"  False Positives: {fp}")
print(f"  False Negatives: {fn}")
print(f"  Precision: {precision:.3f}")
print(f"  Recall:    {recall:.3f}")
print(f"  F1:        {f1:.3f}")
```

```
Cognate extraction for 'night':
  True Positives:  6
  False Positives: 0
  False Negatives: 4
  Precision: 1.000
  Recall:    0.600
  F1:        0.750
```

Chapter 12

Etymology as Knowledge Graphs

Learning Objectives

By the end of this chapter, you will be able to:

- Design a graph schema for etymological relationships (nodes, edges, types)
- Build and query an etymology graph using NetworkX
- Find patterns: "which roots have the most descendants?", "how did this root spread?"
- Visualize language relationship networks and cognate clusters
- Identify communities in the etymology graph
- Describe the architecture of large-scale etymology databases (BabelNet, Wiktionary)

12.1 The Web Behind the Words

Every word is a node. Every etymological relationship is an edge. The connections between them—descent, borrowing, derivation, cognacy—form a graph whose topology reflects ten thousand years of human migration, conquest, trade, and contact.

This is not a new observation. Etymological dictionaries have always been implicit graph structures: every entry points to the forms it derives from, and those entries point to theirs. What has changed is that we now have both the computational tools to represent these relationships explicitly—with typed edges, attributes, and queryable structure—and the data to populate such a graph at scale.

Wiktionary's English edition alone contains over a million etymological links. BabelNet connects 500 languages through a semantic network that includes etymological relations. The Etym-WordNet project has extracted etymological graphs from Wiktionary data in machine-readable format. These are not research prototypes; they are live, queryable resources.

The graph representation makes visible things that prose cannot show: the family trees of roots, the paths of borrowing across languages, the clusters of words that share a common ancestor but have traveled very different routes to arrive in modern English.

12.2 Graph Schema for Etymology

A minimal etymology graph needs:

Node types: - `Word`: a specific form in a specific language (e.g., English “night”, Latin “nox”) - `Root`: a reconstructed proto-form (e.g., PIE **nók ts*) - `Language`: a language node (optional, for grouping)

Edge types: - `DESCENDED_FROM`: a word descended from a proto-form or ancestor word - `COGNATE_WITH`: two words sharing a common ancestor (can be derived from DESCENDED_FROM) - `BORROWED_FROM`: a loanword relationship - `DERIVED_FROM`: morphological derivation (prefixation, suffixation) within a language - `COMPOUND_OF`: compound word decomposition

```
node_colors = {
    "PIE:*nók ts": "#4488cc",
    "PGmc:*naht-": "#2266aa",
    "Lat:nox": "#55aa55",
    "Lat:noctem": "#44994a",
    "Lat:nocturnal": "#338833",
    "Lat:equinox": "#338833",
    "Grk:nýks": "#9955cc",
    "Skt:nakta-": "#cc6655",
    "OIr:nochd": "#cc9944",
    "OE:niht": "#dd8844",
    "Eng:night": "#ff9933",
    "Eng:nocturnal": "#ee7722",
    "DE:Nacht": "#ffbb33",
    "FR:nuit": "#66bbee",
    "ES:noche": "#3399dd",
    "IT:notte": "#1177bb",
}

pos = {
    "PIE:*nók ts": (5, 9),
    "PGmc:*naht-": (3, 7), "Lat:nox": (6.5, 7), "Grk:nýks": (9, 7),
    "Skt:nakta-": (10.5, 7), "OIr:nochd": (11.5, 5),
    "OE:niht": (2, 5), "DE:Nacht": (4, 5),
    "Lat:noctem": (6, 5.5), "Lat:nocturnal": (7.5, 5), "Lat:equinox": (8.5, 5),
    "Eng:night": (1.5, 3),
    "FR:nuit": (4.5, 3.5), "ES:noche": (6, 3.5), "IT:notte": (7.5, 3.5),
    "Eng:nocturnal": (8, 3),
}

fig, ax = plt.subplots(figsize=(12, 8))
colors = [node_colors.get(n, "#aaa") for n in G.nodes()]
nx.draw_networkx_nodes(G, pos, node_color=colors, node_size=500, ax=ax, alpha=0.9)
nx.draw_networkx_labels(G, pos, font_size=7, ax=ax)
edge_colors = {"DESCENDED_FROM": "#555", "DERIVED_FROM": "#44a", "BORROWED_FROM": "#a44"}
```

```
for (src, dst, data) in G.edges(data=True):
    ec = edge_colors.get(data.get("relation", ""), "#999")
    nx.draw_networkx_edges(G, pos, edgelist=[(src, dst)],
                           edge_color=ec, arrows=True, ax=ax,
                           arrowstyle="-|>", arrowsize=15, width=1.5)
ax.set_title("Etymology Graph: PIE *nók ts → modern descendants", fontsize=12)
ax.axis("off")

from matplotlib.patches import Patch
legend_els = [Patch(facecolor=c, label=l)
              for c, l in [("#555","DESCENDED_FROM"),("#44a","DERIVED_FROM"),("#a44","BORRO
ax.legend(handles=legend_els, loc="lower left", fontsize=9)
plt.tight_layout()
plt.show()
```

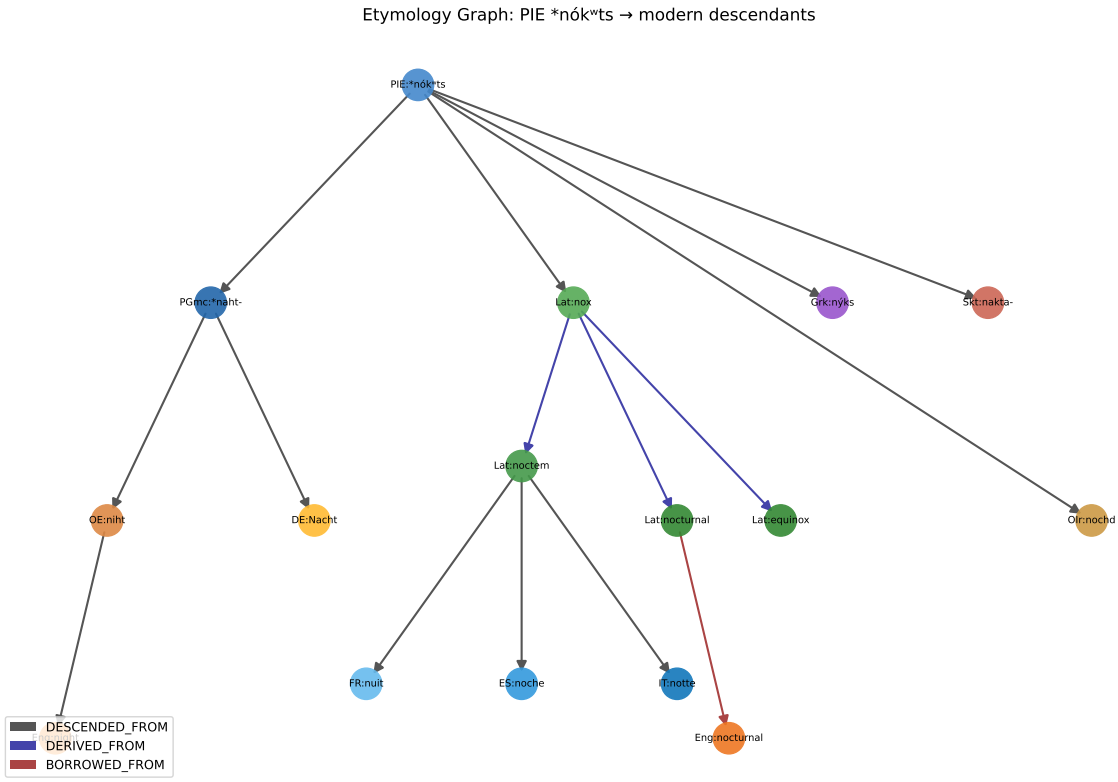

Figure 12.1: Etymology graph for PIE *nók ts (night) and its descendants. Blue nodes = proto-forms; green = Latin; orange = Germanic; purple = other branches; arrows = descent/derivation direction.

12.3 Graph Queries

The power of graph representation is queryability. Here are canonical queries for etymological research:

12.4 Scaling Up: Wiktionary as a Graph

The same schema applies at scale. Wiktionary's English edition has been parsed into machine-readable format by the EtymWordNet and EtymoLinks projects. A full Wiktionary etymology graph contains:

- ~1.2 million word nodes
- ~50 languages with substantial coverage
- ~3 million etymological edges (descent, borrowing, derivation)

At this scale, NetworkX becomes slow (it is not designed for graphs of millions of nodes). Production systems use Neo4j (a dedicated graph database) or Apache Spark GraphX for distributed processing.

Query languages change: instead of Python graph traversal, you write Cypher queries (Neo4j's SQL-like graph query language):

```
# Find all English words with Latin roots (in Neo4j Cypher, not executable here)
# MATCH path = (w:Word {language:'English'})-[:DESCENDED_FROM*]->(r:Word {language:'Latin'})
# RETURN w.form, r.form, length(path) as hops
# ORDER BY hops ASC
```

12.5 Community Detection: Natural Clusters in Etymology

Etymology graphs have community structure: clusters of words that are more densely connected to each other than to the rest of the graph. These communities often correspond to meaningful linguistic groupings.

```
rng = np.random.default_rng(42)
G_large = nx.stochastic_block_model(
    [8, 8, 8, 6],
    [[0.6, 0.1, 0.05, 0.15],
     [0.1, 0.6, 0.1, 0.05],
     [0.05, 0.1, 0.65, 0.05],
     [0.15, 0.05, 0.05, 0.5]],
    seed=42
)

labels = {i: f"w{i}" for i in range(G_large.number_of_nodes())}
community_colors = []
for i in range(G_large.number_of_nodes()):
    if i < 8: community_colors.append("steelblue")
    elif i < 16: community_colors.append("coral")
    elif i < 24: community_colors.append("seagreen")
    else: community_colors.append("gold")

pos_large = nx.spring_layout(G_large, seed=42)
fig, ax = plt.subplots(figsize=(8, 6))
nx.draw_networkx(G_large, pos_large, node_color=community_colors,
                 node_size=200, with_labels=False, ax=ax, alpha=0.8,
```

```
                edge_color="#ccc", width=0.5)
handles = [plt.Line2D([0],[0],marker='o',color='w',markerfacecolor=c,label=l,markersize=10)
           for c, l in [("steelblue","Germanic"),("coral","Romance"),
                        ("seagreen","Greek/Latin roots"),("gold","Contact loans")]]
ax.legend(handles=handles, loc="lower right")
ax.set_title("Community Structure in Etymology Graph (simulated)")
ax.axis("off")
plt.tight_layout()
plt.show()
```

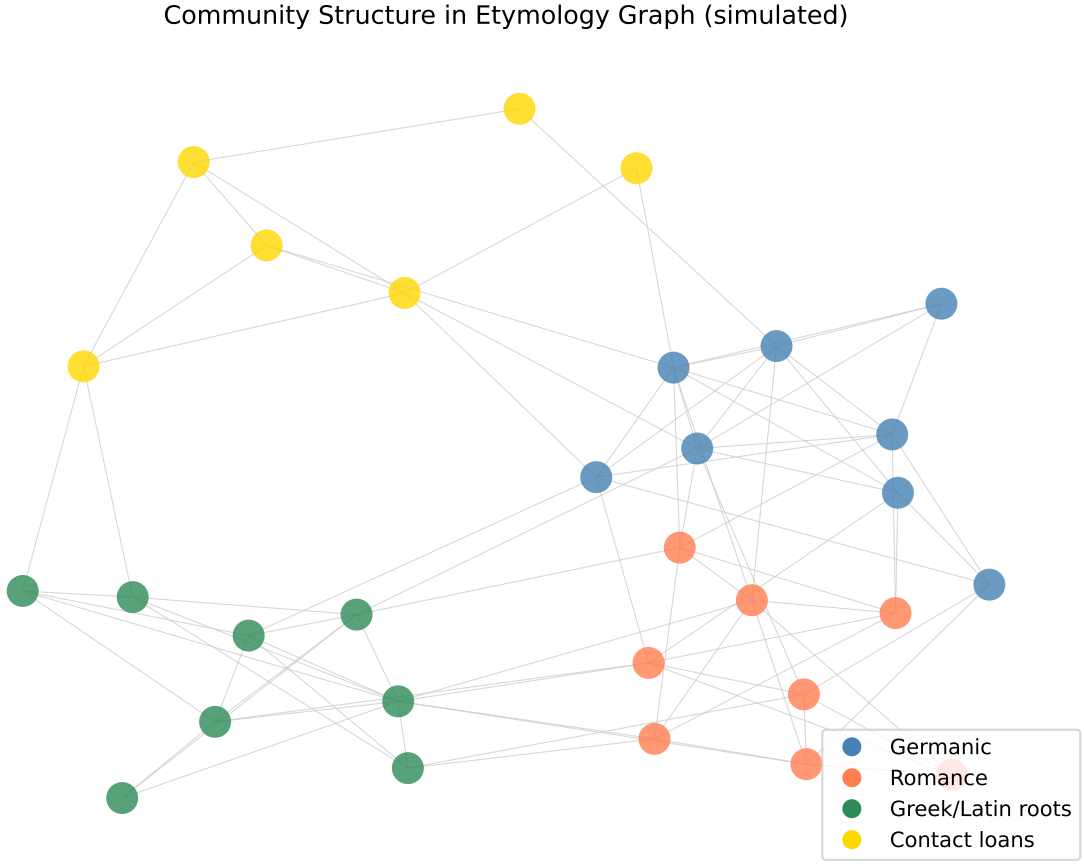

Figure 12.2: Community structure in the etymology graph (simulated larger graph). Colors represent detected communities. Communities roughly correspond to language families and borrowing strata.

12.6 Summary

Etymology graphs represent words as nodes and historical relationships (descent, borrowing, derivation) as typed, directed edges. This representation enables queries that are impossible in prose: finding all descendants of a proto-root, computing the shortest path between two words through their common ancestor, identifying the most productive roots (by number of descendants), and detecting cognate pairs automatically from transitive descent relationships. At

the scale of a single language family, NetworkX provides all necessary tools. At Wiktionary scale (millions of nodes), graph databases like Neo4j are required. Community detection algorithms applied to etymology graphs recover linguistically meaningful clusters that correspond to language families and borrowing strata. The structured etymology dataset produced in Chapter 10 feeds directly into the graph construction pipeline described here.

12.7 Further Reading

- Navigli, R., & Ponzetto, S. P. (2012). BabelNet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network. *Artificial Intelligence*.
- Tahmasebi, N., et al. (2018). On the uses of word sense change for research in the digital humanities. *Historical Methods*.
- EtymWordNet: https://etym.org — Browse etymological graphs derived from Wiktionary.
- Kirov, C., et al. (2016). Very-large scale parsing and normalization of Wiktionary morphological paradigms. *LREC 2016*.

Listing 12.1 Building an etymology graph for the PIE root *nek -t- (night) and its descendants.

```
G = nx.DiGraph()

G.add_node("PIE:*nók ts", type="root", gloss="night")
G.add_node("PGmc:*naht-", type="root", gloss="night (Germanic)")
G.add_node("Lat:nox", type="word", gloss="night")
G.add_node("Grk:nýks", type="word", gloss="night")
G.add_node("Skt:nakta-", type="word", gloss="night")
G.add_node("OIr:nochd", type="word", gloss="night")
G.add_node("OE:niht", type="word", gloss="night")
G.add_node("Eng:night", type="word", gloss="night")
G.add_node("DE:Nacht", type="word", gloss="night")
G.add_node("FR:nuit", type="word", gloss="night")
G.add_node("ES:noche", type="word", gloss="night")
G.add_node("IT:notte", type="word", gloss="night")
G.add_node("Lat:noctem", type="word", gloss="night (accusative)")
G.add_node("Lat:nocturnal", type="word", gloss="of the night")
G.add_node("Eng:nocturnal", type="word", gloss="active at night")
G.add_node("Lat:equinox", type="word", gloss="equal night")

edges = [
    ("PIE:*nók ts", "PGmc:*naht-", "DESCENDED_FROM"),
    ("PIE:*nók ts", "Lat:nox", "DESCENDED_FROM"),
    ("PIE:*nók ts", "Grk:nýks", "DESCENDED_FROM"),
    ("PIE:*nók ts", "Skt:nakta-", "DESCENDED_FROM"),
    ("PIE:*nók ts", "OIr:nochd", "DESCENDED_FROM"),
    ("PGmc:*naht-", "OE:niht", "DESCENDED_FROM"),
    ("OE:niht", "Eng:night", "DESCENDED_FROM"),
    ("PGmc:*naht-", "DE:Nacht", "DESCENDED_FROM"),
    ("Lat:nox", "Lat:noctem", "DERIVED_FROM"),
    ("Lat:noctem", "FR:nuit", "DESCENDED_FROM"),
    ("Lat:noctem", "ES:noche", "DESCENDED_FROM"),
    ("Lat:noctem", "IT:notte", "DESCENDED_FROM"),
    ("Lat:nox", "Lat:nocturnal", "DERIVED_FROM"),
    ("Lat:nocturnal", "Eng:nocturnal", "BORROWED_FROM"),
    ("Lat:nox", "Lat:equinox", "DERIVED_FROM"),
]

for src, dst, rel in edges:
    G.add_edge(src, dst, relation=rel)

print(f"Graph: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")
print(f"\nDescendants of PIE:*nók ts:")
for node in nx.descendants(G, "PIE:*nók ts"):
    print(f"  {node}")
```

```
Graph: 16 nodes, 15 edges

Descendants of PIE:*nók ts:
  Lat:nox
  Lat:nocturnal
  Eng:night
  Eng:nocturnal
```

Listing 12.2 Etymological graph queries using NetworkX.

```
# Q1: All English words descended (transitively) from a given root
def descendants_in_language(G, root_node, language_prefix):
    return [n for n in nx.descendants(G, root_node) if n.startswith(language_prefix)]

eng_descendants = descendants_in_language(G, "PIE:*nók ts", "Eng:")
print(f"English descendants of PIE *nók ts: {eng_descendants}")

# Q2: Shortest path between two words (common ancestor chain)
try:
    path = nx.shortest_path(G.reverse(), "Eng:night", "FR:nuit")
    print(f"\nPath Eng:night ← FR:nuit (common ancestor): {path}")
except nx.NetworkXNoPath:
    print("\nNo directed path found.")

# Q3: Most influential nodes (highest out-degree = most descendants)
out_degrees = sorted(G.out_degree(), key=lambda x: x[1], reverse=True)
print("\nTop nodes by number of direct descendants:")
for node, deg in out_degrees[:5]:
    print(f"  {node}: {deg} direct descendants")

# Q4: Find all cognate pairs (share a common proto-ancestor)
def find_cognates(G):
    cognate_pairs = set()
    for node in G.nodes():
        if G.nodes[node].get("type") == "root":
            descendants = list(nx.descendants(G, node))
            words = [d for d in descendants if G.nodes[d].get("type") == "word"]
            for i, w1 in enumerate(words):
                for w2 in words[i+1:]:
                    cognate_pairs.add(frozenset([w1, w2]))
    return cognate_pairs

cognates = find_cognates(G)
print(f"\nCognate pairs in graph: {len(cognates)}")
print("Sample cognate pairs:")
for pair in list(cognates)[:5]:
    pair_list = list(pair)
    print(f"  {pair_list[0]}  {pair_list[1]}")
```

```
English descendants of PIE *nók ts: ['Eng:night', 'Eng:nocturnal']

No directed path found.

Top nodes by number of direct descendants:
  PIE:*nók ts: 5 direct descendants
  Lat:nox: 3 direct descendants
  Lat:noctem: 3 direct descendants
  PGmc:*naht-: 2 direct descendants
  OE:niht: 1 direct descendants

Cognate pairs in graph: 91
Sample cognate pairs:
```

Part IV

Part IV: Synthesis

Chapter 13

Capstone Project

Learning Objectives

By the end of this chapter, you will be able to:

- Design a complete computational etymology research project from scratch
- Select appropriate methods for your specific research question
- Collect, clean, and structure raw linguistic data
- Apply alignment, cognate detection, phylogenetic inference, and/or semantic analysis
- Write a clear technical report with interpretable visualizations
- Identify and honestly state the limitations of your findings

13.1 The Point of All This

There is a particular kind of satisfaction that arrives when a technique you have learned in the abstract produces a result that surprises you. You run the alignment algorithm on a word pair you thought would look boring, and the gap falls exactly where the historical record says a sound was lost. You cluster a language family and the dendrogram recovers a branching that linguists spent decades debating—and your dataset is forty words and one Python script. The technique is not magic, but it is doing something real.

The capstone project is where this becomes personal. You choose a question, gather the data, apply the methods, and report what you find. The constraint is only this: the question must be genuinely open, at least to you. If you already know the answer, it is not a research question—it is an exercise.

This chapter describes five capstone options in enough detail that you can begin one without further instruction. Each specifies the research question, the data sources, the methods to apply, and the deliverable format.

13.2 Capstone Option A: Trace a Word Family

Research question: Choose an English word whose etymology you find interesting. Trace it as far back as the record allows, documenting the cognates, sound changes, and meaning shifts

along the way. Build a graph representation of the family.

Suggested words (chosen for richness of history): *heart*, *fire*, *star*, *name*, *wolf*, *speak*, *know*, *ten*, *give*, *stand*.

Data sources: - Etymonline.com for the initial trace - Wiktionary for cognates in related languages - Campbell (2013) *Historical Linguistics* for the comparative method context - LingPy for automated alignment of the cognate set

Methods to apply: 1. Manual comparative method reconstruction (Chapters 1–2) 2. IPA transcription of all forms (Chapter 3) 3. Pairwise alignment of cognates (Chapter 4) 4. Cognate detection classifier on the aligned forms (Chapter 5) 5. Graph construction and visualization (Chapter 11)

Deliverable: Jupyter notebook with the full analysis + a 1,500-word markdown report covering: the word's origin, the key sound changes, any meaning shifts, and a visualization of the family tree or graph. Cite every claim about etymology to a dictionary or scholarly source.

13.3 Capstone Option B: Language Influence Study

Research question: Quantify the vocabulary that English borrowed from a specific source language during a defined historical period. What kinds of words were borrowed? What patterns characterize the loan vocabulary?

Suggested source languages: Norman French (post-1066), Old Norse (Danelaw), Classical Latin (Renaissance), Greek (scientific terminology).

Data sources: - The Oxford English Dictionary online (university library access) has dates of first attestation and source languages for over 600,000 words - Etymonline.com (free) for a substantial subset - The CELEX English lexical database (free academic license)

Methods to apply: 1. Borrowing detection classifier (Chapter 9) to identify likely loans 2. Feature analysis: what phonotactic features characterize this language's loans vs. others? 3. Semantic analysis: what semantic domains are overrepresented in the loan vocabulary? 4. Timeline visualization: when did borrowing peak?

Deliverable: A dataset of at least 500 classified words (inherited vs. borrowed from source language), with 5–8 visualizations and a 2,000-word analysis report.

13.4 Capstone Option C: Semantic Evolution Study

Research question: Choose 5–10 words and track how their meanings have changed over a period of at least 100 years using historical text corpora. Identify the type of change (pejoration, amelioration, broadening, narrowing, metaphorical extension) and estimate when it occurred.

Data sources: - Google Ngrams Viewer (books.google.com/ngrams) for frequency data - Corpus of Historical American English (COHA) — free access at corpus.byu.edu - Project Gutenberg for raw text by decade

Methods to apply: 1. Diachronic word embeddings (Chapter 8): train Word2Vec on text by decade 2. Orthogonal Procrustes alignment of embedding spaces 3. Cosine distance over time as a change measure 4. Nearest-neighbor analysis: what words cluster near the target word in each decade?

Deliverable: Embedding change trajectories for each target word, with nearest-neighbor tables by decade, and a 1,500-word narrative interpreting the results against known historical events or cultural shifts.

13.5 Capstone Option D: Language Relationship Discovery

Research question: Apply cognate detection and phylogenetic inference to a language family of your choice. Compare your computed tree to the expert consensus. Where do they agree and where do they disagree?

Suggested families (with available CLDF data): Austronesian, Bantu, Turkic, Dravidian, Sino-Tibetan.

Data sources: - CLLD (Cross-Linguistic Linked Data): https://clld.org — provides CLDF-format wordlists for many language families - LingPy ships with several built-in test datasets

Methods to apply: 1. Load a CLDF wordlist (Chapter 3) 2. Run LexStat cognate detection (Chapter 5) 3. Compute distance matrix from cognate proportions (Chapter 6) 4. Build UPGMA and Neighbor-Joining trees (Chapter 6) 5. Compare to expert tree using Robinson-Foulds distance 6. Identify specific subtrees where your tree disagrees with expert consensus and investigate why

Deliverable: Both computed trees + the expert reference tree, RF distance calculation, and a 2,000-word discussion of where and why the methods agree or disagree with expert consensus.

13.6 Capstone Option E: Build an Etymology Tool

Research question: Design and build a computational tool that solves one specific etymological problem better (in some measurable dimension) than existing solutions.

Suggested tools: - A web scraper + LLM extractor that builds a structured cognate dataset from Wiktionary entries for a specific language family - A streamlit app that lets users input a wordlist and receive an automatic phylogenetic tree with cognate clusters highlighted - A semantic drift tracker that accepts a word and returns its 5-year semantic trajectory from COHA data

Methods to apply: All of the above, as appropriate to your tool.

Deliverable: A working Python application + a benchmark evaluation showing its performance on a held-out test set + documentation sufficient for another student to run it. Open-source on GitHub.

13.7 Report Structure (All Options)

Regardless of option, your final report should follow this structure:

1. **Introduction** (200–400 words): What question are you answering and why does it matter?
2. **Data** (300–500 words): What data did you use, where did it come from, what are its known limitations?
3. **Methods** (400–600 words): What did you do, and why did you choose these methods over alternatives?
4. **Results** (600–1000 words): What did you find? Tables, figures, and specific numbers.
5. **Discussion** (400–600 words): What do your results mean? What would you do differently? What are the limits of your conclusions?
6. **Conclusion** (100–200 words): Three bullet points: what you found, what you would do next, one caveat.
7. **References**: Every claim about a language or etymology cited to a source.

A note on honest uncertainty: Computational etymology is a field where the tools are powerful and the data is imperfect. Your classifier may have 75% accuracy, not 99%. Your phylogenetic tree may disagree with experts on one branch. Your semantic drift signal may be noise. Report these results honestly. An honest null result or a modest success with clear limitations is more valuable than an overclaimed success that doesn't survive scrutiny.

The goal is not to solve etymology. The goal is to do something real with the methods, report it clearly, and learn where they work and where they do not.

13.8 Evaluation Rubric

Component	Weight	Criteria
Research question	10%	Clear, specific, genuinely open
Data documentation	15%	Sources cited, limitations stated
Methods	25%	Appropriate choice, correctly implemented
Results	25%	Accurate, clearly presented, numbers reported
Discussion	15%	Honest interpretation, limitations stated
Code quality	10%	Readable, reproducible, commented where non-obvious

Total: 100 points

The most common reason for low scores is overclaiming: asserting that a 75% F1 classifier "proves" a linguistic relationship, or interpreting a noisy semantic drift signal as definitive evidence of meaning change. Let the numbers speak for themselves.

13.9 Getting Started

Choose your option. Spend thirty minutes gathering data before you commit. If the data is unavailable, inaccessible, or thinner than expected, switch options before investing further. The data availability check is the most important step.

Then: write the Introduction first. Before you run a single line of code, write two paragraphs about what you are trying to find out and why. This forces you to have a hypothesis, which is the difference between analysis and fishing.

Good luck.

References

Linguistic Glossary

Alignment: The assignment of correspondence between phonemes in two words, used to identify which sounds in one word correspond to which sounds in another.

Amelioration: A type of semantic change in which a word's meaning becomes more positive over time. Example: "nice" (once "foolish") → pleasant.

Articulation, place of: The location in the vocal tract where a speech sound is produced (bilabial, alveolar, velar, etc.).

Articulation, manner of: The type of obstruction in the vocal tract that produces a sound (stop, fricative, nasal, approximant).

Attention mechanism: In neural networks, a mechanism that allows the decoder to selectively weight different parts of the encoder's output at each step of generation.

B-cubed metric: A precision/recall metric for clustering and alignment evaluation that measures how well automatically determined correspondences match gold-standard correspondences.

Bootstrap support: In phylogenetics, a measure of confidence for a tree branching, computed by re-sampling the data many times and checking how often the same branch appears.

Borrowing: The process by which words from one language are adopted into another. Distinguished from cognacy by direction: borrowed words move between contemporaneous languages; cognates descend from a common ancestor.

Character Error Rate (CER): The edit distance between a predicted string and a true string, normalized by the length of the true string. Used to evaluate proto-form reconstruction.

CLDF: Cross-Linguistic Data Formats — a standardized set of CSV-based formats for multilingual wordlists, cognate sets, and linguistic annotations.

Cognate: A pair (or set) of words in different languages that descend from the same ancestral word. Cognacy implies shared descent, not merely surface similarity.

Comparative method: The classical technique of historical linguistics: collecting cognate sets, identifying sound correspondences, and reconstructing proto-forms.

Contact zone: A geographic area where two or more languages are spoken in proximity, leading to mutual borrowing and sometimes structural convergence.

Distributional hypothesis: The principle that words occurring in similar contexts have similar meanings. Foundation of word embedding models.

Diachronic: Across time. Diachronic linguistics studies language change; synchronic linguistics studies language at a fixed point in time.

Edit distance (Levenshtein distance): The minimum number of insertions, deletions, and substitutions needed to transform one string into another.

Encoder-decoder: A neural network architecture in which one component (encoder) reads the input and another (decoder) generates the output, typically connected through an attention mechanism.

Feature vector: A numerical representation of a phoneme in terms of binary articulatory features (voiced/voiceless, stop/fricative, etc.).

Fine-tuning: The process of continuing to train a pre-trained neural network on a small domain-specific dataset to adapt it to a new task.

F1 score: The harmonic mean of precision and recall, used as a single summary metric for classification performance.

Glottocode: A unique identifier for a language variety in the Glottolog database, analogous to an ISBN for languages.

Grimm's Law: The systematic shift of stop consonants in Proto-Germanic relative to other Indo-European languages (PIE $p \rightarrow$ *Gmc* f; PIE $t \rightarrow$ *Gmc* þ; PIE $k \rightarrow$ *Gmc* h), formulated by Jakob Grimm in the 1820s.

IPA (International Phonetic Alphabet): A writing system designed to represent all sounds used in human language, with one symbol per sound.

Knowledge graph: A structured representation of entities and their relationships, represented as a graph of typed nodes and typed edges.

Language family: A group of languages that descend from a common ancestor (proto-language). Example: the Indo-European family (English, Latin, Sanskrit, and thousands more).

LexStat: An algorithm developed by Johann-Mattis List for automatic cognate detection in multilingual wordlists, using permutation-based alignment score normalization.

LingPy: An open-source Python library for computational historical linguistics, providing tools for tokenization, alignment, cognate detection, and phylogenetic distance computation.

Morphology: The study of word structure—how words are built from smaller meaningful units (morphemes): roots, prefixes, suffixes, etc.

Narrowing: A type of semantic change in which a word's meaning becomes more restricted. Example: "meat" (once "any food") $\rightarrow$ flesh of animals.

Needleman-Wunsch algorithm: A dynamic programming algorithm for global sequence alignment, originally developed for protein sequences and adapted for phonetic word alignment.

Neighbor-Joining: A phylogenetic tree construction algorithm that minimizes total tree length by iteratively joining the pair of taxa whose joining most reduces branch length sum.

Neogrammarians: A school of 19th-century German linguists who formulated the principle that sound changes are exceptionless and regular.

Pejoration: A type of semantic change in which a word's meaning becomes more negative. Example: "awful" (once "awe-inspiring") → very bad.

Phoneme: The smallest unit of sound that can distinguish word meaning in a language. English /p/ and /b/ are different phonemes because they distinguish "pat" from "bat."

Phonotactics: The rules governing which sequences of phonemes are permissible in a given language.

Phylogenetics: The field of science concerned with inferring evolutionary trees (phylogenies) from observed data. In linguistics, applied to language family trees.

Precision: In information retrieval, the proportion of retrieved items that are relevant. Precision = TP / (TP + FP).

Proto-language: A reconstructed ancestral language from which a set of descendant languages derive. Written with an asterisk: Proto-Indo-European, Proto-Germanic.

Recall: In information retrieval, the proportion of relevant items that are retrieved. Recall = TP / (TP + FN).

Reconstruction: The inference of the form of a proto-language word from its modern descendants, using the comparative method or computational models.

Reflex: A modern descendant of an ancestral form. Spanish "noche" is a reflex of Latin "noctem."

Reticulation: Non-tree-like patterns of language relationship, typically caused by borrowing and contact. Represented by phylogenetic networks rather than trees.

Robinson-Foulds distance: A metric for comparing two phylogenetic trees, counting the number of bipartitions (splits of the leaf set) that appear in one tree but not the other.

Semantic drift: The gradual change of a word's meaning over time. Also called semantic change or lexical semantic change.

Sound correspondence: A systematic relationship between sounds in corresponding cognate words in different languages. The *p/f* correspondence between Latin and Germanic is a sound correspondence.

Sprachbund: A geographic area in which unrelated or distantly related languages share structural features due to prolonged contact. The Balkan Sprachbund is the classic example.

Swadesh list: A set of 100 or 207 basic vocabulary items (body parts, numerals, kinship terms, etc.) selected for their cross-linguistic stability, used as a standard dataset for comparative work.

UPGMA: Unweighted Pair Group Method with Arithmetic Mean — a hierarchical clustering algorithm used for phylogenetic tree construction.

Word embedding: A dense, low-dimensional vector representation of a word, learned from co-occurrence patterns in large text corpora. Words with similar meanings have vectors that are geometrically close.

Word2Vec: A neural model for learning word embeddings, introduced by Mikolov et al. (2013), using either CBOW (predict word from context) or Skip-gram (predict context from word) training.

Tools & Resources

Python Libraries

LingPy (lingpy.org) Primary library for computational historical linguistics in Python. Provides IPA tokenization, phoneme feature extraction, sequence alignment (SCA algorithm), cognate detection (LexStat), and distance matrix computation.

```
pip install lingpy
```

Gensim (radimrehurek.com/gensim) Industry-standard library for Word2Vec, FastText, and GloVe embeddings. Handles large corpora efficiently.

```
conda install -c conda-forge gensim
```

NetworkX (networkx.org) Graph data structures and algorithms for Python. Used throughout Chapter 11 for building and querying etymology graphs.

```
conda install -c conda-forge networkx
```

scikit-learn (scikit-learn.org) Standard Python machine learning library. Used for cognate detection (Chapter 5), borrowing detection (Chapter 9), and all classification tasks.

```
conda install -c conda-forge scikit-learn
```

Hugging Face Transformers (huggingface.co/transformers) Library for pre-trained transformer models including FLAN-T5 for fine-tuning (Chapter 10).

```
pip install transformers
```

NLTK (nltk.org) General-purpose NLP library with useful tokenizers and corpus readers.

```
conda install -c conda-forge nltk
```

SciPy (scipy.org) Scientific computing library. Used for hierarchical clustering (UPGMA) and distance matrix operations.

```
conda install -c conda-forge scipy
```

Linguistics Software

EDICTOR (digling.org/edictor) Web-based tool for creating, editing, and annotating etymological wordlists in CLDF format. The best tool for building gold-standard alignment datasets.

SplitsTree (splitstree.org) Desktop software for phylogenetic network visualization, including NeighborNet for reticulate evolution. Free for academic use.

BEAST2 (beast2.org) Bayesian phylogenetic inference software with time-calibrated tree reconstruction. Used in the major computational historical linguistics papers on Indo-European dating.

PRAAT (fon.hum.uva.nl/praat) Standard tool for acoustic phonetics: recording, spectrograms, formant analysis. Required for working with actual speech data rather than transcribed text.

Data Sources

CLLD — Cross-Linguistic Linked Data (clld.org) Hub for CLDF-format linguistic databases. Key datasets: - WALS (World Atlas of Language Structures): typological features of 2,600 languages - Glottolog: genealogical classification of all documented languages - Concepticon: standardized concept list linking Swadesh lists across studies

Etymonline (etymonline.com) The Online Etymology Dictionary. Free, well-curated, with entries for 50,000+ English words. The best free source for English etymology.

Wiktionary (wiktionary.org) Crowdsourced multilingual dictionary with etymological sections. Quality varies but coverage is unmatched. Parseable via the Wiktionary API or dump files.

COHA — Corpus of Historical American English (corpus.byu.edu/coha) 400 million words of American English from 1810–2009, balanced by decade. Primary resource for diachronic embedding studies. Free access with registration.

Google Ngrams (books.google.com/ngrams) Frequency data for words and phrases across Google Books (1500–2019). Useful for spotting when words gained or lost usage, not a substitute for a full corpus.

ASJP (asjp.clld.org) Automated Similarity Judgments Program. 40-item wordlists for 6,000+ languages in a consistent orthographic format. Useful for broad-coverage studies.

IELex (ielex.mpi.nl) Indo-European Lexical Cognacy Database. 208-item Swadesh lists for ~50 Indo-European languages with expert cognate judgments.

Reference Books

Ladefoged, P., & Johnson, K. (2014). *A Course in Phonetics* (7th ed.). Cengage. The standard phonetics textbook. Clear, well-illustrated, and available in most university libraries.

Campbell, L. (2013). *Historical Linguistics: An Introduction* (3rd ed.). MIT Press. Comprehensive, authoritative, and readable. The reference for the comparative method.

Fortson, B. W. (2010). *Indo-European Language and Culture: An Introduction* (2nd ed.). Wiley-Blackwell. Best single-volume reference for Indo-European specifically.

Haspelmath, M., & Tadmor, U. (2009). *Loanwords in the World's Languages*. De Gruyter Mouton. Systematic cross-linguistic study of borrowing patterns.

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Available free at web.stanford.edu/~jurafsky/slp3/. The standard NLP textbook. Covers word embeddings, sequence models, and language processing at graduate level.

Key Papers by Chapter

Chapter 4 (Sequence Alignment) - Needleman & Wunsch (1970) — the original alignment algorithm - Kondrak (2000) — phonetic alignment specifically

Chapter 5 (Cognate Detection) - List (2012) — LexStat - Jäger, List & Sofroniev (2017) — SVM approach

Chapter 6 (Phylogenetics) - Saitou & Nei (1987) — Neighbor-Joining - Gray & Atkinson (2003) — Nature paper on Indo-European dating

Chapter 7 (Proto-reconstruction) - Ciobanu & Dinu (2018) — Ab Initio - Meloni et al. (2021) — Ab Antiquo (transformer approach)

Chapter 8 (Semantic Drift) - Hamilton, Leskovec & Jurafsky (2016) — diachronic embeddings

Chapter 9 (Borrowing) - Haspelmath & Tadmor (2009) — systematic borrowing survey

Chapter 10 (LLMs) - Brown et al. (2020) — GPT-3 few-shot learning - Chung et al. (2022) — FLAN-T5

Chapter 11 (Graphs) - Navigli & Ponzetto (2012) — BabelNet